

spintent

SPANISH PARSE INTENTS

v0.99 2026-09-08*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1 Descripción	1	3.3 El comando <code>\spmoney</code>	6
1.1 Carga del paquete	1	3.3.1 Llaves para <code>\spmoney</code>	7
1.2 El paquete <code>babel</code>	2	3.3.2 Monedas y alias para <code>\spmoney</code>	7
1.3 Fuentes tipográficas	2	3.4 El comando <code>\spshort</code>	7
1.4 El paquete <code>hyperref</code>	2	3.4.1 Llaves para <code>\spshort</code>	9
2 El comando <code>\setspintent</code>	3	3.4.2 Argumentos para <code>\spshort</code>	9
3 The big four	3	4 Utilidades generales	11
3.1 El comando <code>\spnum</code>	3	5 Utilidades para símbolos	11
3.1.1 Llaves para <code>\spnum</code>	3	6 Utilidades para números	13
3.2 El comando <code>\spunit</code>	4	7 Utilidades para geometría plana	17
3.2.1 Llaves para <code>\spunit</code>	4	8 Referencias	24
3.2.2 Argumentos para <code>\spunit</code>	4	9 Change history	25
3.2.3 El comando <code>\spnewunit</code>	6	10 Índice de la Documentación	26
3.2.4 El comando <code>\spunitalias</code>	6	11 Implementation	28
		12 Índice de la Implementación	126

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-09-08.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx  ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno valido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* valido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '10 00 00' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘;’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en *coma*, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en *punto*, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: *cdot*

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en *times* imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en *cdot* imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad)}</code>
<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>
<code>\spunit[key = val]{(número unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad*unidad)}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^2}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra* ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^2}` se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra* ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {true | false}`

default: *false*

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en *true*, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en *false* se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {true | false}`

default: *false*

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en *true*, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en *false*, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {por | partido-por}`

default: *por*

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra* ‘/’ de separación entre el numerador y el denominador. Si se establece en *por*, NVDA/MathCAT verbalizará «*por*»; si se establece en *partido-por*, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\nueva unidad}\{verbalización NVDA}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \textit{nueva unidad} \rangle\}$ que establece lo que se imprimirá en la salida PDF y $\{\langle \textit{verbalización NVDA} \rangle\}$ que establece como NVDA/MathCAT verbalizará la $\{\langle \textit{nueva unidad} \rangle\}$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\{\langle \text{nueva unidad} \rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle \text{unidad} \rangle\}$ válida para pasar como $\{\langle \text{argumento} \rangle\}$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\{\langle \text{nueva unidad} \rangle\}$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{*nuevo alias* \rangle}\}$ que establece es el nuevo $\{\langle \text{*alias* \rangle}\}$ de $\{\langle \text{*argumento* \rangle}\}$ para `\spunit` y $\{\langle \text{*expresión de unidades* \rangle}\}$ que puede contener multiplicación o división de $\{\langle \text{*unidades* \rangle}\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\langle nuevo alias \rangle}` es «global» y se almacena en el módulo `spintent`. lua como una `{\langle alias \rangle}` valido para pasar como `{\langle argumento \rangle}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3}/s` creará el `{\langle alias \rangle}` ‘caudal’ y cuando se utilice `$\spunit{50 caudal}$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<u>\spmoney</u>	\spmoney{ <i>cifra</i> }
	\spmoney{ <i>cifra moneda</i> }
	\spmoney[<i>key = val</i>]{ <i>cifra moneda</i> }

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{cifra} \rangle \}$ que contiene la «cifra numérica» positiva o negativa la se cual procesa internamente por el comando `\sprnum` al inicio del $\{ \langle \text{argumento} \rangle \}$ acompañada de un «símbolo de moneda» o «alias de moneda» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`<key = val>`] y se ejecuta `\$ \spmoney{500} \$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

- 🔗 El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dolares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: $\$ \spmone{500} \text{ CLP}$ imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}`

default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}`

default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {(true | false)}`

default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{(símbolos de moneda)}` y `{(alias de moneda)}` soportados como `{(argumento)}` para `\spmoney`.

3.4 El comando \spshort

`\spshort {(abreviaturas | acrónimos | números ordinales)}`
`\spshort[key = val] {(abreviaturas | acrónimos | números ordinales)}`
`\spshort[read-arg={verbalización del argumento para NVDA}] {(comandos LATEX)}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{(abreviaturas)}`, `{(acrónimos)}` o `{(números ordinales)}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr. a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{(números ordinales)}` se puede ejecutar `\spshort{1. a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1. o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{(abreviatura)}`, `{(acrónimo)}` o `{(número ordinal)}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{(comandos LATEX)}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{(números ordinales)}` se debe tener cuidado y no confundir la «*letra o voladita*» 'o' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1. o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₴	grivna, grivnas	grivna	₴	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = {⟨verbalización NVDA⟩}` default: no usado

Establece la verbalización del argumento {⟨comandos $\LaTeX$ ⟩} de la nueva {⟨abreviatura⟩}, {⟨acrónimo⟩} o {⟨número ordinal⟩} para NVDA. El valor de esta llave siempre debe pasarse entre '{ }'. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva {⟨abreviatura⟩} que imprimirá '8.^{avo}' en la salida PDF y NVDA verbalizará «octavo».

`small-caps = {⟨true | false⟩}` default: false

Establece si se usarán versalitas para imprimir {⟨acrónimos⟩} y para la «letra volada» en {⟨abreviaturas⟩} y {⟨números ordinales⟩} respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán versalitas. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá 'Dr.^a' en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el *argumento* `{<siglo>}`. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el *argumento* `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «delimitadores» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spusep{<separador>}` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «paréntesis izquierdo».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none> | <big> | <Big> | <bigg> | <Bigg>} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true> | <false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spdsym` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style> | <two-dots> | <slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido> | <dividido-por> | <dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmsym` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para `\spmsym`

<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «antes» de <i>verbalizar</i> el {⟨valor⟩} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>suffix = {⟨sufijo⟩}</code>	default: <i>no usado</i>
Establece el <i>sufijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «después» de <i>verbalizar</i> el {⟨valor⟩} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>set-sym = {⟨cross dot asterisk⟩}</code>	default: <i>dot</i>
Establece el <i>símbolo de multiplicación</i> que se imprimirá en la salida PDF. Si se establece en <code>cross</code> , imprimirá '×'; si se establece en <code>dot</code> , imprimirá '·' y si se establece en <code>asterisk</code> , imprimirá '*'.	
<code>read-sym = {⟨por multiplicado-por veces⟩}</code>	default: <i>por</i>
Establece la <i>forma</i> con la cual NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i> . Si se establece en <code>por</code> , NVDA/MathCAT verbalizará «por»; si se establece en <code>multiplicado-por</code> , NVDA/MathCAT verbalizará «multiplicado por» y si se establece en <code>veces</code> , NVDA/MathCAT verbalizará «veces».	
<code>not-print = {⟨true false⟩}</code>	default: <i>false</i>
Deshabilita la <i>impresión</i> del <i>símbolo de multiplicación</i> . Cuando se establece esta llave, NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i> acorde al valor establecido por la llave <code>read-sym</code> , pero, no se imprimirá este en la salida PDF.	

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

```
\spdiv {⟨dividendo⟩}{⟨divisor⟩}
\spdiv [⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}
```

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spdiv{20}{26}$` imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

<code>wrap-parens = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprimen</i> paréntesis redondos '(...)' alrededor de los <i>argumentos</i> {⟨dividendo⟩} y {⟨divisor⟩} pasados a <code>\spdiv</code> . Si se establece en <code>true</code> , se imprimirán paréntesis redondos alrededor de ambos <i>argumentos</i> ; si se establece en <code>false</code> , no se imprimirán los paréntesis.	
<code>read-parens = {⟨true false⟩}</code>	default: <i>true</i>
Establece si NVDA/MathCAT <i>verbalizará</i> los paréntesis redondos '(...)' alrededor de los <i>argumentos</i> {⟨dividendo⟩} y {⟨divisor⟩} pasados a <code>\spdiv</code> . Si se establece en <code>true</code> , NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en <code>false</code> , NVDA/MathCAT no los verbalizará.	

El comando `\spmul`

```
\spmul {⟨factor⟩}{⟨factor⟩}
\spmul [⟨key = val⟩]{⟨factor⟩}{⟨factor⟩}
```

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨factor⟩} y {⟨factor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spmul{20}{26}$` imprimirá '20 · 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

<code>wrap-parens = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprimen</i> paréntesis redondos '(...)' alrededor de los <i>argumentos</i> {⟨factor⟩} y {⟨factor⟩} pasados a <code>\spmul</code> . Si se establece en <code>true</code> , se imprimirán paréntesis redondos alrededor de ambos <i>argumentos</i> ; si se establece en <code>false</code> , no se imprimirán los paréntesis.	
<code>read-parens = {⟨true false⟩}</code>	default: <i>true</i>

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \textit{factor} \rangle \}$ y $\{ \langle \textit{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \textit{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *m*»; si se establece en `false`, el $(\langle \textit{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \textit{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc( (números separados por coma) )
\spmc[ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = { (true | false) }` default: false

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = { (terse | clear) }` default: clear

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { (true | false) }` default: false

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2, 3, 6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = { (true | false) }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc( (números separados por coma) )
\spmc[ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case` = { `<true>` | `<false>` } default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true` se imprimirá ‘MCM’ en la salida PDF; si se establece en `false` se imprimirá ‘mcm’ en la salida PDF.

`read-cmd` = { `<terse>` | `<clear>` } default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c m*»; si se establece en `clear`, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { `<true>` | `<false>` } default: `false`

Establece si se *imprime* el resultado de calcular el «mínimo común múltiplo» del (`<argumento>`) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis es igual a seis»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «mínimo común múltiplo».

`sample-arg` = { `<true>` | `<false>` } default: `false`

Desactiva la *validación* interna del (`<argumento>`) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «mínimo común múltiplo entre *m* y *n*»; si se establece en `false` el (`<argumento>`) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd` = { `<true>` | `<false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(<argumento>)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (`<argumento>`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ` `\spnZ{<número entero>}`
`\spnZ[<key = val>]{<número entero>}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {`<número entero>`} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* `cifras-sep` y `read-sign` pasadas al comando `\spnum` (§3.1.1).

`wrap-parens` = { `<true>` | `<false>` } default: `false`

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento {`<número entero>`} pasado a `\spnZ`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor del {`<número entero>`}; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens` = { `<true>` | `<false>` } default: `false`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* {`<número entero>`} pasado a `\spnZ`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` `\spmQ{<número entero>}{<numerador>}{<denominador>}`
`\spmQ[<key = val>]{<número entero>}{<numerador>}{<denominador>}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {`<número entero>`}, {`<numerador>`} y {`<denominador>`}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmQ{2}{5}{7}` imprimirá ‘2⁵/₇’ en la salida PDF y NVDA/MathCAT verbalizará «dos y cinco séptimos».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de **spintent**. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «tres más cinco séptimos», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: `y`

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá $1\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: `true`

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: `false`

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spmQ[print-dfrac=true]{2}{5}{7}` imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: `false`

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: `true`

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría plana

El paquete `spintent` provee comandos para trabajar con los objetos y medidas de la *geometría plana* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y comparten el mismo módulo `spintent.lua` para construir la verbalización correcta.

- Todos los comandos de esta sección aceptan como puntos solo *letras mayúsculas* como base, con modificadores opcionales de subíndice ‘`_{\dots}`’ o prima ‘`’`’. La única excepción es `\sprecta` cuando el *argumento* es de la forma `{⟨letra⟩}`, que acepta *letras minúsculas*. Los subíndices siempre deben escribirse de la forma correcta usando `_{\dots}` como `A_{\dots}` y nunca como `A_n` sin llaves que funciona solo por coincidencia.

El comando `\spPoint`

`\spPoint {⟨punto⟩}`

`\spPoint [⟨key = val⟩] {⟨punto⟩}`

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con un modificador opcional de subíndice ‘`_{\dots}`’ o prima ‘`’`’.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spPoint{A}` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

- El comando `\spPoint` solo acepta *un punto*. Para pares o listas de puntos (rectas, rayos, lados, ángulos, arcos, polígonos), véanse los comandos correspondientes de esta sección.

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}`

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`silent-cmd = {⟨true | false⟩}`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «punto». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sprec`

```
\sprec{<punto, punto>}          \sprec{<letra>}
\sprec[<key = val>]{<punto, punto>} \sprec[<key = val>]{<letra>}
```

El comando `\sprec` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<letra>}` o `{<punto, punto>}`. Si el *argumento* es de la forma `{<punto, punto>}` se aplica `\overleftarrow` en la impresión; si el *argumento* no contiene coma y es de la forma `{<letra>}` (mayúscula o minúscula), se imprimirá solo la `{<letra>}` sin aplicar `\overleftarrow`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sprec{A, B}` imprimirá $\overleftarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprec{m}` imprimirá m en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para `\sprec`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}` y el argumento `{<letra>}` cuando está en mayúscula. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprec{A, B}` se verbaliza «recta A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «recta». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «recta». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spray`

```
\spray{<punto, punto>}
\spray[<key = val>]{<punto, punto>}
```

El comando `\spray` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spray{A, B}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\spray`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spray{A, B}` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

```
read-sty = {<rayo | semirrecta>} default: rayo
```

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spray[read-sty=rayo]{A, B}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\spray[read-sty=semirrecta]{A, B}`, NVDA/MathCAT verbalizará «semirrecta A B».

- La impresión $\overrightarrow{AB}$ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

```
\spside{<punto, punto>}
\spside[<key = val>]{<punto, punto>}
```

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spside{A, B}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = {`<school | mathcat>`}

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto>`}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{A, B}` se verbaliza «*lado A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`}

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = {`<true | false>`}

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = {`<lado | segmento | trazo>`}

default: *lado*

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{A, B}`, NVDA/MathCAT verbalizará «*lado A B*»; si se utiliza `\spside[read-sty=segmento]{A, B}`, NVDA/MathCAT verbalizará «*segmento A B*» y si se utiliza `\spside[read-sty=trazo]{A, B}`, NVDA/MathCAT verbalizará «*trazo A B*».

`read-arg` = {`<verbalización NVDA>`}

default: *no usado*

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spside[read-arg={hipotenusa}]{A, B}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*hipotenusa A B*».

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`

`\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* {`<punto, punto>`}. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ' $\overline{AB}$ '; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmside{A, B}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*medida del lado A B*».

Llaves para `\spmside`

`read-cmd` = {`<school | mathcat>`}

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto>`}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{A, B}` se verbaliza «*medida del lado A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`}

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar «*medida del*» seguido por el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = {`<true | false>`}

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de «*medida del*» seguido por el término establecido por la llave `read-sty`. Si se establece en *true*, NVDA/MathCAT NO verbalizará «*medida del*» seguido por el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = {`<lado | segmento | trazo>`}

default: *lado*

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{A, B}`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `\spmside[read-sty=segmento]{A, B}`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `\spmside[read-sty=trazo]{A, B}`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`read-arg` = {`<verbalización NVDA>`}

default: *no usado*

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spmside[read-arg={medida cateto}]{A, B}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*medida cateto A B*».

`print-m` = {`<true | false>`}

default: *false*

Establece si se *imprime* la notación visual 'm' antes del argumento. Si se establece en *true*, `\spmside[print-m=true]{A, B}` imprimirá ' $m\overline{AB}$ ' en la salida PDF, si se establece en *false* imprimirá ' $\overline{AB}$ ' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

El comando `\spangle`

<code>\spangle</code>	<code>\spangle{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
	<code>\spangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\angle` o `\rightangle` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa» y `\spangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{A, O, B}` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\rightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `\spangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa» y `\spmangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

Llaves para `\spmangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{A, O, B}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B».

imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto $A O B$ »; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m = {\true | false}` default: `false`

Establece si se *imprime* la notación visual ‘m’ antes del argumento. Si se establece en `true`, `\spmangle[print-m=true]{A}`, imprimirá ‘ $m\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo $A O B$ ».

El comando `\spang`

```
\spang \spang{\langle grados \rangle}
\spang \spang{\langle grados : minutos \rangle}
\spang \spang{\langle grados : minutos : segundos \rangle}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\langle grados \rangle}`, `{\langle grados : minutos \rangle}` o `{\langle grados : minutos : segundos \rangle}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `\spang{45:30:15}` imprimirá ‘ $45^{\circ}30'15''$ ’ en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

```
\sparc \sparc{\langle punto, punto \rangle} \sparc{\langle punto, punto, punto \rangle}
\sparc \sparc[\langle key = val \rangle]{\langle punto, punto \rangle} \sparc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Representa un arco determinado por dos o tres puntos aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\sparc{A, B}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «arco $A B$ ».

Llaves para `\sparc`

`read-cmd = {\school | mathcat}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{A, B}` se verbaliza «arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {\true | false}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmarc`

```
\spmarc \spmarc{\langle punto, punto \rangle} \spmarc{\langle punto, punto, punto \rangle}
\spmarc \spmarc[\langle key = val \rangle]{\langle punto, punto \rangle} \spmarc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Representa la *medida* del arco determinado por dos o tres puntos. La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\spmarc{A, B}` imprimirá ‘ $m\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco $A B$ ».

Llaves para `\spmarc`

`read-cmd = {\school | mathcat}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{A, B}` se verbaliza «medida del arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {\true | false}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spPoly`

```
\spPoly {<punto, punto, punto, ...>}
\spPoly[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto, punto, ...>}`, una lista de al menos tres `{<puntos>}` separados por comas. El tipo de *símbolo* en la impresión y la verbalización se determinan acorde al número de puntos en el argumento: Si son tres puntos se aplica `\triangle` y añade el término «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhsquare` y añade el término «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spPoly{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\spPoly{A, B, C, D}` imprimirá ' $\square ABCD$ ' y verbalizará «cuadrado A B C D»; `\spPoly{A, B, C, D, E}` imprimirá ' $ABCDE$ ' y verbalizará «A B C D E».

Llaves para `\spPoly`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto, punto, ...>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{A, B, C}` se verbaliza «triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el argumento o los términos «triángulo» y «cuadrado» establecido por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

```
print-sym = {<true | false>} default: true
```

Establece si se usará `\triangle` o `\mdlgwhsquare` en la impresión añadiendo el término correspondiente a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spPoly[print-sym=true]{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; si se establece en `false`, imprimirá ' ABC ' en la salida PDF y NVDA/MathCAT omitirá el término «triángulo», verbalizando solo «A B C».

```
silent-cmd = {<true | false>} default: false
```

Alias de `print-sym=false`, incluido por consistencia con el resto de los comandos de geometría plana.

El comando `\spAfig`

```
\spAfig {<punto, punto, punto, ...>}
\spAfig(<punto, punto, punto, ...>)
\spAfig(<número | letra | letra_{número}>)
\spAfig[<key = val>](<punto, punto, punto, ...>)
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* `(<punto, punto, punto, ...>)` (al menos tres); un *número o letra sueltos* `(<1>)`, `(<n>)`; o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, `(<F>)`, `(<F_{1}>)`.

Si no se proporciona el *argumento opcional* `[<key = val>]` ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área». Con vértices, `\spAfig(A, B, C)` imprimirá ' $\mathcal{A}_{\triangle ABC}$ ' y verbalizará «área del triángulo A B C». Con un número o letra sueltos, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' y verbalizará «área uno»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' y verbalizará «área n». Con una letra de la tabla, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' y verbalizará «área de la figura»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' y verbalizará «área de la figura uno».

Llaves para `\spAfig`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

```
print-sym = {<auto | triangle | square | none>} default: auto
```

Establece si se usará `\triangle` o `\mdlgwhsquare` en la impresión. Con vértices, `auto` infiere el símbolo según el número de puntos (tres o cuatro); `none` lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «área A B C».

Forzar `triangle/square` exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, `auto` y `none` se comportan igual (sin símbolo); `triangle/square` anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\Delta_1}$ ' y verbalizará «área del triángulo uno». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = {\langle true | false \rangle}` default: *false*

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en `true`, `\spAfig[read-sub=true](1)` verbaliza «área sub uno»; si se establece en `false`, verbaliza «área uno». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = {\langle true | false \rangle}` default: *false*

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {\langle nombre \rangle}` default: *no usado*

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapezio A B C D».

El comando `\spPfig`

`\spPfig` `\spPfig`
`\spPfig(\langle punto, punto, punto, ... \rangle)`
`\spPfig(\langle número | letra | letra_{\langle número \rangle} \rangle)`
`\spPfig[\langle key = val \rangle](\langle punto, punto, punto, ... \rangle)`

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`key = val`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro». Con vértices, `\spPfig(A, B, C)` imprimirá ' $\mathcal{P}_{\Delta ABC}$ ' y verbalizará «perímetro del triángulo A B C». Con un número o letra sueltos, `\spPfig(2)` imprimirá ' $\mathcal{P}_2$ ' y verbalizará «perímetro dos». Con una letra de la tabla, `\spPfig(P_{\{2\}})` imprimirá ' $\mathcal{P}_{P_2}$ ' y verbalizará «perímetro del polígono dos».

Llaves para `\spPfig`

`read-cmd = {\langle school | mathcat \rangle}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «perímetro del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {\langle auto | triangle | square | none \rangle}` default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

`read-sub = {\langle true | false \rangle}` default: *false*

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

`silent-cmd = {\langle true | false \rangle}` default: *false*

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {\langle nombre \rangle}` default: *no usado*

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapezio A B C D».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-09-08 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C		Keys for \spdiv provide by spintent :	
Document class:		read-parens	13
article	1	wrap-parens	13
book	1	Keys for \spdsym provide by spintent :	
letter	1	prefix	12
report	1	read-sym	12
Commands provide by spintent :		set-sym	12
\setspintent	3	suffix	12
\spAfig	22	Keys for \spmangle provide by spintent :	
\spPfig	23	prefix	20
\spPoint	17	print-m	21
\spPoly	22	read-cmd	20
\spangle	20	recto	20
\spang	21	silent-cmd	20
\sparc	21	Keys for \spmarc provide by spintent :	
\spdate	11	prefix	21
\spdiv	13	read-cmd	21
\spdsym	12	silent-cmd	21
\spmQ	16	Keys for \spmcd provide by spintent :	
\spmangle	20	prefix	15
\spmarc	21	read-cmd	15
\spmcd	15	result	15
\spmcm	15	sample-arg	15
\spmoney	6	silent-cmd	15
\spmside	19	upper-case	15
\spmsym	12	Keys for \spmcm provide by spintent :	
\spmul	13	prefix	16
\spnD	14	read-cmd	16
\spnM	14	result	16
\spnZ	16	sample-arg	16
\spnewunit	6	silent-cmd	16
\spnum	3	upper-case	16
\sprayo	18	Keys for \spmoney provide by spintent :	
\spread	11	currency	7
\sprecta	18	dolar-math	7
\spshort	7	sub-units	7
\spside	18	Keys for \spmQ provide by spintent :	
\spsiglo	11	join-word	17
\sptime	11	print-dfrac	17
\spunitalias	6	read-parens	17
\spunit	4	use-spnZ	17
\spusep	12	wrap-parens	17
K		Keys for \spmside provide by spintent :	
Keys for \spAfig provide by spintent :		prefix	19
prefix	22	print-m	19
print-sym	22	read-arg	19
read-arg	23	read-cmd	19
read-cmd	22	read-sty	19
read-sub	23	silent-cmd	19
silent-cmd	23	Keys for \spmsym provide by spintent :	
Keys for \spangle provide by spintent :		not-print	13
prefix	20	prefix	13
read-cmd	20	read-sym	13
recto	20	set-sym	13
silent-cmd	20	suffix	13
Keys for \sparc provide by spintent :		Keys for \spmul provide by spintent :	
prefix	21	read-parens	13
read-cmd	21	wrap-parens	13
silent-cmd	21	Keys for \spnD provide by spintent :	
		prefix	14
		result-num	14

result	14	notation-sym	13
sample-arg	14	read-period-sep	13
silent-cmd	14	read-sign	13
Keys for \spnM provide by spintent :		read-sym	13
prefix	14	set-sym	13
result-num	14	Keys for \spdsym provide by spintent :	
result	14	read-sym	12
sample-arg	14	Keys for \spmcd provide by spintent :	
silent-cmd	15	read-cmd	15
Keys for \spnum provide by spintent :		Keys for \spmcm provide by spintent :	
cifras-sep	3	read-cmd	16
notation-sym	4	Keys for \spmoney provide by spintent :	
read-period-sep	3	cifras-sep	7
read-sign	3	Keys for \spmQ provide by spintent :	
Keys for \spnZ provide by spintent :		wrap-parens	17
read-parens	16	Keys for \spmsym provide by spintent :	
wrap-parens	16	read-sym	13
Keys for \spPfig provide by spintent :		Keys for \spmul provide by spintent :	
prefix	23	cifras-sep	13
print-sym	23	notation-sym	13
read-arg	23	read-period-sep	13
read-cmd	23	read-sign	13
read-sub	23	read-sym	13
silent-cmd	23	set-sym	13
Keys for \spPoint provide by spintent :		Keys for \spnZ provide by spintent :	
prefix	17	cifras-sep	16
read-cmd	17	read-sign	16
silent-cmd	17	Keys for \spPoly provide by spintent :	
Keys for \spPoly provide by spintent :		print-sym	22
prefix	22	Keys for \spside provide by spintent :	
print-sym	22	read-sty	19
read-cmd	22	Keys for \spunit provide by spintent :	
silent-cmd	22	cifras-sep	4
Keys for \sprayo provide by spintent :		notation-sym	4
prefix	18	read-period-sep	4
read-cmd	18	Keys for \spusep provide by spintent :	
read-sty	18	silent	12
silent-cmd	18	size	12
Keys for \sprecta provide by spintent :			
prefix	18		
read-cmd	18		
silent-cmd	18		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
prefix	19		
read-arg	19		
read-cmd	19		
read-sty	19		
silent-cmd	19		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
print-cdot	4		
read-cdot	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		
Keys for \spddiv provide by spintent :			
cifras-sep	13		

M

Lua module provide by **spintent**:

spintent.lua	6, 7, 9, 10, 17
--------------------	-----------------

P

Packages :

babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and we will add the only key `mathcal` that can be passed as a package option.

```
13 \ProvidesExplPackage {spintent} {2026-09-08} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n = true,
19   mathcal .value_required:n = true,
20 }
21 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_T_EX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
22 \IfDocumentMetadataTF
23 {
24   \sys_if_engine luatex:TF
25   {
26     \msg_new:nnn { spintent } { package-not-load }
27     {
28       The~'#1'~package~will~be~loaded~as~a~dependency.
29     }
30     \IfPackageLoadedF { unicode-math }
31     {
32       \IfPackageLoadedF { lua-unicode-math }
33       {
34         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
35         \RequirePackage{unicode-math}
36       }
37     }
38     \msg_new:nnn { spintent } { env-success }
39     { Everything~looks~correct,~spintent~will~make~the~intent! }
```

```

40     \msg_info:nn { spintent } { env-success }
41   }
42   {
43     \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
44     { The~spintent~package~requires~LuaLaTeX. }
45     { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
46     \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
47   }
48 }
49 {
50   \msg_new:nnnn { spintent } { fatal-metadata-missing }
51   { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
52   {
53     The~spintent~package~requires~tagged~PDF~active.\\
54     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
55   }
56   \msg_fatal:nn { spintent } { fatal-metadata-missing }
57 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

```

Non-standard kernel variants.

```

58 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
59 \cs_generate_variant:Nn \str_if_eq:nnF { V }
60 \cs_generate_variant:Nn \str_uppercase:n { V }
61 \cs_generate_variant:Nn \str_lowercase:n { V }
62 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal` and `\rightanglesqr`

First, we declare the variables `\l__spintent_luaset_cal_fam_ok_str` and `\l__spintent_luaset_sym_fam_ok_str` before loading the module `spintent.lua`. This is necessary because of the implementation of the functions `\__spintent_luafun_classic_mathcal:nn` and `\__spintent_luafun_right_angle_sqr:` within the Lua module; otherwise, `expl3` will complain about duplicate variable declarations.

```

63 \str_new:N \l__spintent_luaset_cal_fam_ok_str
64 \str_new:N \l__spintent_luaset_sym_fam_ok_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str` and `\l__spintent_luaset_sym_fam_ok_str`.)

11.6 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:V` used by `\spunit` (§11.15), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.17), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.30) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.14) defined in it.

```

65 \lua_load_module:n { spintent.lua }
66 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
67 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
68 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
69 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdfTeX for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

```

mathcal-fallback
right-angle-sqr-fallback

```

First, we'll define two error messages in case our fallback fails.

```

70 \hook_gput_code:nnn { begindocument } { spintent }
71   {
72     \__spintent_luafun_init_math_fallbacks:
73   }
74 \msg_new:nnn { spintent } { mathcal-fallback }
75   {
76     Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~(font~not~found~or~
77     no~free~math~family~available).~\msg_line_context:
78   }
79 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
80   {
81     Using~\token_to_str:N \rightangle \c_space_tl as~fallback~for~the~right-angle~

```

```

82     glyph.~\msg_line_context:
83   }

```

(End of definition for *mathcal-fallback* and *right-angle-sqr-fallback*.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

84 \cs_new_protected:Nn \__spintent_mathcal:n
85   {
86     \bool_if:NTF \__spintent_mathcal_bool
87     {
88       \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
89       { \__spintent_luafun_classic_mathcal:n {#1} }
90       {
91         \msg_warning:nn { spintent } { mathcal-fallback }
92         \mathcal {#1}
93       }
94     }
95     { \mathcal {#1} }
96   }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

97 \cs_new_protected:Nn \__spintent_right_angle_sqr:
98   {
99     \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
100     { \__spintent_luafun_right_angle_sqr: }
101     {
102       \msg_warning:nn { spintent } { right-angle-sqr-fallback }
103       \rightangle
104     }
105   }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Internal copy of the unicode invisible separator

`\__spintent_invisible_sep:`
`\c__spintent_invisible_sep_tl`

The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml[2]` and let `<mo>...</mo>` in the MathML structure embedded in the PDF output.

```

106 \cs_new_protected:Nn \__spintent_invisible_sep:
107   {
108     \__spintent_luafun_invisible_sep:
109   }

```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from LuaTeX which leaves `<mi>...</mi>` in the structure MathML embedded in the output PDF.

```

110 \hook_gput_code:nnn { begindocument } { spintent }
111   {
112     \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
113   }

```

- Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure MathML, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure MathML when we use `\MathMLintent{<text>}`.

(End of definition for `\__spintent_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.9 Normalize argument and affixes

We normalize the `{⟨argument⟩}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

114 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
115   {
116     \tl_if_in:NnT #1 { -- }
117     {
118       \tl_replace_all:Nnn #1 { -- } { - }
119       \__spintent_collapse_hyphens:N #1
120     }
121   }

```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the `{⟨argument⟩}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘ ’ to hyphens ‘-’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

122 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
123   {
124     \tl_if_blank:nTF {#2}
125     { \tl_clear:N #1 }
126     {
127       \tl_set:Ne #1 {#2}
128       \tl_trim_spaces:N #1
129       \tl_replace_all:Nnn #1 { ~ } { - }
130       \__spintent_collapse_hyphens:N #1
131       \tl_put_left:Nn #1 { \q_mark }
132       \tl_put_right:Nn #1 { \q_stop }
133       \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
134       \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
135       \tl_remove_all:Nn #1 { \q_mark }
136       \tl_remove_all:Nn #1 { \q_stop }
137     }
138   }
139 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }

```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure `⟨keys⟩` “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__spintent_setspintent_input_arg_cmd_tl` The variable `\l__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” `{⟨argument⟩}` ‘#1’ of the command `\setspintent`.

```
140 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the `{⟨argument⟩}` ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

141 \msg_new:nnnn { spintent } { unknown-command-setup }
142   {
143     Cannot~configure~unknown~command~'\exp_not:c{#1}'.
144   }
145   {
146     You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
147     command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
148   }

```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

__spintent_setspintent_set_keys:nn
__spintent_setspintent_set_keys:en

```

The function `__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

149 \cs_new_protected:Nn __spintent_setspintent_set_keys:nn
150 {
151   \cs_if_exist:cTF { #1 }
152   {
153     \keys_set:nn { spintent / #1 } { #2 }
154   }
155   {
156     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
157   }
158 }
159 \cs_generate_variant:Nn __spintent_setspintent_set_keys:nn { en }

```

(End of definition for `__spintent_setspintent_set_keys:nn`.)

```

__spintent_setspintent_parse_args:nn
__spintent_setspintent_parse_args:Vn

```

The function `__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `__spintent_setspintent_set_keys:nn`.

```

160 \cs_new_protected:Nn __spintent_setspintent_parse_args:nn
161 {
162   \tl_if_single:nTF { #1 }
163   {
164     \token_if_cs:NTF #1
165     {
166       __spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
167     }
168     {
169       __spintent_setspintent_set_keys:nn { #1 } { #2 }
170     }
171   }
172   {
173     __spintent_setspintent_set_keys:nn { #1 } { #2 }
174   }
175 }
176 \cs_generate_variant:Nn __spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```

177 \NewDocumentCommand \setspintent { m m }
178 {
179   \tl_if_blank:nTF { #1 }
180   {
181     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
182   }
183   {
184     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
185     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
186     __spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
187   }
188 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

```

We define the variable `\l__spintent_command_name_str` which will store the “name” of each command within the implementation along with the messages `unknown-choice` used by `⟨keys⟩` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `⟨key⟩` `unknown` in commands that use `[⟨key = val⟩]`.

```

189 \str_new:N \l__spintent_command_name_str
190 \msg_new:nnn { spintent } { unknown-choice }
191 {
192   The~value~'~#3'~for~'~#1'~key~is~invalid~use~('~#2').
193 }
194 \msg_new:nnnn { spintent } { cmd-key-unknown }
195 {
196   The~key~'~#1'~is~unknown~by~command~

```

```

197     \c_backslash_str \l__spintent_command_name_str \c_space_tl
198     and~is~being~ignored~\msg_line_context:.
199   }
200   {
201     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
202     does~not~have~a~key~called~'#1'.\\
203     Check~that~you~have~spelled~the~key~name~correctly.
204   }
205 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
206 {
207   The~key~'#1=#2'~is~unknown~by~command~
208   \c_backslash_str \l__spintent_command_name_str \c_space_tl
209   and~is~being~ignored~\msg_line_context:.
210 }
211 {
212   The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
213   does~not~have~a~key~called~'#1'.\\
214   Check~that~you~have~spelled~the~key~name~correctly.
215 }

```

(End of definition for `\l__spintent_command_name_str` and others.)

```

\__spintent_unknown_keys:n
\__spintent_unknown_keys:nn

```

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the *<key>* unknown.

```

216 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
217 {
218   \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
219 }
220 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
221 {
222   \tl_if_blank:nTF {#2}
223   { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
224   { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
225 }

```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn`.)

11.12 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the *<argument>* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str

```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

226 \tl_new:N \l__spintent_luaset_sign_tl
227 \tl_new:N \l__spintent_luaset_part_int_tl
228 \tl_new:N \l__spintent_luaset_dec_sep_tl
229 \tl_new:N \l__spintent_luaset_part_dec_tl
230 \tl_new:N \l__spintent_luaset_part_period_tl
231 \str_new:N \l__spintent_luaset_has_integer_str
232 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

233 \str_new:N \l__spintent_luaset_only_part_int_str
234 \str_new:N \l__spintent_luaset_only_part_dec_str
235 \str_new:N \l__spintent_luaset_only_part_period_str
236 \str_new:N \l__spintent_luaset_format_part_int_str
237 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

238 \str_new:N \l__spintent_luaset_millions_str
239 \str_new:N \l__spintent_luaset_decimal_period_str
240 \str_new:N \l__spintent_luaset_not_decimal_period_str
241 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
242 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
243 \str_new:N \l__spintent_luaset_has_exponent_str
244 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

245 \str_new:N \l__spintent_luaset_has_units_str
246 \str_new:N \l__spintent_luaset_status_str
247 \tl_new:N \l__spintent_luaset_arg_above_tl
248 \tl_new:N \l__spintent_luaset_arg_below_tl
249 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

250 \msg_new:nnnn { spintent } { need-math-mode }
251 {
252   Math~mode~required~for~'#1'~\msg_line_context:.
253 }
254 {
255   Use~inside~$...$~or~in~math~environment.
256 }
257 \msg_new:nnnn { spintent } { need-text-mode }
258 {
259   Text~mode~required~for~'#1'~\msg_line_context:.
260 }
261 {
262   Use~outside~of~$...$~or~math~environment.
263 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the *intent functions* passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the *intent* passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the *intent* to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the *intent* function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the *intent* function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```
264 \bool_new:N \l__spintent_spnum_read_terse_bool
265 \str_new:N \l__spintent_spnum_read_period_sep_str
266 \tl_new:N \l__spintent_spnum_notation_sym_tl
267 \str_new:N \l__spintent_spnum_intent_read_sign_str
268 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

```
cifras-sep  Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.
read-sign
read-period-sep
notation-sym

269 \keys_define:nn { spintent/spnum }
270 {
271   cifras-sep                .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
272   cifras-sep                .initial:n = true,
273   cifras-sep                .value_required:n = true,
274   read-sign                 .choices:nn =
275     { terse , clear }
276     {
277       \int_case:nn { \l_keys_choice_int }
278       {
279         {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
280         {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
281       }
282     },
283   read-sign                 .initial:n = terse,
284   read-sign                 .value_required:n = true,
285   read-sign / unknown       .code:n =
286     \msg_error:nneee { spintent } { unknown-choice }
287     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
288   read-period-sep           .choices:nn =
289     { coma , punto }
290     {
291       \int_case:nn { \l_keys_choice_int }
292       {
293         {1} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { coma } }
294         {2} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { punto } }
295       }
296     },
297   read-period-sep           .initial:n = coma,
298   read-period-sep           .value_required:n = true,
299   read-period-sep / unknown .code:n =
300     \msg_error:nneee { spintent } { unknown-choice }
301     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
302   notation-sym              .choices:nn =
303     { times , cdot }
```

```

304     {
305         \int_case:nn { \l_keys_choice_int }
306         {
307             {1} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \times } }
308             {2} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \cdot } }
309         }
310     },
311     notation-sym .initial:n = cdot,
312     notation-sym .value_required:n = true,
313     notation-sym / unknown .code:n =
314     \msg_error:nneee { spintent } { unknown-choice }
315     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
316     unknown .code:n =
317     {
318         \str_set:Nn \__spintent_command_name_str { snum }
319         \__spintent_unknown_keys_cmd:n {#1}
320     },
321 }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

```

\__spintent_snum_render_part:n
\__spintent_snum_render_int:
\__spintent_snum_render_dec:

```

The function `\__spintent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_snum_render_int:` and `\__spintent_snum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintenc`. It evaluates whether the key *cifras-sep* is active or not.

```

322 \cs_new_protected:Nn \__spintent_snum_render_part:n
323 {
324     \MathMLintenc { \str_use:c { l__spintenc_luaset_only_part_#1_str } :number }
325     {
326         \bool_if:NTF \__spintenc_snum_cifras_sep_bool
327         { \str_use:c { l__spintenc_luaset_format_part_#1_str } }
328         { \tl_use:c { l__spintenc_luaset_part_#1_tl } }
329     }
330 }
331 \cs_new_protected:Nn \__spintenc_snum_render_int:
332 {
333     \__spintenc_snum_render_part:n { int }
334 }
335 \cs_new_protected:Nn \__spintenc_snum_render_dec:
336 {
337     \__spintenc_snum_render_part:n { dec }
338 }

```

(End of definition for `\__spintenc_snum_render_part:n`, `\__spintenc_snum_render_int:`, and `\__spintenc_snum_render_dec:`.)

```

\__spintenc_snum_render_only_period:
\__spintenc_snum_print_period_bar:
\__spintenc_snum_render_period_sep:

```

The function `\__spintenc_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintenc` which will be used by `\MathMLarg` in the function `\__spintenc_snum_print_period_bar:`.

```

339 \cs_new_protected:Nn \__spintenc_snum_render_only_period:
340 {
341     \MathMLintenc { \l__spintenc_luaset_only_part_period_str :number }
342     {
343         \l__spintenc_luaset_part_period_tl
344     }
345 }

```

The function `\__spintenc_snum_print_period_bar:` inserts the function `\__spintenc_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintenc`, generating the correct spoken for the “infinite periodic decimal part”.

```

346 \cs_new_protected:Nn \__spintenc_snum_print_period_bar:
347 {
348     \MathMLintenc { _periódico:postfix($dp) }
349     {
350         {
351             \overline
352             {
353                 \MathMLarg { dp } { \__spintenc_snum_render_only_period: }
354             }
355         }
356     }

```

```
357 }
```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```
358 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
359 {
360   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
361   {
362     \MathMLintent { _coma:prefix($pp) }
363     {
364       \mathord{,}
365       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
366     }
367   }
368   {
369     \MathMLintent { _punto:prefix($pp) }
370     {
371       \mathord{.}
372       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
373     }
374   }
375 }
```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

```
\__spintent_snum_render_decimal_sep:
```

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```
376 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
377 {
378   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
379   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
380   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
381   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
382   {
383     \mathord { \l__spintent_luaset_dec_sep_tl }
384     \MathMLarg { dp }
385     {
386       \__spintent_invisible_sep:
387       \__spintent_snum_render_dec:
388     }
389   }
390   % period part
391   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
392   {
393     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
394     {
395       \__spintent_invisible_sep:
396       \__spintent_snum_print_period_bar:
397     }
398     {
399       \MathMLintent { _con:prefix($pnum) }
400       {
401         \__spintent_invisible_sep:
402         \MathMLarg { pnum }
403         {
404           \__spintent_snum_print_period_bar:
405         }
406       }
407     }
408   }
409 }
```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

```
\__spintent_snum_print_part_dec:
\__spintent_snum_print_part_period:
```

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”).

It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

410 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
411 {
412   \tl_if_empty:NF \__spintent_luaset_part_dec_tl
413   {
414     \__spintent_snum_render_decimal_sep:
415   }
416 }
417 \cs_new_protected:Nn \__spintent_snum_print_part_period:
418 {
419   \tl_if_empty:NF \__spintent_luaset_part_period_tl
420   {
421     \str_if_eq:VnTF \__spintent_luaset_not_decimal_period_str { true }
422     { \__spintent_snum_render_period_sep: }
423     {
424       \tl_if_empty:NT \__spintent_luaset_part_dec_tl
425       {
426         \__spintent_snum_render_period_sep:
427       }
428     }
429   }
430 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

431 \cs_new_protected:Nn \__spintent_snum_print_integer:
432 {
433   \str_if_eq:VnTF \__spintent_luaset_not_decimal_not_period_str { true }
434   { \__spintent_snum_render_int: }
435   {
436     \MathMLintent { \__spintent_luaset_only_part_int_str :number:prefix($next) }
437     {
438       \bool_if:NTF \__spintent_snum_cifras_sep_bool
439       { \__spintent_luaset_format_part_int_str }
440       { \__spintent_luaset_part_int_tl }
441       \MathMLarg { next }
442       {
443         \tl_if_empty:NTF \__spintent_luaset_part_dec_tl
444         { \__spintent_snum_print_part_period: }
445         { \__spintent_snum_print_part_dec: }
446       }
447     }
448   }
449 }
450 \cs_new_protected:Nn \__spintent_snum_print_num_start:
451 {
452   \tl_if_empty:NTF \__spintent_luaset_sign_tl
453   {
454     \__spintent_invisible_sep: % need
455     \__spintent_snum_print_integer:
456   }
457   {
458     \bool_if:NTF \__spintent_snum_read_terse_bool
459     {
460       \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }

```

```

461         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _más:prefix($n)} }
462         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _menos:prefix($n)} }
463     }
464     {
465         \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
466         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _positivo:postfix($n) } }
467         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _negativo:postfix($n) } }
468     }
469     \__spintent_invisible_sep: % need
470     \MathMLintent { \__spintent_snum_intent_read_sign_str }
471     {
472         \__spintent_luaset_sign_tl
473         \MathMLarg { n } { \__spintent_snum_print_integer: }
474     }
475 }
476 }
477 \cs_new_protected:Nn \__spintent_snum_print_number_core:
478 {
479     \__spintent_snum_print_num_start:
480     \str_if_eq:VnTF \__spintent_luaset_has_exponent_str { true }
481     {
482         \MathMLintent { _por } { \__spintent_snum_notation_sym_tl }
483         10^{\__spintent_luaset_exponent_tl }
484     }
485 }

```

(End of definition for `\__spintent_snum_print_integer:`, `\__spintent_snum_print_num_start:`, and `\__spintent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

486 \msg_new:nnnn { spintent } { spnum-has-units }
487 {
488     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(''#1'').
489 }
490 {
491     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
492 }
493 \msg_new:nnnn { spintent } { spnum-no-digits }
494 {
495     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
496 }
497 {
498     You~provided~''#1',~which~contains~no~digits.
499 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational\ number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_snum_print_number_core:` to render the final number, and finally closes the group.

```

500 \NewDocumentCommand \spnum { O{} m }
501 {
502     \mode_if_math:TF
503     {
504         \group_begin:
505         \keys_set:nn { spintent/spnum } { #1 }
506         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
507         \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
508         { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
509         {
510             \tl_if_empty:NTF \__spintent_luaset_part_int_tl
511             { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
512             { \__spintent_snum_print_number_core: }

```

```

513     }
514     \group_end:
515 }
516 { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
517 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

```

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

518 \str_new:N \__spintent_spunit_luaset_canonical_str
519 \str_new:N \__spintent_spunit_luaset_lookup_status_str
520 \str_new:N \__spintent_spunit_luaset_read_str
521 \str_new:N \__spintent_spunit_luaset_is_compact_str
522 \str_new:N \__spintent_spunit_luaset_compact_exp_str
523 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
524 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

```

\__spintent_spunit_exponent_tl
\__spintent_spunit_unit_name_tl
\__spintent_spunit_read_slash_str
\__spintent_spunit_exp_seq
\__spintent_spunit_mult_seq
\__spintent_spunit_is_mult_bool
\__spintent_spunit_read_cdot_bool

```

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

525 \tl_new:N \__spintent_spunit_exponent_tl
526 \tl_new:N \__spintent_spunit_unit_name_tl
527 \str_new:N \__spintent_spunit_read_slash_str
528 \seq_new:N \__spintent_spunit_exp_seq
529 \seq_new:N \__spintent_spunit_mult_seq
530 \bool_new:N \__spintent_spunit_is_mult_bool
531 \bool_new:N \__spintent_spunit_read_cdot_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

```

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle argument \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

532 \msg_new:nnnn { spintent } { spunit-invalid-unit }
533 {
534   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
535 }
536 {
537   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
538 }
539 \msg_new:nnnn { spintent } { spunit-no-units }
540 {
541   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
542 }
543 {
544   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
545 }
546 \msg_new:nnnn { spintent } { inline-numeric-denominator }
547 {
548   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
549 }
550 {
551   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
552   does~not~allow~numbers~after~the~slash~(/).
553 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

554 \keys_define:nn { spintent/spunit }
555 {
556   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
557   print-cdot      .initial:n = false,
558   print-cdot      .value_required:n = true,
559   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
560   read-cdot       .initial:n = false,
561   read-cdot       .value_required:n = true,
562   read-slash      .choices:nn =
563     { por , partido-por }
564   {
565     \int_case:nn { \l_keys_choice_int }
566     {
567       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
568       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
569     }
570   },
571   read-slash      .initial:n = por,
572   read-slash      .value_required:n = true,
573   read-slash / unknown .code:n =
574     \msg_error:nneee { spintent } { unknown-choice }
575     { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
576   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
577   cifras-sep      .value_required:n = true,
578   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
579   read-period-sep .value_required:n = true,
580   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
581   notation-sym    .value_required:n = true,
582   unknown         .code:n =
583     {
584       \str_set:Nn \l__spintent_command_name_str { spunit }
585       \__spintent_unknown_keys_cmd:n {#1}
586     },
587 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “*unit symbol*”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “*units*” in standard upright text using `\mathrm`.

The function `\__spintent_spunit_process_base_exp:n` evaluates a “*single unit*” along with its exponent. It first looks up the unit ‘*#1*’ in the dictionary. If the “*unit*” is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot ‘`\cdot`’ intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the ‘`^`’ symbol).

```

588 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
589 {
590   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
591   {
592     { ℓ } { \ell }
593   }
594   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
595 }
596 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
597 {
598   \tl_clear:N \l__spintent_spunit_exponent_tl
599   \__spintent_luafun_spunit_lookup_alias:n { #1 }
600   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
601   {
602     \str_set:Nn \l__spintent_luaset_status_str { invalid }
603   }
604   {
605     \bool_if:NF \l__spintent_spunit_is_mult_bool
606     {
607       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
608       {
609         \bool_if:NTF \l__spintent_spunit_read_cdot_bool

```

```

610         { \MathMLintent { por } { \cdot } }
611         { \MathMLintent { :silent } { \cdot } }
612     }
613     {
614         \bool_if:NTF \l__spintent_spunit_read_cdott_bool
615         { \MathMLintent { por } { \, } }
616         { \, }
617     }
618 }
619 \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
620 {
621     {
622         \MathMLintent { \l__spintent_spunit_luaset_read_str }
623         {
624             \__spintent_spunit_format_canonical:
625         }
626     }
627     ^ { \l__spintent_spunit_luaset_compact_exp_str }
628 }
629 {
630     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
631     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
632     \seq_if_empty:NF \l__spintent_spunit_exp_seq
633     {
634         \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
635     }
636     \MathMLintent { \l__spintent_spunit_luaset_read_str }
637     {
638         \__spintent_spunit_format_canonical:
639     }
640     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
641     {
642         ^ { \l__spintent_spunit_exponent_tl }
643     }
644 }
645 \bool_set_false:N \l__spintent_spunit_is_mult_bool
646 }
647 }

```

(End of definition for `\__spintent_spunit_format_canonical:` and `\__spintent_spunit_process_base_exp:n`.)

`\__spintent_spunit_process_mult:n`
`\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the `{\argument}` at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

648 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
649 {
650     \bool_set_true:N \l__spintent_spunit_is_mult_bool
651     \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
652     \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
653 }
654 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

655 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
656 {
657     \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
658     {
659         \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
660         \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
661         {
662             \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
663             \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
664         }
665     }
666 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “*without any space in between*”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

667 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
668 {
669   \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
670   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
671   {
672     \str_if_eq:VnTF \l__spintent_luaset_status_str { valid }
673     {
674       \str_if_eq:VnTF
675       \l__spintent_luaset_denom_has_number_str { true }
676       {
677         \msg_error:nnn { spintent }
678         { inline-numeric-denominator } { #1 }
679       }
680       {
681         \__spintent_luafun_spunit_lookup_alias:V \l__spintent_luaset_arg_above_tl
682         \str_if_eq:VnTF \l__spintent_spunit_luaset_is_sexagesimal_str { true }
683         {
684           \tl_if_empty:NF \l__spintent_luaset_part_int_tl
685           {
686             \__spintent_spnum_print_number_core:
687           }
688           \MathMLintent { \l__spintent_spunit_luaset_read_str }
689           {
690             \mathord
691             {
692               \str_case:Vn \l__spintent_spunit_luaset_canonical_str
693               {
694                 { " } { \prime \! \prime }
695                 { ' } { \prime }
696                 { ° } { ° }
697               }
698             }
699           }
700         }
701         {
702           \tl_if_empty:NF \l__spintent_luaset_part_int_tl
703           {
704             \__spintent_spnum_print_number_core: \,
705           }
706           \__spintent_print_spunit_hierarchy:
707         }
708       }
709     }
710     { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
711   }
712 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit`

The public user command `\spunit` accepts an optional `[⟨key = val⟩]` argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

713 \NewDocumentCommand \spunit { O{} m }
714 {
715   \mode_if_math:TF
716   {
717     \group_begin:
718     \keys_set:nn { spintent/spunit } { #1 }
719     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }

```

```

720         \__spintenc_spunit_safe_exec:n { #2 }
721     \group_end:
722 }
723 { \msg_error:nnn { spintenc } { need-math-mode } { \spunit } }
724 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintenc.lua` via `\__spintenc_luafun_define_custom_unit:nn` and `\__spintenc_luafun_define_spunit_alias:nn`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintenc_spunit_luaseta_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

725 \msg_new:nnnn { spintenc } { spnewunit-duplicate }
726 {
727     The~unit~symbol~or~alias~'#1'~is~already~defined!
728 }
729 {
730     You~cannot~redefine~with~\token_to_str:N \spnewunit.
731 }
732 \msg_new:nnnn { spintenc } { spunitalias-duplicate }
733 {
734     The~alias~'#1'~is~already~defined~or~reserved!
735 }
736 {
737     You~cannot~redefine~an~existing~unit~or~another~alias.
738 }
739 \msg_new:nnnn { spintenc } { spunitalias-invalid-expr }
740 {
741     The~expression~'#2'~for~alias~'#1'~is~invalid.
742 }
743 {
744     It~must~contain~only~defined~units,~exponents~and~one~'/'~.
745 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintenc_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

746 \NewDocumentCommand \spnewunit { m m }
747 {
748     \mode_if_math:TF
749     {
750         \msg_error:nnn { spintenc } { need-text-mode } { \spnewunit }
751     }
752     {
753         \__spintenc_luafun_define_custom_unit:nn { #1 } { #2 }
754         \str_if_eq:VnT \l__spintenc_spunit_luaseta_status_str { duplicate }
755         {
756             \msg_error:nnn { spintenc } { spnewunit-duplicate } { #1 }
757         }
758     }
759 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintenc_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

760 \NewDocumentCommand \spunitalias { m m }

```

```

761 {
762   \mode_if_math:TF
763   {
764     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
765   }
766   {
767     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
768     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
769     {
770       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
771     }
772     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
773     {
774       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
775     }
776   }
777 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

778 \str_new:N \l__spintent_spmoney_luaset_symbol_str
779 \str_new:N \l__spintent_spmoney_luaset_position_str
780 \str_new:N \l__spintent_spmoney_luaset_sing_str
781 \str_new:N \l__spintent_spmoney_luaset_plur_str
782 \str_new:N \l__spintent_spmoney_luaset_conde_str
783 \str_new:N \l__spintent_spmoney_luaset_subsing_str
784 \str_new:N \l__spintent_spmoney_luaset_subplur_str
785 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
786 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
787 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

788 \bool_new:N \l__spintent_spmoney_sub_units_bool
789 \str_new:N \l__spintent_spmoney_intent_money_str
790 \str_new:N \l__spintent_spmoney_intent_sign_str
791 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
792 \str_new:N \l__spintent_spmoney_res_main_voice_str
793 \int_new:N \l__spintent_spmoney_subnum_int
794 \str_new:N \l__spintent_spmoney_extracted_curr_str
795 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

796 \msg_new:nnnn { spintent } { currency-no-subunits }
797 {
798   The~currency~'#1'~does~not~support~subunits.~
799   The~option~'sub-units=true'~is~ignored.
800 }

```

```

801 {
802     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
803     do~not~have~fractional~subunits~registered~in~Lua~module.~
804 }
805 \msg_new:nnnn { spintents } { invalid-currency }
806 {
807     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
808 }
809 {
810     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
811     the~Lua~module.
812 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.17.2 Auxiliary functions for \spmoney

```

__spintents_spmoney_set_currency:n
__spintents_spmoney_normalize_key:n
__spintents_spmoney_normalize_key:V

```

The function `\__spintents_spmoney_set_currency:n` validates the currency identifier provided in ‘`#1`’ against the database. It delegates the lookup to the helper function `\__spintents_spmoney_normalize_key:n`, which interfaces with the Lua module `spintents.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintents_spmoney_luaset_currency_arg_str` for subsequent processing.

```

813 \cs_new_protected:Nn __spintents_spmoney_set_currency:n
814 {
815     __spintents_spmoney_normalize_key:n {#1}
816     \str_if_eq:VnTF \__spintents_spmoney_luaset_status_str { notfound }
817     {
818         \msg_error:nnn { spintents } { invalid-currency } {#1}
819     }
820     { \str_set:Nn \__spintents_spmoney_luaset_currency_arg_str {#1} }
821 }
822 \cs_new_protected:Nn __spintents_spmoney_normalize_key:n
823 {
824     __spintents_luafun_spmoney_normalize_key:n { #1 }
825 }
826 \cs_generate_variant:Nn __spintents_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintents_spmoney_set_currency:n` and `\__spintents_spmoney_normalize_key:n`.)

11.17.3 Definition of keys for \spmoney

```

currency
sub-units
dolar-math
cifras-sep

```

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

827 \keys_define:nn { spintents / spmoney }
828 {
829     currency .code:n = { __spintents_spmoney_set_currency:n {#1} },
830     currency .initial:n = { pesos },
831     currency .value_required:n = true,
832     sub-units .bool_set:N = \__spintents_spmoney_sub_units_bool,
833     sub-units .value_required:n = true,
834     dolar-math .bool_set:N = \__spintents_spmoney_dolar_math_bool,
835     dolar-math .initial:n = { true },
836     dolar-math .value_required:n = true,
837     cifras-sep .meta:nn = { spintents/spnum } { cifras-sep = {#1} },
838     cifras-sep .value_required:n = true,
839     unknown .code:n =
840     {
841         \str_set:Nn \__spintents_command_name_str { spmoney }
842         __spintents_unknown_keys_cmd:n {#1}
843     },
844 }

```

(End of definition for `currency` and others.)

11.17.4 Internal functions for \spmoney

```

__spintents_spmoney_validate_money:

```

The function `\__spintents_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\__spintents_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\__spintents_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\__spintents_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintents.lua` via `\__spintents_luafun_spmoney_lookup_data:V`.

```

845 \cs_new_protected:Nn __spintents_spmoney_validate_money:
846 {
847     \str_if_eq:VnT \__spintents_luaset_has_units_str { true }
848     {

```

```

849     \__spintent_spmoney_normalize_key:V \__spintent_luaset_arg_above_tl
850     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
851       { \msg_error:nne { spintent } { invalid-currency } { \__spintent_luaset_arg_above_tl } }
852       {
853         \str_set:NV
854           \__spintent_spmoney_luaset_currency_arg_str
855           \__spintent_luaset_arg_above_tl
856       }
857     }
858     \str_set:Nc \__spintent_spmoney_luaset_status_str { found }
859     \__spintent_luafun_spmoney_lookup_data:V \__spintent_spmoney_luaset_currency_arg_str
860   }

```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:`

The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\__spintent_spmoney_luaset_print_iso_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

861 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
862 {
863   \str_if_eq:VnTF \__spintent_spmoney_luaset_print_iso_str { true }
864     { \mathrm { \__spintent_spmoney_luaset_currency_arg_str } }
865     {
866       \str_if_eq:VnTF \__spintent_spmoney_luaset_symbol_str { $ }
867       {
868         \bool_if:NTF \__spintent_spmoney_dolar_math_bool
869           { \$ } { \text { \$ } }
870       }
871       { \__spintent_spmoney_luaset_symbol_str }
872     }
873 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

874 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
875 {
876   \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
877   {
878     \__spintent_spmoney_print_symbol:
879     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
880   }
881   {
882     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
883     \__spintent_spmoney_print_symbol:
884   }
885 }

```

(End of definition for `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

886 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
887 {
888   \__spintent_spnum_render_int:
889   \__spintent_spnum_print_part_dec:
890 }
891 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
892 {
893   \__spintent_spnum_render_int:
894 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`
`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

895 \cs_new_protected:Nn \__spintent_spmoney_print_int:
896 {
897   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
898   {
899     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
900     { \__spintent_spmoney_set_and_print_sub_units: }
901     { \__spintent_spmoney_intent_int: }
902   }
903   {
904     \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
905     {
906       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
907     }
908     {
909       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
910     }
911     \__spintent_invisible_sep:
912     \MathMLintent { \l__spintent_spmoney_intent_sign_str }
913     {
914       \l__spintent_luaset_sign_tl
915       \MathMLarg { n }
916       {
917         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
918         { \__spintent_spmoney_set_and_print_sub_units: }
919         { \__spintent_spmoney_intent_int: }
920       }
921     }
922   }
923 }
924 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
925 {
926   \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
927   {
928     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
929     {
930       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
931     }
932     {
933       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
934     }
935     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
936     {
937       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
938     }
939   }
940   {
941     \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
942     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
943     {
944       \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
945       {
946         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
947       }
948       {
949         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur
950       }
951     }
952   }
953   \__spintent_invisible_sep: % need for MathCAT
954   \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
955   {
956     \__spintent_spmoney_render_layout:

```

```

957     }
958 }

```

(End of definition for `\__spintent_spmoney_print_int:` and `\__spintent_spmoney_intent_int:`)

```

\__spintent_spmoney_check_sub_units:
spintent_spmoney_set_and_print_sub_units:

```

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

959 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
960 {
961   \str_if_empty:NT \__spintent_spmoney_luaset_subsing_str
962   {
963     \msg_warning:nne { spintent } { currency-no-subunits }
964     { \__spintent_spmoney_luaset_currency_arg_str }
965     \bool_set_false:N \__spintent_spmoney_sub_units_bool
966   }
967 }
968 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
969 {
970   \int_compare:nTF { \str_count:N \__spintent_luaset_only_part_dec_str == 1 }
971   {
972     \int_set:Nn \__spintent_spmoney_subnum_int
973     { \__spintent_luaset_only_part_dec_str * 10 }
974     \str_set:Ne \__spintent_luaset_only_part_dec_str
975     { \__spintent_luaset_only_part_dec_str 0 }
976   }
977   {
978     \int_set:Nn \__spintent_spmoney_subnum_int
979     { \__spintent_luaset_only_part_dec_str }
980   }
981   \int_compare:nTF { \__spintent_spmoney_subnum_int == 1 }
982   {
983     \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
984     { \__spintent_spmoney_luaset_subsing_str :postfix($sub) }
985   }
986   {
987     \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
988     { \__spintent_spmoney_luaset_subplur_str :postfix($sub) }
989   }
990   \str_if_eq:VnTF \__spintent_luaset_only_part_int_str { 1 }
991   {
992     \str_set:NV \__spintent_spmoney_res_main_voice_str
993     \__spintent_spmoney_luaset_sing_str
994   }
995   {
996     \str_set:NV \__spintent_spmoney_res_main_voice_str
997     \__spintent_spmoney_luaset_plur_str
998   }
999   \str_if_eq:VnT \__spintent_luaset_millions_str { true }
1000   {
1001     \str_set:NV \__spintent_spmoney_res_main_voice_str
1002     \__spintent_spmoney_luaset_conde_str
1003   }
1004   \__spintent_spmoney_build_tree:
1005 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`)

```

\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:

```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

1006 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1007 {

```

```

1008 \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1009 {
1010   \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1011   {
1012     \__spintent_spmoney_print_symbol:
1013     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1014   }
1015   \__spintent_spmoney_print_sep_con:
1016   \__spintent_spmoney_print_dec_sub_unit:
1017 }
1018 {
1019   \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1020   {
1021     \__spintent_invisible_sep:
1022     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1023   }
1024   \__spintent_spmoney_print_sep_con:
1025   \__spintent_spmoney_print_dec_sub_unit:
1026   \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1027 }
1028 }
1029 %^^A \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
1030 %^^A {
1031 %^^A   \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1032 %^^A   {
1033 %^^A     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1034 %^^A   }
1035 %^^A }
1036 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1037 {
1038   \MathMLintent { con }
1039   {
1040     \mathord { \l__spintent_luaset_dec_sep_tl }
1041   }
1042 }
1043 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1044 {
1045   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1046   {
1047     \__spintent_invisible_sep:
1048     \MathMLarg { sub }
1049     {
1050       \__spintent_spmoney_render_dec:
1051     }
1052   }
1053 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{(argument)}` passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1054 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1055 {
1056   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1057   \__spintent_luafun_clean_split_and_set:V \l__spintent_spmoney_extracted_num_tl
1058   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1059   {
1060     \str_set:Ne \l__spintent_luaset_has_units_str { true }
1061     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1062   }
1063   \__spintent_spmoney_validate_money:
1064   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1065   { \__spintent_spmoney_check_sub_units: }
1066   \__spintent_spmoney_print_int:
1067 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney` The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1068 \NewDocumentCommand \spmoney { 0{ } m }
1069 {
1070   \mode_if_math:TF
1071   {
1072     \group_begin:
1073     \keys_set:nn { spintent / spmoney } { #1 }
1074     \__spintent_spmoney_parse_and_route:n { #2 }
1075     \group_end:
1076   }
1077   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1078 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1079 \str_new:N \__spintent_spshort_luaset_status_str
1080 \str_new:N \__spintent_spshort_luaset_layout_str
1081 \str_new:N \__spintent_spshort_luaset_expanded_str
1082 \str_new:N \__spintent_spshort_luaset_output_str
1083 \str_new:N \__spintent_spshort_luaset_base_str
1084 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{\argument}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1085 \msg_new:nnnn { spintent } { unknown-abbreviation }
1086 { The~abbreviation~or~ordinal~'~#1'~is~invalid~or~not~recognized. }
1087 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn` The internal function `\__spintent_spshort_tag_span:nn` processes the `{\argument}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{\argument}` passed in ‘#1’ for proper reading with NVDA.

```

1088 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1089 {
1090   \tag_mc_end_push:
1091   \tag_struct_begin:n { tag=Span, E={ #1 } }
1092   \tag_mc_begin:n { tag=Span }
1093   \tag_suspend:n { \spshort }
1094   #2
1095   \tag_resume:n { \spshort }
1096   \tag_mc_end:
1097   \tag_struct_end:
1098   \tag_mc_begin_pop:n {}
1099 }
1100 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for \spshort

small-caps Now we add the keys `small-caps` and `read-arg`.

```
read-arg 1101 \keys_define:nn { spintent / spshort }
1102 {
1103     small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
1104     small-caps .initial:n = false,
1105     small-caps .value_required:n = true,
1106     read-arg .str_set:N = \l__spintent_spshort_read_arg_str,
1107     read-arg .initial:n = {},
1108     read-arg .value_required:n = true,
1109     unknown .code:n =
1110     {
1111         \str_set:Nn \l__spintent_command_name_str { spshort }
1112         \__spintent_unknown_keys_cmd:n {#1}
1113     },
1114 }
```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\l__spintent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in `'#1'`. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{<argument>}` passed in `'#2'`. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```
1115 \cs_new_protected:Nn \__spintent_spshort_print:nn
1116 {
1117     \str_clear:N \l__spintent_spshort_read_arg_str
1118     \keys_set:nn { spintent / spshort } {#1}
1119     \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1120     {
1121         \__spintent_spshort_process_and_render:n {#2}
1122     }
1123     {
1124         \__spintent_spshort_render_bypass:n {#2}
1125     }
1126 }
```

(End of definition for `\__spintent_spshort_print:nn`.)

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{<argument>}` passed in `'#1'`.

It subsequently evaluates the execution state stored within the string variable `\l__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```
1127 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1128 {
1129     \__spintent_luafun_spshort_process:n {#1}
1130     \str_case:NnF \l__spintent_spshort_luaset_status_str
1131     {
1132         { success } { \__spintent_spshort_render_success: }
1133         { fallback } { \__spintent_spshort_render_fallback: }
1134         { error } { \__spintent_spshort_render_error:n {#1} }
1135     }
1136     { \__spintent_spshort_render_error:n {#1} }
1137 }
```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the `{<argument>}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_rende` triggers a structural exception while yielding the unformatted `{<argument>}` to prevent fatal compilation.

```
1138 \cs_new_protected:Nn \__spintent_spshort_render_success:
1139 {
```

```

1140     \__spintent_spshort_tag_span:en
1141     { \__spintent_spshort_luaset_expanded_str }
1142     { \__spintent_spshort_execute_layout: }
1143   }
1144   \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1145   {
1146     \__spintent_spshort_tag_span:en
1147     { \__spintent_spshort_luaset_expanded_str }
1148     {
1149       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1150       { \textsc{ \__spintent_spshort_luaset_output_str } }
1151       { \__spintent_spshort_luaset_output_str }
1152     }
1153   }
1154   \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1155   {
1156     \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }
1157   }
1158   \cs_new_protected:Nn \__spintent_spshort_render_error:n
1159   {
1160     \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1161   }

```

(End of definition for __spintent_spshort_render_success: and others.)

__spintent_spshort_execute_layout:

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\text{\LaTeX}$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1162   \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1163   {
1164     \str_case:VnF \__spintent_spshort_luaset_layout_str
1165     {
1166       { linear_regular }
1167       {
1168         \bool_if:NTF \__spintent_spshort_smallcaps_bool
1169         { \textsc{ \__spintent_spshort_luaset_output_str } }
1170         { \__spintent_spshort_luaset_output_str }
1171       }
1172       { linear_caps }
1173       { \__spintent_spshort_luaset_output_str }
1174       { linear_mixed }
1175       { \__spintent_spshort_luaset_output_str }
1176       { superscript }
1177       {
1178         \__spintent_spshort_luaset_base_str
1179         \bool_if:NTF \__spintent_spshort_smallcaps_bool
1180         {
1181           \textsuperscript{
1182             \textsc{
1183               \str_lowercase:V \__spintent_spshort_luaset_suffix_str
1184             }
1185           }
1186         }
1187         {
1188           \textsuperscript{ \__spintent_spshort_luaset_suffix_str }
1189         }
1190       }
1191       { small_caps_pure }
1192       {
1193         \bool_if:NTF \__spintent_spshort_smallcaps_bool
1194         {
1195           \textsc{
1196             \str_lowercase:V \__spintent_spshort_luaset_output_str
1197           }
1198         }
1199         { \__spintent_spshort_luaset_output_str }
1200       }
1201       { acronym_long }
1202       { \__spintent_spshort_luaset_output_str }

```

```

1203     }
1204     { \l__spintent_spsshort_luaset_output_str }
1205 }

```

(End of definition for \l__spintent_spsshort_execute_layout:.)

\spsshort The public document command **\spsshort** manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts **\mode_leave_vertical:** and establishes a local group to call the function **\l__spintent_spsshort_print:nn** for processing and printing.

```

1206 \NewDocumentCommand \spsshort { 0{} m }
1207 {
1208     \mode_if_math:TF
1209     {
1210         \msg_error:nnn { spintent } { need-text-mode } { \spsshort }
1211     }
1212     {
1213         \mode_leave_vertical:
1214         \group_begin:
1215         \l__spintent_spsshort_print:nn {#1} {#2}
1216         \group_end:
1217     }
1218 }

```

(End of definition for \spsshort. This function is documented on page 7.)

11.19 The command \spsiglo

The public document command **\spsiglo** provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module **spintent.lua** to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible **MathML** intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an **alt** key.

\l__spintent_spsiglo_luaset_error_str These internal string variables handle data transfer and synchronization with the Lua module **spintent.lua**.
\l__spintent_spsiglo_luaset_arabic_str They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase
\l__spintent_spsiglo_luaset_roman_min_str Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.
\l__spintent_spsiglo_print_era_str

```

1219 \str_new:N \l__spintent_spsiglo_luaset_error_str
1220 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1221 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1222 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for \l__spintent_spsiglo_luaset_error_str and others.)

11.19.1 Definition of error messages for \spsiglo

invalid-century-format This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module **spintent.lua** cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1223 \msg_new:nnnn { spintent } { invalid-century-format }
1224 {
1225     Invalid~century~format~in~'#1'.
1226 }
1227 {
1228     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1229     Ensure~the~value~is~between~1~and~4000.
1230 }

```

(End of definition for invalid-century-format.)

11.19.2 Definition of keys for \spsiglo

print-era Now we add the key **print-era**. User configuration determining if an era designation is appended, available values are **none**, **ac** (antes de cristo) and **dc** (después de cristo).

```

1231 \keys_define:nn { spintent / spsiglo }
1232 {
1233     print-era .choices:nn =
1234     { none , ac , dc }
1235     {
1236         \int_case:nn { \l_keys_choice_int }
1237         {

```

```

1238         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1239         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1240         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1241     }
1242 },
1243 print-era .initial:n = none,
1244 print-era .value_required:n = true,
1245 print-era / unknown .code:n =
1246     \msg_error:nneee { spintent } { unknown-choice }
1247     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1248 unknown .code:n =
1249     {
1250         \str_set:Nn \l__spintent_command_name_str { spsiglo }
1251         \__spintent_unknown_keys_cmd:n {#1}
1252     },
1253 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1254 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1255 {
1256     \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1257 }
1258 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1259 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1260 {
1261     \str_if_eq:nnTF { #1 } { ac }
1262     {
1263         \MathMLintent { antes-de-cristo:postfix($z) }
1264         {
1265             \__spintent_invisible_sep:
1266             \MathMLarg { z }
1267             {
1268                 \MathMLintent { #2:number }
1269                 {
1270                     \text { \textsc {#3}~a.~C. }
1271                 }
1272             }
1273         }
1274     }
1275     {
1276         \MathMLintent { después-de-cristo:postfix($z) }
1277         {
1278             \__spintent_invisible_sep:
1279             \MathMLarg { z }
1280             {
1281                 \MathMLintent { #2:number }
1282                 {
1283                     \text { \textsc {#3}~d.~C. }
1284                 }
1285             }
1286         }
1287     }
1288 }
1289 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1290 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1291 {
1292     \tag_mc_end_push:

```

```

1293     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1294     \tag_mc_begin:n { tag=Span }
1295     \tag_suspend:n { \spsiglo }
1296     #2
1297     \tag_resume:n { \spsiglo }
1298     \tag_mc_end:
1299     \tag_struct_end:
1300     \tag_mc_begin_pop:n {}
1301   }
1302   \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

(End of definition for \__spintent_spsiglo_tag_span:nn.)

```

```

\__spintent_spsiglo_print_math:n
\__spintent_spsiglo_print_text:n

```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1303   \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1304   {
1305     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1306     {
1307       \msg_error:nnnn { spintent } { invalid-century-format }
1308       { \spsiglo } { #1 }
1309     }
1310     {
1311       \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1312       {
1313         \__spintent_spsiglo_render_simple:VV
1314         \l__spintent_spsiglo_luaset_arabic_str
1315         \l__spintent_spsiglo_luaset_roman_min_str
1316       }
1317       {
1318         \__spintent_spsiglo_render_with_era:VVV
1319         \l__spintent_spsiglo_print_era_str
1320         \l__spintent_spsiglo_luaset_arabic_str
1321         \l__spintent_spsiglo_luaset_roman_min_str
1322       }
1323     }
1324   }
1325   \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1326   {
1327     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1328     {
1329       \msg_error:nnnn { spintent } { invalid-century-format }
1330       { \spsiglo } { #1 }
1331     }
1332     {
1333       \str_case:Vn \l__spintent_spsiglo_print_era_str
1334       {
1335         { none }
1336         {
1337           \__spintent_spsiglo_tag_span:en
1338           { \l__spintent_spsiglo_luaset_arabic_str }
1339           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1340         }
1341         { ac }
1342         {
1343           \__spintent_spsiglo_tag_span:en
1344           { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1345           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1346         }
1347         { dc }
1348         {
1349           \__spintent_spsiglo_tag_span:en
1350           { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1351           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1352         }
1353       }
1354     }
1355   }

```

```

1353     }
1354   }
1355 }

```

(End of definition for `\__spinttent_spsiglo_print_math:n` and `\__spinttent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function `\__spinttent_luafun_spsiglo_parse:n` defined in the Lua module `spinttent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1356 \NewDocumentCommand \spsiglo { 0{} m }
1357 {
1358   \group_begin:
1359     \keys_set:nn { spinttent / spsiglo } { #1 }
1360     \__spinttent_luafun_spsiglo_parse:n { #2 }
1361     \mode_if_math:TF
1362       { \__spinttent_spsiglo_print_math:n { #2 } }
1363       {
1364         \mode_leave_vertical:
1365         \__spinttent_spsiglo_print_text:n { #2 }
1366       }
1367   \group_end:
1368 }

```

(End of definition for `\spsiglo`. This function is documented on page 11.)

11.20 The command `\sptime`

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spinttent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spinttent_sptime_luaset_seconds_str
\__spinttent_sptime_luaset_has_seconds_str
\__spinttent_sptime_luaset_error_str
\__spinttent_sptime_luaset_is_fraction_str
\__spinttent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spinttent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1369 \str_new:N \__spinttent_sptime_luaset_seconds_str
1370 \str_new:N \__spinttent_sptime_luaset_has_seconds_str
1371 \str_new:N \__spinttent_sptime_luaset_error_str
1372 \str_new:N \__spinttent_sptime_luaset_is_fraction_str
1373 \str_new:N \__spinttent_sptime_luaset_base_str

```

(End of definition for `\__spinttent_sptime_luaset_seconds_str` and others.)

11.20.1 Definition of error messages

`invalid-time-format`

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1374 \msg_new:nnnn { spinttent } { invalid-time-format }
1375 {
1376   Invalid~time~format~in~#1. \\\
1377   The~argument~'~#2'~does~not~match~any~supported~time~format.
1378 }
1379 {
1380   Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1381   Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1382 }

```

(End of definition for `invalid-time-format`.)

11.20.2 Internal functions for `\sptime`

`\__spinttent_sptime_build_seconds:nn`

The internal function `\__spinttent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1383 \cs_new_protected:Nn \__spinttent_sptime_build_seconds:nn
1384 {
1385   \MathMLintent { #1:prefix($z) }
1386   {
1387     \text { : }
1388     \MathMLarg { z }
1389     {
1390       \MathMLintent { segundos:postfix($m) }
1391       {

```

```

1392         \__spintent_invisible_sep:
1393         \MathMLLarg { m }
1394         {
1395             \text { #2 }
1396         }
1397     }
1398 }
1399 }
1400 }

```

(End of definition for __spintent_sptime_build_seconds:nn.)

__spintent_sptime_build_tree: The internal function __spintent_sptime_build_tree: builds the complete MathML structure for the time expression. It checks the string variables \l__spintent_sptime_luaset_has_seconds_str and \l__spintent_sptime_luaset_is_fraction_str to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls __spintent_sptime_build_seconds: with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```

1401 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1402 {
1403     \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1404     {
1405         \MathMLintent { :time }
1406         {
1407             \text { \l__spintent_sptime_luaset_base_str }
1408         }
1409         \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1410         {
1411             \__spintent_sptime_build_seconds:nn { con }
1412             { \l__spintent_sptime_luaset_seconds_str }
1413         }
1414         {
1415             \__spintent_sptime_build_seconds:nn { minutos-con }
1416             { \l__spintent_sptime_luaset_seconds_str }
1417         }
1418     }
1419     {
1420         \MathMLintent { :time }
1421         {
1422             \text { \l__spintent_sptime_luaset_base_str }
1423         }
1424         \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1425         {
1426             \MathMLintent { minutos }
1427             {
1428                 \__spintent_invisible_sep:
1429             }
1430         }
1431     }
1432 }

```

(End of definition for __spintent_sptime_build_tree:.)

\sptime The command \sptime is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using __spintent_luafun_sptime_parse:n and checks the error status in \l__spintent_sptime_luaset_error_str. If an invalid format is detected, it issues the invalid-time-format error; otherwise, it calls __spintent_sptime_build_tree: to assemble the MathML output.

```

1433 \NewDocumentCommand \sptime { m }
1434 {
1435     \mode_if_math:TF
1436     {
1437         \group_begin:
1438         \__spintent_luafun_sptime_parse:n { #1 }
1439         \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1440         {
1441             \msg_error:nnnn { spintent } { invalid-time-format }
1442             { \sptime } { #1 }
1443         }
1444         {
1445             \__spintent_sptime_build_tree:

```

```

1446         }
1447     \group_end:
1448 }
1449 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1450 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\__spintent_spdate_luaset_output_str
\__spintent_spdate_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1451 \str_new:N \__spintent_spdate_luaset_output_str
1452 \str_new:N \__spintent_spdate_luaset_error_str

```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

```
invalid-date-format
```

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1453 \msg_new:nnnn { spintent } { invalid-date-format }
1454 {
1455     Invalid~date~or~unsupported~format~in~#1. \\\
1456     The~argument~'#2'~does~not~match~a~supported~format.
1457 }
1458 {
1459     Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1460     Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1461 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

```
\__spintent_spdate_process:n
```

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```

1462 \cs_new_protected:Nn \__spintent_spdate_process:n
1463 {
1464     \__spintent_luafun_spdate_parse:n { #1 }
1465     \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1466     {
1467         \msg_error:nnnn { spintent } { invalid-date-format }
1468         { \spdate } { #1 }
1469     }
1470     {
1471         \MathMLintent { :date }
1472         {
1473             \text { \__spintent_spdate_luaset_output_str }
1474         }
1475     }
1476 }

```

(End of definition for `\__spintent_spdate_process:n`.)

```
\spdate
```

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```

1477 \NewDocumentCommand \spdate { m }
1478 {
1479     \mode_if_math:TF
1480     {
1481         \group_begin:
1482         \__spintent_spdate_process:n { #1 }
1483         \group_end:
1484     }
1485     { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1486 }

```

(End of definition for `\spdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *(keys)* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

spusep-empty-arg
spusep-invalid-arg

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *{(argument)}*, the second this is when an input is invalid.

```

1487 \msg_new:nnnn { spintent } { spusep-empty-arg }
1488 {
1489   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1490 }
1491 {
1492   \token_to_str:N \spusep~requires~a~non-empty~argument.
1493 }
1494 \msg_new:nnn { spintent } { spusep-invalid-arg }
1495 {
1496   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1497 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

__spintent_spusep_arg_tl
__spintent_spusep_left_tl
__spintent_spusep_right_tl

The variable `\__spintent_spusep_arg_tl` stores the *{(argument)}*, the variable `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` stores the parentheses.

```

1498 \tl_new:N \__spintent_spusep_arg_tl
1499 \tl_new:N \__spintent_spusep_left_tl
1500 \tl_new:N \__spintent_spusep_right_tl

```

(End of definition for `\__spintent_spusep_arg_tl`, `\__spintent_spusep_left_tl`, and `\__spintent_spusep_right_tl`.)

size
silent
unknown

We define the *(keys)* `size` and `silent`.

```

1501 \keys_define:nn { spintent / spusep }
1502 {
1503   size .choices:nn =
1504     { none , big , Big, bigg, Bigg }
1505     {
1506       \int_case:nn { \_keys_choice_int }
1507       {
1508         { 1 }
1509         {
1510           \tl_clear:N \__spintent_spusep_left_tl
1511           \tl_clear:N \__spintent_spusep_right_tl
1512         }
1513         { 2 }
1514         {
1515           \tl_set:Nn \__spintent_spusep_left_tl { \bigl }
1516           \tl_set:Nn \__spintent_spusep_right_tl { \bigr }
1517         }
1518         { 3 }
1519         {
1520           \tl_set:Nn \__spintent_spusep_left_tl { \Bigl }
1521           \tl_set:Nn \__spintent_spusep_right_tl { \Bigr }
1522         }
1523         { 4 }
1524         {
1525           \tl_set:Nn \__spintent_spusep_left_tl { \biggl }
1526           \tl_set:Nn \__spintent_spusep_right_tl { \biggr }
1527         }
1528         { 5 }
1529         {
1530           \tl_set:Nn \__spintent_spusep_left_tl { \Biggl }
1531           \tl_set:Nn \__spintent_spusep_right_tl { \Biggr }
1532         }
1533       }
1534     }

```

```

1534     },
1535     size          .initial:n = none,
1536     size          .value_required:n = true,
1537     size / unknown .code:n =
1538       \msg_error:nneee { spintent } { unknown-choice }
1539     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1540     silent        .bool_set:N = \l__spintent_spusep_silent_bool,
1541     silent        .initial:n = false,
1542     silent        .value_required:n = true,
1543     unknown       .code:n =
1544       {
1545         \str_set:Nn \l__spintent_command_name_str { spusep }
1546         \l__spintent_unknown_keys_cmd:n {#1}
1547       },
1548   }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\l__spintent_spusep_set_parens:nn`

The function `\l__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1549 \cs_new_protected:Nn \l__spintent_spusep_set_parens:nn
1550 {
1551   \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1552   \bool_if:NTF \l__spintent_spusep_silent_bool
1553   {
1554     \MathMLintent { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1555   }
1556   {
1557     \tl_use:c { \l__spintent_spusep_ #1 _tl }
1558   }
1559 }

```

(End of definition for `\l__spintent_spusep_set_parens:nn`.)

`\l__spintent_spusep_convert:n`

The function `\l__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\l__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1560 \cs_new_protected:Nn \l__spintent_spusep_convert:n
1561 {
1562   \tl_set:Ne \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1563   \tl_if_empty:NT \l__spintent_spusep_arg_tl
1564   {
1565     \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1566   }
1567   \str_case:VnF \l__spintent_spusep_arg_tl
1568   {
1569     { ( }
1570     {
1571       \l__spintent_spusep_set_parens:nn { left } { \lparen }
1572     }
1573     { ) }
1574     {
1575       \l__spintent_spusep_set_parens:nn { right } { \rparen }
1576     }
1577     { [ ] }
1578     {
1579       \l__spintent_spusep_set_parens:nn { left } { \lbrack }
1580     }
1581     { ] }
1582     {
1583       \l__spintent_spusep_set_parens:nn { right } { \rbrack }
1584     }
1585     { \{ }
1586     {
1587       \l__spintent_spusep_set_parens:nn { left } { \lbrace }
1588     }
1589     { \} }
1590     {
1591       \l__spintent_spusep_set_parens:nn { right } { \rbrace }
1592     }

```

```

1592     }
1593   }
1594   {
1595     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1596   }
1597 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1598 \NewDocumentCommand \spusep { O{} m }
1599 {
1600   \mode_if_math:TF
1601   {
1602     \group_begin:
1603     \keys_set:nn { spintent / spusep } {#1}
1604     \__spintent_spusep_convert:n {#2}
1605     \group_end:
1606   }
1607   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1608 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for **MathCAT** of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spread-empty-args The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1609 \msg_new:nnnn { spintent } { spread-empty-args }
1610 { Empty~argument~in~\token_to_str:N \spread. }
1611 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

__spintent_spread_intent_tl The variable `\__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1612 \tl_new:N \__spintent_spread_intent_tl

```

(End of definition for `\__spintent_spread_intent_tl`.)

__spintent_spread_exec:nn The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1613 \cs_new_protected:Nn \__spintent_spread_exec:nn
1614 {
1615   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1616   { \msg_error:nn { spintent } { spread-empty-args } }
1617   {
1618     \__spintent_sanitize_affix:Nn \__spintent_spread_intent_tl { #1 }
1619     \MathMLintent { \__spintent_spread_intent_tl } { #2 }
1620   }
1621 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

\spread The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1622 \NewDocumentCommand \spread { m m }
1623 {
1624   \mode_if_math:TF
1625   {
1626     \group_begin:
1627     \__spintent_spread_exec:nn { #1 } { #2 }
1628     \group_end:
1629   }
1630   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1631 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.23.2 The command \spdsym

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\l__spintent_spdsym_read_sym_tl
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

```
prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
```

```
1636 \keys_define:nn { spintent / spdsym }
1637 {
1638   prefix .code:n =
1639   { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1640   prefix .initial:n = ,
1641   prefix .value_required:n = true,
1642   suffix .code:n =
1643   { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1644   suffix .initial:n = ,
1645   suffix .value_required:n = true,
1646   set-sym .choices:nn =
1647   { old-style , two-dots , slash }
1648   {
1649     \int_case:nn { \l_keys_choice_int }
1650     {
1651       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1652       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1653       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1654     }
1655   },
1656   set-sym .initial:n = two-dots,
1657   set-sym .value_required:n = true,
1658   set-sym / unknown .code:n =
1659   \msg_error:nneee { spintent } { unknown-choice }
1660   { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1661   read-sym .choices:nn =
1662   { dividido , dividido-por , dividido-entre }
1663   {
1664     \int_case:nn { \l_keys_choice_int }
1665     {
1666       {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1667       {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1668       {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1669     }
1670   },
1671   read-sym .initial:n = dividido,
1672   read-sym .value_required:n = true,
1673   read-sym / unknown .code:n =
1674   \msg_error:nneee { spintent } { unknown-choice }
1675   { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1676   { \exp_not:n {#1} },
1677   unknown .code:n =
1678   {
1679     \str_set:Nn \l__spintent_command_name_str { spdsym }
1680     \l__spintent_unknown_keys_cmd:n {#1}
1681   },
1682 }
```

(End of definition for `prefix` and others.)

`\spdsym` Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```
1683 \NewDocumentCommand \spdsym { 0{ } }
1684 {
1685   \mode_if_math:TF
1686   {
1687     \group_begin:
1688     \keys_set:nn { spintent / spdsym } { #1 }
1689     \MathMLintent
1690     {
```

```

1691         \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1692         { \l__spintent_spdsym_prefix_tl - }
1693         \l__spintent_spdsym_read_sym_tl
1694         \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1695         { - \l__spintent_spdsym_suffix_tl }
1696     }
1697     { \l__spintent_spdsym_symbol_tl }
1698 \group_end:
1699 }
1700 { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1701 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1702 \tl_new:N \l__spintent_spmsym_read_sym_tl
1703 \tl_new:N \l__spintent_spmsym_symbol_tl
1704 \tl_new:N \l__spintent_spmsym_prefix_tl
1705 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix    We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym   1706 \keys_define:nn { spintent / spmsym }
           {
read-sym   1708     prefix                .code:n =
           { \l__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
not-print  1709     prefix                .initial:n = ,
           1710     prefix                .value_required:n = true,
unknown    1711     suffix                .code:n =
           1712     { \l__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
           1713     suffix                .initial:n = ,
           1714     suffix                .value_required:n = true,
           1715     set-sym                .choices:nn =
           1716     { cross , dot , asterisk }
           1717     {
           1718     \int_case:nn { \l_keys_choice_int }
           1719     {
           1720     {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
           1721     {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
           1722     {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
           1723     }
           1724     },
           1725     set-sym                .initial:n = dot,
           1726     set-sym                .value_required:n = true,
           1727     set-sym / unknown .code:n =
           1728     \msg_error:nneee { spintent } { unknown-choice }
           1729     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
           1730     read-sym                .choices:nn =
           1731     { por , multiplicado-por , veces }
           1732     {
           1733     \int_case:nn { \l_keys_choice_int }
           1734     {
           1735     {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
           1736     {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
           1737     {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
           1738     }
           1739     },
           1740     read-sym                .initial:n = por,
           1741     read-sym                .value_required:n = true,
           1742     read-sym / unknown .code:n =
           1743     \msg_error:nneee { spintent } { unknown-choice }
           1744     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
           1745     not-print                .bool_set:N = \l__spintent_spmsym_not_print_bool,
           1746     not-print                .initial:n = false,
           1747     not-print                .value_required:n = true,
           1748     unknown                .code:n =
           1749     {
           1750     \str_set:Nn \l__spintent_command_name_str { spmsym }
           1751     \__spintent_unknown_keys_cmd:n {#1}
           1752

```

```

1753     },
1754 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *⟨keys⟩*, safely attaching prefixes and suffixes if present.

```

1755 \NewDocumentCommand \spmsym { 0{} }
1756 {
1757   \mode_if_math:TF
1758   {
1759     \group_begin:
1760     \keys_set:nn { spintent / spmsym } { #1 }
1761     \MathMLintent
1762     {
1763       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1764       { \l__spintent_spmsym_prefix_tl - }
1765       \l__spintent_spmsym_read_sym_tl
1766       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1767       { - \l__spintent_spmsym_suffix_tl }
1768     }
1769     {
1770       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1771       {
1772         \invisibletimes
1773       }
1774       { \l__spintent_spmsym_symbol_tl }
1775     }
1776     \group_end:
1777   }
1778   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1779 }

```

(End of definition for *\spmsym*. This function is documented on page 12.)

11.24 The command **\spdiv**

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *⟨dividend⟩* and the *⟨divisor⟩*. Both *⟨arguments⟩* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *⟨dividend⟩* nor the *⟨divisor⟩* is *blank*.

11.24.1 Definition of keys for **\spdiv**

We define the *⟨keys⟩* **wrap-parens** and **read-parens** for **\spdiv** command. Number-related *⟨keys⟩* **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using **.meta:nn**, while semantic operator *⟨keys⟩* **set-sym** and **read-sym** are forwarded to the **\spdsym** command using **.meta:nn**.

```

1780 \keys_define:nn { spintent / spdiv }
1781 {
1782   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1783   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1784   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1785   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1786   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1787   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1788   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1789   wrap-parens     .initial:n = false,
1790   wrap-parens     .value_required:n = true,
1791   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1792   read-parens     .initial:n = true,
1793   read-parens     .value_required:n = true,
1794   unknown         .code:n =
1795   {
1796     \str_set:Nn \l__spintent_command_name_str { spdiv }
1797     \l__spintent_unknown_keys_cmd:n {#1}
1798   },
1799 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1800 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1801 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1802 {
1803   You~must~provide~both~a~dividend~and~a~divisor.\\
1804 }

```

(End of definition for `spdiv-empty-argument`.)

11.24.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1805 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1806 {
1807   \spnum {#1}
1808   \MathMLintent { \__spintent_spdsym_read_sym_tl }
1809   {
1810     \__spintent_spdsym_symbol_tl
1811   }
1812   \spnum {#2}
1813 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1814 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1815 {
1816   \bool_if:NTF \__spintent_spdiv_read_parens_bool
1817   {
1818     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1819   }
1820   {
1821     \MathMLintent { :silent } { ( }
1822     \__spintent_spdiv_render_op:nn {#1} {#2}
1823     \MathMLintent { :silent } { ) }
1824   }
1825 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn` The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1826 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1827 {
1828   \keys_set:nn { spintent / spdiv } { #1 }
1829   \tl_if_blank:nTF { #2 }
1830   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1831   {
1832     \tl_if_blank:nTF { #3 }
1833     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1834     {
1835       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1836       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1837       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1838     }
1839   }
1840 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1841 \NewDocumentCommand \spdiv { O{} m m }
1842 {
1843   \mode_if_math:TF
1844   {
1845     \group_begin:
1846     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1847     \group_end:
1848   }
1849   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1850 }

```

(End of definition for `\spd\div`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{\langle first factor \rangle}` and the `{\langle second factor \rangle}`. Both `{\langle arguments \rangle}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{\langle first factor \rangle}` nor the `{\langle second factor \rangle}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `\keys` wrap-parens and read-parens for `\spmul` command. Number-related `\keys` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `\keys` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1851 \keys_define:nn { spintent / spmul }
1852 {
1853   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1854   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1855   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1856   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1857   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1858   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1859   wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1860   wrap-parens     .initial:n  = false,
1861   wrap-parens     .value_required:n = true,
1862   read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1863   read-parens     .initial:n  = true,
1864   read-parens     .value_required:n = true,
1865   unknown         .code:n =
1866     {
1867       \str_set:Nn \l__spintent_command_name_str { spmul }
1868       \__spintent_unknown_keys_cmd:n {#1}
1869     },
1870 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{\langle first factor \rangle}` or the `{\langle second factor \rangle}` is missing or *blank*.

```

1871 \msg_new:nnnn { spintent } { spmul-empty-argument }
1872 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1873 {
1874   You~must~provide~both~first~factor~and~second~factor.\\
1875 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both `{\langle arguments \rangle}` through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1876 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1877 {
1878   \spnum {#1}
1879   \MathMLintent { \l__spintent_spmsym_read_sym_tl }
1880   {
1881     \l__spintent_spmsym_symbol_tl
1882   }
1883   \spnum {#2}
1884 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1885 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1886 {
1887   \bool_if:NTF \l__spintent_spmul_read_parens_bool
1888   {
1889     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1890   }
1891   {
1892     \MathMLintent { :silent } { ( }

```

```

1893     \__spintent_spmul_render_op:nn {#1} {#2}
1894     \MathMLintent { :silent } { ) }
1895   }
1896 }

```

(End of definition for __spintent_spmul_render_parens:nn.)

__spintent_spmul_process:nnn

The function `\__spintent_spmul_process:nnn` sets $\langle keys \rangle$, validates $\langle arguments \rangle$, and decides on parenthesis wrapping.

```

1897 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1898 {
1899   \keys_set:nn { spintent / spmul } { #1 }
1900   \tl_if_blank:nTF { #2 }
1901   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1902   {
1903     \tl_if_blank:nTF { #3 }
1904     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1905     {
1906       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1907       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1908       { \__spintent_spmul_render_op:nn {#2} {#3} }
1909     }
1910   }
1911 }

```

(End of definition for __spintent_spmul_process:nnn.)

\spmul

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1912 \NewDocumentCommand \spmul { O{} m m }
1913 {
1914   \mode_if_math:TF
1915   {
1916     \group_begin:
1917     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1918     \group_end:
1919   }
1920   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1921 }

```

(End of definition for \spmul. This function is documented on page 13.)

11.26 The commands \spgcd and \spmcn

The user-level commands `\spgcd` and `\spmcn` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for \spgcd and \spmcn

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

\__spintent_spmcm_spmcd_luaset_error_str
\__spintent_mc_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl

```

```

1922 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
1923 \seq_new:N \__spintent_mc_args_seq
1924 \clist_map_inline:nn { mcd , mcm }
1925 {
1926   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
1927   \bool_new:c { \__spintent_sp #1 _print_result_bool }
1928   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1929   \bool_new:c { \__spintent_sp #1 _print_upper_bool }
1930   \tl_new:c { \__spintent_sp #1 _prefix_tl }
1931   \tl_new:c { \__spintent_sp #1 _suffix_tl }
1932 }

```

(End of definition for __spintent_spmcm_spmcd_luaset_error_str and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1933 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1934 {
1935   Invalid~arguments~provided~to~#1.
1936 }
1937 {
1938   The~arguments~must~be~a~comma-separated~list. \\\
1939 }

```

(End of definition for `mc-invalid-arguments`.)

11.26.3 Keys for `\spmc`d and `\spmc`m

We define the shared *(keys)* `result`, `prefix`, `read-cmd`, `upper-case`, `sample-arg` and `silent-cmd`.

```

1940 \clist_map_inline:nn { mcd , mcm }
1941 {
1942   \keys_define:nn { spintent / sp #1 }
1943   {
1944     result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
1945     result      .initial:n = false,
1946     result      .value_required:n = true,
1947     prefix      .code:n =
1948       { \__spintent_sanitise_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
1949     prefix      .initial:n = ,
1950     prefix      .value_required:n = true,
1951     read-cmd    .choices:nn =
1952       { terse , clear }
1953       {
1954         \int_case:nn { \l_keys_choice_int }
1955         {
1956           {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
1957           {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
1958         }
1959       },
1960     read-cmd    .initial:n = clear,
1961     read-cmd    .value_required:n = true,
1962     read-cmd / unknown .code:n =
1963       \msg_error:nneee { spintent } { unknown-choice }
1964       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1965     upper-case  .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
1966     upper-case  .initial:n = false,
1967     upper-case  .value_required:n = true,
1968     sample-arg  .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
1969     sample-arg  .initial:n = false,
1970     sample-arg  .value_required:n = true,
1971     silent-cmd  .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
1972     silent-cmd  .initial:n = false,
1973     silent-cmd  .value_required:n = true,
1974     unknown     .code:n =
1975       {
1976         \str_set:Nn \l__spintent_command_name_str { sp #1 }
1977         \__spintent_unknown_keys_cmd:n {##1}
1978       },
1979   }
1980 }

```

(End of definition for `result` and others.)

11.26.4 Internal functions for `\spmc`d and `\spmc`m

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its arguments ‘#2’, injecting MathML intents.

```

1981 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1982 {
1983   \seq_set_from_clist:Nn \l__spintent_mc_args_seq {#2}
1984   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
1985   {
1986     \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
1987     { \MathMLintent { :silent } { ##1 } }
1988     \MathMLintent { entre:silent } { ( }
1989     \seq_use:Nnnn \l__spintent_mc_args_seq
1990     { \MathMLintent { _y:silent } { , } }
1991     { \MathMLintent { :silent } { , } }
1992     { \MathMLintent { _y:silent } { , } }
1993   }
1994   {
1995     \MathMLintent { entre } { ( }
1996     \seq_use:Nnnn \l__spintent_mc_args_seq
1997     { \MathMLintent { _y } { , } }
1998     { \MathMLintent { :silent } { , } }
1999     { \MathMLintent { _y } { , } }
2000   }

```

```

2001   \MathMLintent { :silent } { } }
2002 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmc`d and `\spmc`m commands. It takes the command identifier ‘#1’, its full reading string ‘#2’, the local *(keys)* ‘#3’, and the optional values ‘#4’. It dynamically resolves variables using the identifier to build the `MathML` `intent` string (including any prefixes or suffixes) and prints the math operator. If values are provided in ‘#4’, it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```

2003 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2004 {
2005   \keys_set:nn { spintent / sp #1 } { #3 }
2006   \MathMLintent
2007   {
2008     \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2009     { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
2010     \bool_if:cTF { \__spintent_sp #1 _read_terse_bool }
2011     { #1 } { #2 }
2012     \bool_if:cT { \__spintent_sp #1 _silent_cmd_bool }
2013     {
2014       \tl_if_no_value:nTF {#4}
2015       { :silent }
2016       { \bool_if:cT { \__spintent_sp #1 _sample_arg_bool } { :silent } }
2017     }
2018   }
2019   {
2020     \operatorname
2021     {
2022       \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2023       { \text_uppercase:n {#1} } { #1 }
2024     }
2025   }
2026   \tl_if_no_value:nF { #4 }
2027   {
2028     \bool_if:cTF { \__spintent_sp #1 _sample_arg_bool }
2029     {
2030       \__spintent_mc_process_args:nn {#1} {#4}
2031     }
2032     {
2033       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2034       \str_if_eq:VnTF \__spintent_spmcm_spmcd_luaset_error_str { true }
2035       {
2036         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2037       }
2038       {
2039         \__spintent_mc_process_args:nn {#1} {#4}
2040         \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2041         {
2042           \MathMLintent { es-igual-a } { = }
2043           \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2044         }
2045       }
2046     }
2047   }
2048 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc`d The commands `\spmc`d and `\spmc`m act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2049 \NewDocumentCommand \spmc { 0{ } d() }
2050 {
2051   \mode_if_math:TF
2052   {
2053     \group_begin:
2054     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2055     \group_end:

```

```

2056     }
2057     { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
2058   }
2059   \NewDocumentCommand \spmcm { 0{} d() }
2060   {
2061     \mode_if_math:TF
2062     {
2063       \group_begin:
2064         \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2065       \group_end:
2066     }
2067     { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2068   }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spnM_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spnM_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spnM_print_result_bool
\__spintent_spnD_print_result_bool
\__spintent_spnM_silent_cmd_bool
\__spintent_spnD_silent_cmd_bool
\__spintent_spnM_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spnM_print_result_num_tl
\__spintent_spnD_print_result_num_tl
2069 \seq_new:N \__spintent_md_out_seq
2070 \clist_map_inline:nn { nM , nD }
2071 {
2072   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2073   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2074   \bool_new:c { \__spintent_sp #1 _print_result_bool }
2075   \bool_new:c { \__spintent_sp #1 _silent_cmd_bool }
2076   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2077   \tl_new:c { \__spintent_sp #1 _print_result_num_tl }
2078 }

```

(End of definition for `\__spintent_md_out_seq` and others.)

11.27.2 Keys for `\spnM` and `\spnD`

We define the shared (*keys*) `result`, `silent-cmd`, `result-num` and `prefix`.

```

result
silent-cmd
result-num
prefix
sample-arg
2079 \clist_map_inline:nn { nM , nD }
2080 {
2081   \keys_define:nn { spintent / sp #1 }
2082   {
2083     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2084     result      .initial:n = false,
2085     result      .value_required:n = true,
2086     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
2087     silent-cmd .initial:n = false,
2088     silent-cmd .value_required:n = true,
2089     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2090     result-num .initial:n = ,
2091     prefix      .code:n =
2092       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2093     prefix      .initial:n = ,
2094     prefix      .value_required:n = true,
2095     sample-arg .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
2096     sample-arg .initial:n = false,
2097     sample-arg .value_required:n = true,
2098     unknown     .code:n =
2099       {
2100         \str_set:Nn \__spintent_command_name_str { sp #1 }
2101         \__spintent_unknown_keys_cmd:n {##1}
2102       },
2103   }
2104 }

```

(End of definition for `result` and others.)

11.27.3 Definition of error messages for \spnM and \spnD

md-invalid-argument Exception triggered when the argument for multiples or divisors is invalid.

```
2105 \msg_new:nnnn { spintenc } { md-invalid-argument }
2106 { Invalid~argument~provided~to~#1. }
2107 {
2108   The~argument~must~be~a~single~positive~integer. \
2109 }
```

(End of definition for md-invalid-argument.)

11.27.4 Internal functions for \spnM and \spnD

__spintenc_spnM_spnD_process:nnnn

The internal function `\__spintenc_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintenc_spnM_and_D_exec:nnnn`.

```
2110 \cs_new_protected:Nn \__spintenc_spnM_spnD_process:nnnn
2111 {
2112   \keys_set:nn { spintenc / sp #1 } { #4 }
2113   \__spintenc_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2114 }
```

(End of definition for __spintenc_spnM_spnD_process:nnnn.)

__spintenc_spnM_spnD_intent:nn

The internal function `\__spintenc_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```
2115 \cs_new:Nn \__spintenc_spnM_spnD_intent:nn
2116 {
2117   \tl_if_empty:cF { l__spintenc_sp #1 _prefix_tl }
2118   { \tl_use:c { l__spintenc_sp #1 _prefix_tl } - }
2119   \str_case:nnF { #2 }
2120   { { múltiplos } { múltiplos } }
2121   { #2 }
2122 }
```

(End of definition for __spintenc_spnM_spnD_intent:nn.)

__spintenc_spnM_spnD_silent:nn

The internal function `\__spintenc_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2123 \cs_new_protected:Nn \__spintenc_spnM_spnD_silent:nn
2124 {
2125   \MathMLintent { :silent } { \operatorname { #1 } }
2126   \MathMLintent { :silent } { ( }
2127   \MathMLintent { :silent } { \__spintenc_invisible_sep: }
2128   \MathMLintent { :silent } { #2 }
2129   \MathMLintent { :silent } { ) }
2130 }
```

(End of definition for __spintenc_spnM_spnD_silent:nn.)

__spintenc_spnM_spnD_result:n

The internal function `\__spintenc_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2131 \cs_new_protected:Nn \__spintenc_spnM_spnD_result:n
2132 {
2133   \use:c { __spintenc_luafun_calcular_ #1 :nn }
2134   { \l__spintenc_luaseta_part_int_tl }
2135   { \tl_use:c { l__spintenc_sp #1 _print_result_num_tl } }
2136   \MathMLintent { son } { = \{ }
2137   \seq_set_from_clist:Ne
2138   \l__spintenc_md_out_seq { \tl_use:c { l__spintenc_sp #1 _luaseta_result_tl } }
2139   \seq_use:Nnnn \l__spintenc_md_out_seq
2140   { \MathMLintent { _y } { , } }
2141   { \MathMLintent { :silent } { , } }
2142   { \MathMLintent { _y } { , } }
```

```

2143 \str_if_eq:vnT
2144 { \__spintrent_sp #1 \luaset_is_truncated_str } { true }
2145 {
2146   \MathMLintent { :silent } { , } ~ \dots
2147 }
2148 \MathMLintent { :silent } { \} }
2149 }

```

(End of definition for `\__spintrent_spnM_spnD_result:n`.)

`\__spintrent_spnM_spnD_exec:nnnn`

The function `\__spintrent_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the `{\langle argument \rangle}` ‘#1’ (the command name), the `{\langle argument \rangle}` ‘#2’ (the mathematical symbol), the base reading `{\langle argument \rangle}` ‘#3’, and the optional `{\langle argument \rangle}` ‘#4’. If no `{\langle argument \rangle}` is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an `{\langle argument \rangle}` is given, it evaluates the key `sample-arg`. If true, it processes the `{\langle argument \rangle}` directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the `{\langle argument \rangle}` as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```

2150 \cs_new_protected:Nn \__spintrent_spnM_spnD_exec:nnnn
2151 {
2152   \tl_if_novalue:nTF { #4 }
2153   {
2154     \bool_if:cTF { \__spintrent_sp #1 _silent_cmd_bool }
2155     { \__spintrent_spnM_spnD_silent:nn { #2 } { n } }
2156     {
2157       \MathMLintent
2158       { \__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2159       { \operatorname { #2 } ( n ) }
2160     }
2161   }
2162   {
2163     \bool_if:cTF { \__spintrent_sp #1 _sample_arg_bool }
2164     {
2165       \bool_if:cTF { \__spintrent_sp #1 _silent_cmd_bool }
2166       { \__spintrent_spnM_spnD_silent:nn { #2 } { #4 } }
2167       {
2168         \MathMLintent
2169         { \__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2170         { \operatorname { #2 } } ( #4 )
2171       }
2172     }
2173     {
2174       \__spintrent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2175       \str_if_eq:VnTF \__spintrent_luaset_has_natural_str { true }
2176       {
2177         \MathMLintent
2178         { \__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2179         { \operatorname { #2 } } ( #4 )
2180         \bool_if:cT { \__spintrent_sp #1 _print_result_bool }
2181         { \__spintrent_spnM_spnD_result:n { #1 } }
2182       }
2183       {
2184         \msg_error:nne { spintrent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2185       }
2186     }
2187   }
2188 }

```

(End of definition for `\__spintrent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintrent_spnM_spnD_process:nnnnn`, passing along the optional `\keys` ‘#1’ and the numeric `{\langle argument \rangle}` ‘#2’.

```

2189 \NewDocumentCommand \spnM { O{} d() }
2190 {
2191   \mode_if_math:TF
2192   {
2193     \group_begin:
2194     \__spintrent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2195     \group_end:
2196   }

```

```

2197     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2198   }
2199   \NewDocumentCommand \spnD { 0{} d() }
2200   {
2201     \mode_if_math:TF
2202     {
2203       \group_begin:
2204       \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } {#1} {#2}
2205       \group_end:
2206     }
2207     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2208   }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2209 \str_new:N \l__spintent_spang_luaset_grado_str
2210 \str_new:N \l__spintent_spang_luaset_minuto_str
2211 \str_new:N \l__spintent_spang_luaset_segundo_str
2212 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.28.1 Definition of error messages for `\spang`

invalid-angle-format

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2213 \msg_new:nnnn { spintent } { invalid-angle-format }
2214 { Invalid~angle~format~in~#1. }
2215 {
2216   The~argument~'#2'~could~not~be~parsed. \
2217 }

```

(End of definition for `invalid-angle-format`.)

11.28.2 Internal functions for `\spang`

__spintent_spang_process:n

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2218 \cs_new_protected:Nn \__spintent_spang_process:n
2219 {
2220   \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2221   \__spintent_luafun_spang_parse:n { #1 }
2222   \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2223   {
2224     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2225   }
2226   {
2227     \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2228     {
2229       \spunit{ \l__spintent_spang_luaset_grado_str °}
2230     }
2231     \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2232     {
2233       \MathMLintent { minutos : postfix($m) }
2234       {
2235         \MathMLarg { m }
2236         {
2237           \spunit{ \l__spintent_spang_luaset_minuto_str ' }

```

```

2238         }
2239     }
2240 }
2241 \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2242 {
2243     \MathMLintent { segundos : postfix($s) }
2244     {
2245         \MathMLarg { s }
2246         {
2247             \spunit { \l__spintent_spang_luaset_segundo_str " }
2248         }
2249     }
2250 }
2251 }
2252 }

```

(End of definition for `\__spintent_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2253 \NewDocumentCommand \spang { m }
2254 {
2255     \mode_if_math:TF
2256     {
2257         \group_begin:
2258         \__spintent_spang_process:n { #1 }
2259         \group_end:
2260     }
2261     { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2262 }

```

(End of definition for `\spang`. This function is documented on page 21.)

11.29 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.29.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2263 \keys_define:nn { spintent / spnZ }
2264 {
2265     cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2266     read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2267     wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2268     wrap-parens .initial:n = false,
2269     wrap-parens .value_required:n = true,
2270     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2271     read-parens .initial:n = true,
2272     read-parens .value_required:n = true,
2273     unknown .code:n =
2274     {
2275         \str_set:Nn \l__spintent_command_name_str { spnZ }
2276         \__spintent_unknown_keys_cmd:n {#1}
2277     },
2278 }

```

(End of definition for `cifras-sep` and others.)

11.29.2 Definition of error messages

spnz-not-integer Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2279 \msg_new:nnnn { spintent } { spnz-not-integer }
2280 { The~value~'~#1'~is~not~a~valid~integer. }
2281 {

```

```

2282 Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2283 }

```

(End of definition for `spnz-not-integer`.)

11.29.3 Internal functions for `\spnZ`

`\__spintent_spnz_render_parens:` The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2284 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2285 {
2286   \bool_if:NTF \__spintent_spnz_read_parens_bool
2287   {
2288     ( \__spintent_spnz_print_num_start: )
2289   }
2290   {
2291     \MathMLintent { :silent } { ( }
2292     \__spintent_spnz_print_num_start:
2293     \MathMLintent { :silent } { ) }
2294   }
2295 }

```

(End of definition for `\__spintent_spnz_render_parens:`.)

`\__spintent_spnz_process:nn`

The function `\__spintent_spnz_process:nn` first processes the `<keys>` passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnz_print_num_start:`.

```

2296 \cs_new_protected:Nn \__spintent_spnz_process:nn
2297 {
2298   \keys_set:nn { spintent / spnZ } {#1}
2299   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2300   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2301   {
2302     \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2303     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2304     {
2305       \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2306       { \__spintent_spnz_render_parens: }
2307       { \__spintent_spnz_print_num_start: }
2308     }
2309   }
2310   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2311 }

```

(End of definition for `\__spintent_spnz_process:nn`.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2312 \NewDocumentCommand \spnZ { O{} m }
2313 {
2314   \mode_if_math:TF
2315   {
2316     \group_begin:
2317     \__spintent_spnz_process:nn { #1 } { #2 }
2318     \group_end:
2319   }
2320   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2321 }

```

(End of definition for `\spnZ`. This function is documented on page 16.)

11.30 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.30.1 Definition of variables for \spmQ

The variable `\l__spintrent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintrent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spintrent_spmQ_print_scale_int:n`.

The variables `\l__spintrent_spmQ_wrap_parens_l_tl` and `\l__spintrent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintrent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\l__spintrent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2322 \str_new:N \l__spintrent_spmQ_join_word_str
2323 \str_new:N \l__spintrent_spmQ_intent_read_sign_str
2324 \tl_new:N \l__spintrent_spmQ_wrap_parens_l_tl
2325 \tl_new:N \l__spintrent_spmQ_wrap_parens_r_tl
2326 \int_new:N \l__spintrent_saved_luamml_flag_int

```

(End of definition for `\l__spintrent_spmQ_join_word_str` and others.)

11.30.2 Definition of error messages for \spmQ

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\langle argument \rangle}` is not actually an integer.

```

2327 \msg_new:nnnn { spintrent } { spmQ-zero-denominator }
2328 {
2329   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2330 }
2331 {
2332   The~denominator~of~a~fraction~cannot~be~zero.\\
2333 }
2334 \msg_new:nnnn { spintrent } { spmQ-not-integer }
2335 {
2336   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2337 }
2338 {
2339   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2340 }

```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.30.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

2341 \keys_define:nn { spintrent / spmQ }
2342 {
2343   join-word .choices:nn =
2344     { entero , enteros , y }
2345     {
2346       \int_case:nn { \l_keys_choice_int }
2347       {
2348         {1} { \str_set:Nn \l__spintrent_spmQ_join_word_str { entero } }
2349         {2} { \str_set:Nn \l__spintrent_spmQ_join_word_str { enteros } }
2350         {3} { \str_set:Nn \l__spintrent_spmQ_join_word_str { y } }
2351       }
2352     },
2353   join-word .initial:n = y,
2354   join-word .value_required:n = true,
2355   join-word/unknown .code:n =
2356     \msg_error:nnnee { spintrent } { unknown-choice }
2357     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2358   use-spnZ .bool_set:N = \l__spintrent_spmQ_use_spnZ_bool,
2359   use-spnZ .initial:n = true,
2360   use-spnZ .value_required:n = true,
2361   print-dfrac .bool_set:N = \l__spintrent_spmQ_print_dfrac_bool,
2362   print-dfrac .initial:n = false,
2363   print-dfrac .value_required:n = true,
2364   wrap-parens .bool_set:N = \l__spintrent_spmQ_wrap_parens_bool,
2365   wrap-parens .initial:n = false,
2366   wrap-parens .value_required:n = true,

```

```

2367     read-parens      .bool_set:N = \__spintent_spmQ_read_parens_bool,
2368     read-parens      .initial:n = true,
2369     read-parens      .value_required:n = true,
2370     unknown          .code:n =
2371     {
2372         \str_set:Nn \__spintent_command_name_str { spmQ }
2373         \__spintent_unknown_keys_cmd:n {#1}
2374     },
2375 }

```

(End of definition for join-word and others.)

11.30.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LaTeX and sets the variables `\__spintent_spmQ_wrap_parens_l_tl` and `\__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2376 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2377 {
2378     \int_case:nnF { \mathstyle }
2379     {
2380         { 0 }
2381         {
2382             \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2383             \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2384         }
2385         { 1 }
2386         {
2387             \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2388             \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2389         }
2390         { 2 }
2391         {
2392             \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2393             \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2394         }
2395         { 3 }
2396         {
2397             \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2398             \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2399         }
2400     }
2401     {
2402         \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \lparen }
2403         \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \rparen }
2404     }

```

If the key `print-dfrac` is active, we must reset the variables `\__spintent_spmQ_wrap_parens_l_tl` and `\__spintent_spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2405     \bool_if:NT \__spintent_spmQ_print_dfrac_bool
2406     {
2407         \tl_set:Nn \__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2408         \tl_set:Nn \__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2409     }
2410 }

```

(End of definition for __spintent_spmQ_set_wrap_parens_size:.)

__spintent_spmQ_start_wrap_parens:

__spintent_spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\__spintent_spmQ_wrap_parens_l_tl` or `\__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2411 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2412 {

```

```

2413 \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2414 {
2415   \__spinttent_spmQ_set_wrap_parens_size:
2416   \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2417     { \l__spinttent_spmQ_wrap_parens_l_tl }
2418     { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_l_tl } }
2419 }
2420 }
2421 \cs_new_protected:Nn \__spinttent_spmQ_stop_wrap_parens:
2422 {
2423   \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2424   {
2425     \__spinttent_spmQ_set_wrap_parens_size:
2426     \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2427       { \l__spinttent_spmQ_wrap_parens_r_tl }
2428       { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_r_tl } }
2429   }
2430 }

```

(End of definition for __spinttent_spmQ_start_wrap_parens: and __spinttent_spmQ_stop_wrap_parens:.)

__spinttent_spmQ_set_join_word:

The function `\__spinttent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spinttent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spinttent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2431 \cs_new_protected:Nn \__spinttent_spmQ_set_join_word:
2432 {
2433   \str_case:Vn \l__spinttent_spmQ_join_word_str
2434   {
2435     { entero } { \MathMLintent { _entero } { \c__spinttent_invisible_sep_tl } }
2436     { enteros } { \MathMLintent { _enteros } { \c__spinttent_invisible_sep_tl } }
2437     { y } { \MathMLintent { _y- } { \c__spinttent_invisible_sep_tl } }
2438   }
2439 }

```

(End of definition for __spinttent_spmQ_set_join_word:.)

__spinttent_spmQ_print_scale_int:

The function `\__spinttent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spinttent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spinttent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spinttent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spinttent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2440 \cs_new_protected:Nn \__spinttent_spmQ_print_scale_int:
2441 {
2442   \int_set_eq:Nc \l__spinttent_saved_luamml_flag_int { \l__luamml_flag_int }
2443   \int_set:Nn \l__luamml_flag_int { 0 }
2444   \bool_if:NTF \l__spinttent_spmQ_print_dfrac_bool
2445     { \__spinttent_luafun_spmQ_scale_int:Vn \l__spinttent_luaset_part_int_tl { dfraction } }
2446     { \__spinttent_luafun_spmQ_scale_int:Vn \l__spinttent_luaset_part_int_tl { fraction } }
2447   \int_set_eq:cN { \l__luamml_flag_int } \l__spinttent_saved_luamml_flag_int
2448 }

```

(End of definition for __spinttent_spmQ_print_scale_int:.)

__spinttent_spmQ_print_scale_int:n

The function `\__spinttent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in `#1`. It first calls `\__spinttent_luafun_clean_split_and_set:n` on `#1` to parse and populate the shared Lua variables, then checks the status of `\l__spinttent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spinttent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spinttent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spinttent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An

`\__spintent_invisible_sep`: call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2449 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:n
2450 {
2451   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2452   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2453   {
2454     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2455     {
2456       \__spintent_spmQ_print_scale_int:
2457     }
2458     {
2459       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2460       {
2461         \str_set:Nn
2462           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2463       }
2464       {
2465         \str_set:Nn
2466           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2467       }
2468       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2469       {
2470         \l__spintent_luaset_sign_tl \__spintent_invisible_sep:
2471         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2472       }
2473     }
2474   }
2475   { \msg_error:nnn { spintent } { spmQ-not-integer } { #1 } }
2476 }

```

(End of definition for `\__spintent_spmQ_print_scale_int:n`.)

`\__spintent_spmQ_print_spmZ_dfrac:nn`

The function `\__spintent_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spmZ_bool` and `\l__spintent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `print-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2477 \cs_new_protected:Nn \__spintent_spmQ_print_spmZ_dfrac:nn
2478 {
2479   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2480   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { false }
2481   {
2482     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2483   }
2484   \bool_if:NTF \l__spintent_spmQ_use_spmZ_bool
2485   {
2486     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2487     { \spnZ { #1 } } { \spnZ { #2 } }
2488   }
2489   {
2490     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2491     { #1 } { #2 }
2492   }
2493 }

```

(End of definition for `\__spintent_spmQ_print_spmZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spmZ_dfrac:nn` to

typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `__spintent_spmQ_stop_wrap_parens:`.

```

2494 \cs_new_protected:Nn __spintent_spmQ_print:nnn
2495 {
2496   __spintent_spmQ_start_wrap_parens:
2497   __spintent_spmQ_print_scale_int:n { #1 }
2498   __spintent_spmQ_set_join_word:
2499   __spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2500   __spintent_spmQ_stop_wrap_parens:
2501 }

```

(End of definition for `__spintent_spmQ_print:nnn`.)

`__spintent_spmQ_exec:nnnn`

The internal function `__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `__spintent_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2502 \cs_new_protected:Nn __spintent_spmQ_exec:nnnn
2503 {
2504   \keys_set:nn { spintent / spmQ } { #1 }
2505   __spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2506   \str_if_eq:VnTF __spintent_luaaset_has_integer_str { true }
2507   {
2508     \int_compare:nNnTF { __spintent_luaaset_part_int_tl } = { 0 }
2509     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2510     { __spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2511   }
2512   { __spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2513 }

```

(End of definition for `__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```

2514 \NewDocumentCommand \spmQ { O{} m m m }
2515 {
2516   \mode_if_math:TF
2517   {
2518     \group_begin:
2519     __spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2520     \group_end:
2521   }
2522   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2523 }

```

(End of definition for `\spmQ`. This function is documented on page 16.)

11.31 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `__spintent_geo_luaaset_error_str`, `__spintent_geo_luaaset_intent_str` and `__spintent_geo_luaaset_print_tl`, constructing a valid MathCAT intent.

11.31.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

__spintent_geo_luaaset_error_str
__spintent_geo_luaaset_intent_str
__spintent_geo_luaaset_print_tl
__spintent_geo_luaaset_is_name_str
__spintent_geo_luaaset_poly_sym_str
__spintent_geo_luaaset_angulo_case_str
__spintent_geo_luaaset_fig_op_tl
__spintent_geo_luaaset_concept_str
__spintent_geo_luaaset_letra_word_str
__spintent_geo_luaaset_sub_spoken_str
__spintent_geo_luaaset_nombre_es_num_str

```

```

2524 \str_new:N __spintent_geo_luaaset_error_str
2525 \str_new:N __spintent_geo_luaaset_intent_str
2526 \tl_new:N __spintent_geo_luaaset_print_tl
2527 \clist_new:N __spintent_geo_luaaset_points_clist
2528 \str_new:N __spintent_geo_luaaset_is_name_str
2529 \str_new:N __spintent_geo_luaaset_poly_sym_str
2530 \str_new:N __spintent_geo_luaaset_angulo_case_str
2531 \tl_new:N __spintent_geo_luaaset_fig_op_tl
2532 \str_new:N __spintent_geo_luaaset_concept_str
2533 \str_new:N __spintent_geo_luaaset_letra_word_str
2534 \str_new:N __spintent_geo_luaaset_sub_spoken_str
2535 \str_new:N __spintent_geo_luaaset_nombre_es_num_str

```

(End of definition for `\l__spintent_geo_luaset_error_str` and others.)

11.32 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.32.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```
\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
\l__spintent_spPoint_kv_tl
2536 \str_new:N \l__spintent_spPoint_mode_str
2537 \tl_new:N \l__spintent_spPoint_prefix_tl
2538 \tl_new:N \l__spintent_spPoint_intent_arg_tl
2539 \tl_new:N \l__spintent_spPoint_kv_tl
```

(End of definition for `\l__spintent_spPoint_mode_str` and others.)

11.32.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point` The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```
2540 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2541 {
2542   Invalid~point~format~in~\token_to_str:N \spPoint::~'#1'.
2543 }
2544 {
2545   The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2546   subscript~(\_{...})~or~prime~(').\ \
2547 }
```

(End of definition for `spPoint-invalid-point`.)

11.32.3 Definition of keys for `\spPoint`

We define the *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```
read-cmd
prefix
silent-cmd
unknown
2548 \keys_define:nn { spintent / spPoint }
2549 {
2550   read-cmd .choices:nn = { school , mathcat }
2551   {
2552     \int_case:nn { \l_keys_choice_int }
2553     {
2554       {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
2555       {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
2556     }
2557   },
2558   read-cmd .initial:n = school,
2559   read-cmd .value_required:n = true,
2560   read-cmd / unknown .code:n =
2561     \msg_error:nneee { spintent } { unknown-choice }
2562     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2563   prefix .code:n =
2564     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
2565   prefix .initial:n = ,
2566   prefix .value_required:n = true,
2567   silent-cmd .bool_set:N = \l__spintent_spPoint_silent_cmd_bool,
2568   silent-cmd .initial:n = false,
2569   silent-cmd .value_required:n = true,
2570   concept .bool_set:N = \l__spintent_spPoint_concept_bool,
2571   concept .initial:n = true,
2572   concept .value_required:n = true,
2573   unknown .code:n =
2574     {
2575       \str_set:Nn \l__spintent_command_name_str { spPoint }
2576       \__spintent_unknown_keys_cmd:n {#1}
2577     },
2578 }
```

(End of definition for `read-cmd` and others.)

11.32.4 Internal functions for \spPoint

__spintenc_spPoint_print:n

The internal function `\__spintenc_spPoint_print:n` process the $\{\langle argument \rangle\}$ passed in ‘#1’. First, call the function `\__spintenc_luafun_geo_parse_and_set:n` to process the $\{\langle argument \rangle\}$, returning an “error” if the input is invalid. Then, if `silent-cmd` is `true` or `concept` is `false`, set `\l__spintenc_spPont_intent_arg` to `:silent($pt)`; otherwise set it to `punto:prefix($pt)`, prefixed with `prefix` when given. Finally, call `\__spintenc_spPoint_mathml:`.

```

2579 \cs_new_protected:Nn \__spintenc_spPoint_print:n
2580 {
2581   \__spintenc_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2582   \str_if_eq:VnT \l__spintenc_geo_luaset_error_str { true }
2583   {
2584     \msg_error:nne { spintenc } { spPoint-invalid-point } { \exp_not:n {#1} }
2585   }
2586   \bool_if:nTF
2587   {
2588     \bool_lazy_or_p:nn
2589     { \l__spintenc_spPoint_silent_cmd_bool }
2590     { ! \l__spintenc_spPoint_concept_bool }
2591   }
2592   {
2593     \tl_set:Nn \l__spintenc_spPoint_intent_arg_tl { :silent($pt) }
2594   }
2595   {
2596     \tl_set:Ne \l__spintenc_spPoint_intent_arg_tl
2597     {
2598       \tl_if_empty:NF \l__spintenc_spPoint_prefix_tl
2599       {
2600         \tl_use:N \l__spintenc_spPoint_prefix_tl -
2601       }
2602       punto:prefix($pt)
2603     }
2604   }
2605   \__spintenc_spPoint_mathml:
2606 }

```

(End of definition for `\__spintenc_spPoint_print:n`.)

__spintenc_spPoint_mathml:

The function `\__spintenc_spPoint_mathml:` generates the `<mrow>` and prints `\l__spintenc_geo_luaset_print`. `\__spintenc_invisible_sep:` is called first (luamml issue #26).

The kv string is `arg="pt", intent="_-` when `silent-cmd` is `true`; otherwise, for `read-cmd=school` it is `arg="pt", intent="_body-` (literal spelling); for `read-cmd=mathcat` it is `arg="pt"` alone. A single call to `\__spintenc_luafun_wrap_mrow_intent_arg:nn` applies it, with `\l__spintenc_spPont_intent_arg_tl` as the concept `intent`.

```

2607 \cs_new_protected:Nn \__spintenc_spPoint_mathml:
2608 {
2609   \__spintenc_invisible_sep:
2610   \bool_if:NTF \l__spintenc_spPoint_silent_cmd_bool
2611   {
2612     \tl_set:Nn \l__spintenc_spPoint_kv_tl { arg = "pt", intent = "_-" }
2613   }
2614   {
2615     \str_if_eq:VnTF \l__spintenc_spPoint_mode_str { school }
2616     {
2617       \tl_set:Ne \l__spintenc_spPoint_kv_tl
2618       { arg = "pt", intent = "_ \l__spintenc_geo_luaset_intent_str -" }
2619     }
2620     {
2621       \tl_set:Nn \l__spintenc_spPoint_kv_tl { arg = "pt" }
2622     }
2623   }
2624   \__spintenc_luafun_wrap_mrow_intent_arg:nn
2625   { \l__spintenc_spPoint_intent_arg_tl }
2626   { \l__spintenc_spPoint_kv_tl }
2627   { \l__spintenc_geo_luaset_print_tl }
2628 }

```

(End of definition for `\__spintenc_spPoint_mathml:`.)

__spintenc_spPoint_exec:nn

The function `\__spintenc_spPoint_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintenc_spPoint_print:n`.

```

2629 \cs_new_protected:Nn \__spintenc_spPoint_exec:nn

```

```

2630 {
2631   \keys_set:nn { spintent / spPoint } { #1 }
2632   \__spintent_spPoint_print:n { #2 }
2633 }

```

(End of definition for __spintent_spPoint_exec:nn.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nn`.

```

2634 \NewDocumentCommand \spPoint { 0{} m }
2635 {
2636   \mode_if_math:TF
2637   {
2638     \group_begin:
2639     \__spintent_spPoint_exec:nn { #1 } { #2 }
2640     \group_end:
2641   }
2642   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
2643 }

```

(End of definition for `\spPoint`. This function is documented on page 17.)

`\__spintent_mathmlarg_point:nnn` Annotates one point (`#3`) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. `#1` is the reference name (`arg="#1"`); `#2` is the calling command's name, used to look up its own `silent-cmd/read-cmd` variables. Same kv logic as `\__spintent_spPoint_mathml:`. Concept `intent` is passed empty on purpose: the caller applies it once, externally, via a native `\MathMLintent`.

```

2644 \tl_new:N \__spintent_mathmlarg_point_ref_tl
2645 \str_new:N \__spintent_mathmlarg_point_mode_str
2646 \tl_new:N \__spintent_mathmlarg_point_kv_tl
2647 \bool_new:N \__spintent_mathmlarg_point_silent_bool
2648 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2649 {
2650   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
2651   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2652   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_#2_silent_cmd_bool }
2653   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_#2_mode_str }
2654   \__spintent_invisible_sep:
2655   \bool_if:NTF \__spintent_mathmlarg_point_silent_bool
2656   {
2657     \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2658     { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_" }
2659   }
2660   {
2661     \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2662     {
2663       \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2664       { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_luaaset_" }
2665     }
2666     {
2667       \tl_set:Nc \__spintent_mathmlarg_point_kv_tl { arg = "\__spintent_mathmlarg_point_ref_tl" }
2668     }
2669   }
2670   \__spintent_luafun_wrap_mrow_intent_arg:nn
2671   { } % debe ir vacío
2672   { \__spintent_mathmlarg_point_kv_tl }
2673   { \__spintent_geo_luaaset_print_tl }
2674 }
2675 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\__spintent_mathmlarg_point:nnn`.)

11.33 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input `{⟨argument⟩}`: when the argument contains two points (separated by an optional comma), `\overlefttrightarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.33.1 Definition of variables for \sprecta

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

2676 \str_new:N \__spintent_sprecta_mode_str
2677 \tl_new:N \__spintent_sprecta_prefix_tl
2678 \seq_new:N \__spintent_sprecta_pt_seq
2679 \tl_new:N \__spintent_sprecta_refs_tl
2680 \tl_new:N \__spintent_sprecta_intent_arg_tl
2681 \tl_new:N \__spintent_sprecta_kv_tl
2682 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

11.33.2 Definition of error messages for \sprecta

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

2683 \msg_new:nnnn { spintent } { sprecta-invalid-points }
2684 {
2685   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
2686 }
2687 {
2688   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
2689 }

```

(End of definition for `sprecta-invalid-points`.)

11.33.3 Definition of keys for \sprecta

We define the *(keys)* `read-cmd`, `prefix` and `silent-cmd`.

```

2690 \keys_define:nn { spintent / sprecta }
2691 {
2692   read-cmd .choices:nn = { school , mathcat }
2693   {
2694     \int_case:nn { \_keys_choice_int }
2695     {
2696       {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
2697       {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
2698     }
2699   },
2700   read-cmd .initial:n = school,
2701   read-cmd .value_required:n = true,
2702   read-cmd / unknown .code:n =
2703     \msg_error:nneee { spintent } { unknown-choice }
2704     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2705   prefix .code:n =
2706     { \__spintent_sanitize_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
2707   prefix .initial:n = ,
2708   prefix .value_required:n = true,
2709   silent-cmd .bool_set:N = \__spintent_sprecta_silent_cmd_bool,
2710   silent-cmd .initial:n = false,
2711   silent-cmd .value_required:n = true,
2712   concept .bool_set:N = \__spintent_sprecta_concept_bool,
2713   concept .initial:n = true,
2714   concept .value_required:n = true,
2715   unknown .code:n =
2716     {
2717       \str_set:Nn \__spintent_command_name_str { sprecta }
2718       \__spintent_unknown_keys_cmd:n {#1}
2719     },
2720 }

```

(End of definition for `read-cmd` and others.)

11.33.4 Internal functions for \sprecta

The internal function `\__spintent_sprecta_print:n` process the *{(argument)}* passed in ‘#1’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process the *{(argument)}*, returning an “error” if the input is invalid, then calls `\__spintent_sprecta_mathml:`. Unlike `\spPoint`, no `intent` is built here: both branches of `\__spintent_sprecta_mathml:` build their own, since a proper name (m, l, ...) and a list of points need different amounts of it (a single `$pt` versus `$pt1`, `$pt2`, `\ldots`).

```

2721 \cs_new_protected:Nn \__spintent_sprecta_print:n
2722 {

```

```

2723     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
2724     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2725     {
2726         \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
2727     }
2728     \__spintent_sprecta_mathml:
2729 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml: The function __spintent_sprecta_mathml: branches on \l__spintent_geo_luaset_is_name_str. For a proper name (m, l, ...), the logic mirrors \l__spintent_spPoint_mathml: exactly: the concept intent is :silent(\$pt) when silent-cmd is true or concept is false; otherwise it is recta:prefix(\$pt), prefixed with prefix when given. The kv string, however, depends *only* on silent-cmd – concept=false alone must not silence the letter itself: arg="pt", intent="-" when silent; otherwise arg="pt" plus a literal intent for read-cmd=school, or arg="pt" alone for read-cmd=mathcat. A single call to __spintent_luafun_wrap_mrow_intent_arg:nn applies both. For a list of points, \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element, unmerged) is split into \l__spintent_sprecta_pt_seq and walked once to build \l__spintent_sprecta_refs_tl, the comma-separated \$pt1,\$pt2,... reference list, naming each point by position. The concept intent is _-:nofix(refs) when silent-cmd is true or concept is false; otherwise recta:prefix(refs), prefixed as above. It is applied with a native \MathMLintent wrapping \overleftrightharrow, preceded by __spintent_luafun_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>, Luamml issue #26). The sequence is walked a second time, calling __spintent_mathmlarg_point:nnn once per point with its pt*n* reference name and silent-cmd/read-cmd looked up under sprecta: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

2730 \cs_new_protected:Nn \__spintent_sprecta_mathml:
2731 {
2732     \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
2733     {
2734         \__spintent_invisible_sep:
2735         \bool_if:nTF
2736         {
2737             \bool_lazy_or_p:nn
2738             { \l__spintent_sprecta_silent_cmd_bool }
2739             { ! \l__spintent_sprecta_concept_bool }
2740         }
2741         {
2742             \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { :silent($pt) }
2743         }
2744         {
2745             \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
2746             {
2747                 \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2748                 { \tl_use:N \l__spintent_sprecta_prefix_tl - }
2749                 recta:prefix($pt)
2750             }
2751         }
2752         \bool_if:NTF \l__spintent_sprecta_silent_cmd_bool
2753         {
2754             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "-" }
2755         }
2756         {
2757             \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2758             {
2759                 \tl_set:Ne \l__spintent_sprecta_kv_tl
2760                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
2761             }
2762             {
2763                 \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
2764             }
2765         }
2766         \__spintent_luafun_wrap_mrow_intent_arg:nn
2767         { \l__spintent_sprecta_intent_arg_tl }
2768         { \l__spintent_sprecta_kv_tl }
2769         { \l__spintent_geo_luaset_print_tl }
2770     }
2771     {
2772         \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
2773         \tl_clear:N \l__spintent_sprecta_refs_tl

```

```

2774 \int_zero:N \l__spintent_sprecta_pt_idx_int
2775 \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
2776 {
2777   \int_incr:N \l__spintent_sprecta_pt_idx_int
2778   \tl_if_empty:NF \l__spintent_sprecta_refs_tl
2779   { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
2780   \tl_put_right:Ne \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
2781 }
2782 \bool_if:nTF
2783 {
2784   \bool_lazy_or_p:nn
2785   { \l__spintent_sprecta_silent_cmd_bool }
2786   { ! \l__spintent_sprecta_concept_bool }
2787 }
2788 {
2789   \tl_set:Ne \l__spintent_sprecta_intent_arg_tl { _-:nofix( \l__spintent_sprecta_refs_tl
2790 }
2791 {
2792   \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
2793   {
2794     \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2795     { \tl_use:N \l__spintent_sprecta_prefix_tl - }
2796     recta:prefix( \l__spintent_sprecta_refs_tl )
2797   }
2798 }
2799 \__spintent_invisible_sep:
2800 \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
2801 {
2802   \__spintent_luafun_inner_sep:
2803   \overleftrightharrow
2804   {
2805     \int_zero:N \l__spintent_sprecta_pt_idx_int
2806     \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
2807     {
2808       \int_incr:N \l__spintent_sprecta_pt_idx_int
2809       \__spintent_mathmlarg_point:nnn
2810       { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
2811       { sprecta }
2812       { ##1 }
2813     }
2814   }
2815 }
2816 }
2817 }

```

(End of definition for `\__spintent_sprecta_mathml:`)

`\__spintent_sprecta_exec:nn`

The function `\__spintent_sprecta_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *(argument)* passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

2818 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
2819 {
2820   \keys_set:nn { spintent / sprecta } { #1 }
2821   \__spintent_sprecta_print:n { #2 }
2822 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta`

The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

2823 \NewDocumentCommand \sprecta { 0{} m }
2824 {
2825   \mode_if_math:TF
2826   {
2827     \group_begin:
2828     \__spintent_sprecta_exec:nn { #1 } { #2 }
2829     \group_end:
2830   }
2831   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
2832 }

```

(End of definition for `\sprecta`. This function is documented on page 18.)

11.34 The command \sprayo

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, `rayo` or `semirrecta`, both rendered identically with `\overrightarrow`.

11.34.1 Definition of variables for \sprayo

The variable `\l__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
2833 \str_new:N \l__spintent_sprayo_mode_str
2834 \tl_new:N \l__spintent_sprayo_prefix_tl
2835 \str_new:N \l__spintent_sprayo_read_sty_str
```

(End of definition for `\l__spintent_sprayo_mode_str`, `\l__spintent_sprayo_prefix_tl`, and `\l__spintent_sprayo_read_sty_str`.)

11.34.2 Definition of error messages for \sprayo

sprayo-invalid-points

Raised when the argument is not a comma-separated pair of uppercase letters.

```
2836 \msg_new:nnnn { spintent } { sprayo-invalid-points }
2837 {
2838   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
2839 }
2840 {
2841   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
2842 }
```

(End of definition for `sprayo-invalid-points`.)

11.34.3 Definition of keys for \sprayo

read-cmd
prefix
silent-cmd
read-sty
unknown

We define the `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept` and `read-sty`.

```
2843 \keys_define:nn { spintent / sprayo }
2844 {
2845   read-cmd          .choices:nn = { school , mathcat }
2846   {
2847     \int_case:nn { \l_keys_choice_int }
2848     {
2849       {1} { \str_set:Nn \l__spintent_sprayo_mode_str { school } }
2850       {2} { \str_set:Nn \l__spintent_sprayo_mode_str { mathcat } }
2851     }
2852   },
2853   read-cmd          .initial:n = school,
2854   read-cmd          .value_required:n = true,
2855   read-cmd / unknown .code:n =
2856     \msg_error:nneee { spintent } { unknown-choice }
2857     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2858   prefix            .code:n =
2859     { \__spintent_sanitizе_affix:Nn \l__spintent_sprayo_prefix_tl {#1} },
2860   prefix            .initial:n = ,
2861   prefix            .value_required:n = true,
2862   silent-cmd        .bool_set:N = \l__spintent_sprayo_silent_cmd_bool,
2863   silent-cmd        .initial:n = false,
2864   silent-cmd        .value_required:n = true,
2865   concept           .bool_set:N = \l__spintent_sprayo_concept_bool,
2866   concept           .initial:n = true,
2867   concept           .value_required:n = true,
2868   read-sty          .choices:nn = { rayo , semirrecta }
2869   {
2870     \int_case:nn { \l_keys_choice_int }
2871     {
2872       {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
2873       {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
2874     }
2875   },
2876   read-sty          .initial:n = rayo,
2877   read-sty          .value_required:n = true,
2878   read-sty / unknown .code:n =
2879     \msg_error:nneee { spintent } { unknown-choice }
2880     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
2881   unknown           .code:n =
2882   {
```

```

2883         \str_set:Nn \l__spintent_command_name_str { sprayo }
2884         \l__spintent_unknown_keys_cmd:n {#1}
2885     },
2886 }

```

(End of definition for read-cmd and others.)

11.34.4 Internal functions for \spray

Auxiliary variables used by \l__spintent_spray_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1,\$pts2,... reference list.

```

2887 \seq_new:N \l__spintent_spray_pt_seq
2888 \tl_new:N \l__spintent_spray_refs_tl
2889 \tl_new:N \l__spintent_spray_intent_arg_tl
2890 \int_new:N \l__spintent_spray_pt_idx_int

```

(End of definition for \l__spintent_spray_pt_seq and others.)

The internal function \l__spintent_spray_print:n process the $\langle argument \rangle$ passed in ‘#1’. It calls \l__spintent_luafun_geo_parse_and_set:n to process the $\langle argument \rangle$, returning an “error” if the input is invalid, then calls \l__spintent_spray_mathml:. Unlike \sprecta, \spray has no proper-name argument: it always takes a list of points, so no intent is built here.

```

2891 \cs_new_protected:Nn \l__spintent_spray_print:n
2892 {
2893     \l__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
2894     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2895     {
2896         \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
2897     }
2898     \l__spintent_spray_mathml:
2899 }

```

(End of definition for \l__spintent_spray_print:n.)

The function \l__spintent_spray_mathml: splits \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element) into \l__spintent_spray_pt_seq and walks it once to build \l__spintent_spray_refs_tl, the comma-separated \$pts1,\$pts2,... reference list, naming each point by position.

The concept intent is _-:nofix(refs) when silent-cmd is true or concept is false; otherwise it is \l__spintent_spray_read_sty_str:prefix(refs) — the concept word itself (rayo or semirecta) comes from the read-sty key, not a literal, since \spray verbalizes under two names — prefixed with prefix when given. It is applied with a native \MathMLintent wrapping \overrightarrow, preceded by \l__spintent_luafun_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>, luamml issue #26). The sequence is walked a second time, calling \l__spintent_mathmlarg_point:nnn once per point with its ptsn reference name and silent-cmd/read-cmd looked up under sprayo: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

2900 \cs_new_protected:Nn \l__spintent_spray_mathml:
2901 {
2902     \seq_set_from_clist:NN \l__spintent_spray_pt_seq \l__spintent_geo_luaset_points_clist
2903     \tl_clear:N \l__spintent_spray_refs_tl
2904     \int_zero:N \l__spintent_spray_pt_idx_int
2905     \seq_map_inline:Nn \l__spintent_spray_pt_seq
2906     {
2907         \int_incr:N \l__spintent_spray_pt_idx_int
2908         \tl_if_empty:NF \l__spintent_spray_refs_tl
2909         { \tl_put_right:Nn \l__spintent_spray_refs_tl { , } }
2910         \tl_put_right:Ne \l__spintent_spray_refs_tl { $ pts \int_use:N \l__spintent_spray_pt_idx_int }
2911     }
2912     \bool_if:nTF
2913     {
2914         \bool_lazy_or_p:nn
2915         { \l__spintent_spray_silent_cmd_bool }
2916         { ! \l__spintent_spray_concept_bool }
2917     }
2918     {
2919         \tl_set:Ne \l__spintent_spray_intent_arg_tl { _-:nofix( \l__spintent_spray_refs_tl ) }
2920     }
2921     {
2922         \tl_set:Ne \l__spintent_spray_intent_arg_tl
2923         {
2924             \tl_if_empty:NF \l__spintent_spray_prefix_tl

```

```

2925         { \tl_use:N \l__spintent_sprayo_prefix_tl - }
2926         \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
2927     }
2928 }
2929 \__spintent_invisible_sep:
2930 \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
2931 {
2932     \__spintent_luafun_inner_sep:
2933     \overrightarrow
2934     {
2935         \int_zero:N \l__spintent_sprayo_pt_idx_int
2936         \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
2937         {
2938             \int_incr:N \l__spintent_sprayo_pt_idx_int
2939             \__spintent_mathmlarg_point:nnn
2940             { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
2941             { sprayo }
2942             { ##1 }
2943         }
2944     }
2945 }
2946 }

```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn`

The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argument \rangle \}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

2947 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
2948 {
2949     \keys_set:nn { spintent / sprayo } { #1 }
2950     \__spintent_sprayo_print:n { #2 }
2951 }

```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo`

The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

2952 \NewDocumentCommand \sprayo { O{} m }
2953 {
2954     \mode_if_math:TF
2955     {
2956         \group_begin:
2957         \__spintent_sprayo_exec:nn { #1 } { #2 }
2958         \group_end:
2959     }
2960     { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
2961 }

```

(End of definition for `\sprayo`. This function is documented on page 18.)

11.35 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — *lado*, *segmento*, or *trazo*, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.35.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\l__spintent_spside_mode_str` and `\l__spintent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spside_prefix_tl` and `\l__spintent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\l__spintent_spside_read_sty_str` and `\l__spintent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\l__spintent_spside_read_arg_tl` and `\l__spintent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

2962 \clist_map_inline:nn { side , mside }
2963 {

```

```

2964     \str_new:c { l__spintent_sp #1 _mode_str      }
2965     \tl_new:c { l__spintent_sp #1 _prefix_tl      }
2966     \str_new:c { l__spintent_sp #1 _read_sty_str  }
2967     \tl_new:c { l__spintent_sp #1 _read_arg_tl    }
2968 }

```

(End of definition for `l__spintent_spside_mode_str` and others.)

11.35.2 Definition of error messages for `\spside` and `\spmside`

`spside-invalid-points`
`spmside-invalid-points`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘`#1`’ to the command.

```

2969 \clist_map_inline:nn { side , mside }
2970 {
2971     \msg_new:nnnn { spintent } { sp #1 -invalid-points }
2972     {
2973         Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
2974     }
2975     {
2976         See~the~documentation~of~##1~for~the~accepted~point~formats.
2977     }
2978 }

```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.35.3 Definition of keys for `\spside` and `\spmside`

`read-cmd`
`prefix`
`silent-cmd`
`concept`
`read-sty`
`read-arg`
`print-m`
`unknown`

We define the shared `(keys)` `read-cmd`, `prefix`, `silent-cmd`, `concept`, `read-sty`, `read-arg` and `print-m`.

```

2979 \clist_map_inline:nn { side , mside }
2980 {
2981     \keys_define:nn { spintent / sp #1 }
2982     {
2983         read-cmd .choices:nn = { school , mathcat }
2984         {
2985             \int_case:nn { \l_keys_choice_int }
2986             {
2987                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
2988                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
2989             }
2990         },
2991         read-cmd .initial:n = school,
2992         read-cmd .value_required:n = true,
2993         read-cmd / unknown .code:n =
2994             \msg_error:nneee { spintent } { unknown-choice }
2995             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
2996         prefix .code:n =
2997             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
2998         prefix .initial:n = ,
2999         prefix .value_required:n = true,
3000         silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3001         silent-cmd .initial:n = false,
3002         silent-cmd .value_required:n = true,
3003         concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
3004         concept .initial:n = true,
3005         concept .value_required:n = true,
3006         read-sty .choices:nn = { lado , segmento , trazo }
3007         {
3008             \int_case:nn { \l_keys_choice_int }
3009             {
3010                 {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
3011                 {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3012                 {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
3013             }
3014         },
3015         read-sty .initial:n = lado,
3016         read-sty .value_required:n = true,
3017         read-sty / unknown .code:n =
3018             \msg_error:nneee { spintent } { unknown-choice }
3019             { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3020         read-arg .code:n =
3021         {
3022             \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3023         },

```

```

3024         read-arg          .initial:n = ,
3025         read-arg          .value_required:n = true,
3026         print-m           .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3027         print-m           .initial:n = false,
3028         print-m           .value_required:n = true,
3029         unknown           .code:n =
3030         {
3031             \str_set:Nn \l__spintent_command_name_str { sp #1 }
3032             \__spintent_unknown_keys_cmd:n {##1}
3033         },
3034     }
3035 }

```

🌱 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

```

\l__spintent_spside_intent_tl \l__spintent_spside_pt_seq \l__spintent_spside_refs_tl
\l__spintent_spside_intent_arg_tl \l__spintent_spside_pt_idx_int
3036 \tl_new:N \l__spintent_spside_intent_tl
3037 \seq_new:N \l__spintent_spside_pt_seq
3038 \tl_new:N \l__spintent_spside_refs_tl
3039 \tl_new:N \l__spintent_spside_intent_arg_tl
3040 \int_new:N \l__spintent_spside_pt_idx_int

```

(End of definition for `\l__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn` The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘`#1`’ and the $\langle\textit{argument}\rangle$ in ‘`#2`’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘`#2`’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘`medida-del-`’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘`medida-del-`’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_mathml` builds after splitting the points.

```

3041 \cs_new_protected:Nn \__spintent_spside_process:nn
3042 {
3043     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3044     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3045     {
3046         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3047         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3048     }
3049     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3050     {
3051         \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str } }
3052         \str_if_eq:eeT {#1} { mside }
3053         {
3054             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3055         }
3056     }
3057     {
3058         \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3059         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3060         {
3061             \str_if_eq:eeT {#1} { mside }
3062             {
3063                 \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3064             }
3065         }
3066     }
3067     \__spintent_spside_mathml:n { #1 }
3068 }

```

(End of definition for `\__spintent_spside_process:nn`.)

`\__spintent_spside_mathml:n` The function `\__spintent_spside_mathml:n` receives the short command name in ‘`#1`’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1, $pts2, \dots` reference list, naming each point by position.

The concept `intent` is `_(refs)` when `silent-cmd` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given. It is applied with a native `\MathMLintent`, preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).

Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3069 \cs_new_protected:Nn \__spintent_spside_mathml:n
3070 {
3071   \seq_set_from_clist:NN \__spintent_spside_pt_seq \__spintent_geo_lua_set_points_clist
3072   \tl_clear:N \__spintent_spside_refs_tl
3073   \int_zero:N \__spintent_spside_pt_idx_int
3074   \seq_map_inline:Nn \__spintent_spside_pt_seq
3075   {
3076     \int_incr:N \__spintent_spside_pt_idx_int
3077     \tl_if_empty:NF \__spintent_spside_refs_tl
3078     { \tl_put_right:Nn \__spintent_spside_refs_tl { , } }
3079     \tl_put_right:Ne \__spintent_spside_refs_tl { $ pts \int_use:N \__spintent_spside_pt_idx_int }
3080   }
3081   \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
3082   {
3083     \tl_set:Ne \__spintent_spside_intent_arg_tl { _ ( \__spintent_spside_refs_tl ) }
3084   }
3085   {
3086     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
3087     {
3088       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
3089       {
3090         \tl_set:Ne \__spintent_spside_intent_arg_tl
3091         {
3092           \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
3093           { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
3094           \__spintent_spside_intent_tl :prefix( \__spintent_spside_refs_tl )
3095         }
3096       }
3097       {
3098         \tl_set:Ne \__spintent_spside_intent_arg_tl { _ ( \__spintent_spside_refs_tl ) }
3099       }
3100     }
3101     {
3102       \tl_set:Ne \__spintent_spside_intent_arg_tl
3103       {
3104         \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
3105         { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
3106         \__spintent_spside_intent_tl :prefix( \__spintent_spside_refs_tl )
3107       }
3108     }
3109   }
3110   \__spintent_invisible_sep: % (luamml issues #26)
3111   \MathMLintent { \__spintent_spside_intent_arg_tl }
3112   {
3113     \__spintent_luafun_inner_sep:
3114     \str_if_eq:eeTF {#1} { side }
3115     {
3116       \overline { \__spintent_spside_points_loop:n {#1} }
3117     }
3118     {
3119       \bool_if:cTF { \__spintent_sp #1 _print_m_bool }
3120       {
3121         \__spintent_luafun_wrap_mrow_intent_arg:nn
3122         { } { intent = "silent" }
3123         { \operatorname { m } } }
3124       \overline { \__spintent_spside_points_loop:n {#1} }
3125     }

```

```

3126         {
3127         \__spintent_spside_points_loop:n {#1}
3128         }
3129     }
3130 }
3131 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (side, msides with and without `print-m`); it takes the short command name in ‘#1’ again since a separate macro does not inherit its caller’s own #1.

```

3132 \cs_new_protected:Nn \__spintent_spside_points_loop:n
3133 {
3134     \int_zero:N \__spintent_spside_pt_idx_int
3135     \seq_map_inline:Nn \__spintent_spside_pt_seq
3136     {
3137         \int_incr:N \__spintent_spside_pt_idx_int
3138         \__spintent_mathmlarg_point:nnn
3139         { pts \int_use:N \__spintent_spside_pt_idx_int }
3140         { sp #1 }
3141         { ##1 }
3142     }
3143 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn`

The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spside_process:nn`.

```

3144 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3145 {
3146     \keys_set:nn { spintent / sp #1 } { #2 }
3147     \__spintent_spside_process:nn { #1 } { #3 }
3148 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

3149 \NewDocumentCommand \spside { 0{} m }
3150 {
3151     \mode_if_math:TF
3152     {
3153         \group_begin:
3154         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3155         \group_end:
3156     }
3157     { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3158 }
3159 \NewDocumentCommand \spmside { 0{} m }
3160 {
3161     \mode_if_math:TF
3162     {
3163         \group_begin:
3164         \__spintent_spside_exec:nnn { msideside } { #1 } { #2 }
3165         \group_end:
3166     }
3167     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3168 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 18.)

11.36 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside`/`\spmside`. The key `print-m` chooses between the `\angle`/`\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle`/`\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.36.1 Definition of variables for \spangle and \spmangle

```
\__spintent_spangle_mode_str
\__spintent_spmangle_mode_str
\__spintent_spangle_prefix_tl
\__spintent_spmangle_prefix_tl
```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\__spintent_spangle_mode_str` and `\__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spangle_prefix_tl` and `\__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
3169 \clist_map_inline:nn { angle , mangle }
3170 {
3171   \str_new:c { \__spintent_sp #1 _mode_str }
3172   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3173 }
```

(End of definition for `\__spintent_spangle_mode_str` and others.)

11.36.2 Definition of error messages for \spangle and \spmangle

```
spangle-invalid-arg
spmangle-invalid-arg
```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```
3174 \clist_map_inline:nn { angle , mangle }
3175 {
3176   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3177   {
3178     Invalid~format~in~##1:~'##2'~\msg_line_context:.
3179   }
3180   {
3181     See~the~documentation~of~##1~for~the~accepted~argument~forms.
3182   }
3183 }
```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.36.3 Definition of keys for \spangle and \spmangle

```
read-cmd
prefix
recto
print-m
silent-cmd
concept
unknown
```

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `recto`, `print-m`, `silent-cmd` and `concept`,

```
3184 \clist_map_inline:nn { angle , mangle }
3185 {
3186   \keys_define:nn { spintent / sp #1 }
3187   {
3188     read-cmd .choices:nn = { school , mathcat }
3189     {
3190       \int_case:nn { \_keys_choice_int }
3191       {
3192         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
3193         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
3194       }
3195     },
3196     read-cmd .initial:n = school,
3197     read-cmd .value_required:n = true,
3198     read-cmd / unknown .code:n =
3199     \msg_error:nneee { spintent } { unknown-choice }
3200     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3201     prefix .code:n =
3202     { \__spintent_sanitizize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
3203     prefix .initial:n = ,
3204     prefix .value_required:n = true,
3205     recto .bool_set:c = { \__spintent_sp #1 _recto_bool },
3206     recto .initial:n = false,
3207     recto .value_required:n = true,
3208     print-m .bool_set:c = { \__spintent_sp #1 _print_m_bool },
3209     print-m .initial:n = false,
3210     print-m .value_required:n = true,
3211     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
3212     silent-cmd .initial:n = false,
3213     silent-cmd .value_required:n = true,
3214     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
3215     concept .initial:n = true,
3216     concept .value_required:n = true,
3217     unknown .code:n =
3218     {
3219       \str_set:Nn \__spintent_command_name_str { sp #1 }
3220       \__spintent_unknown_keys_cmd:n {##1}
3221     },
3222 }
```

```

3222     }
3223 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for `\spangle` and `\spmangle`

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3224 \cs_new_protected:Nn \__spintent_spangle_sym:n
3225 {
3226   \str_if_eq:eeTF {#1} { mangle }
3227   {
3228     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3229     {
3230       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3231       { \__spintent_right_angle_sqr: } { \angle }
3232     }
3233     {
3234       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3235       { \measuredrightangle } { \measuredangle }
3236     }
3237   }
3238   {
3239     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3240     { \__spintent_right_angle_sqr: } { \angle }
3241   }
3242 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_lua` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:`).

```

3243 \tl_new:N \l__spintent_spangle_intent_tl
3244 \seq_new:N \l__spintent_spangle_pt_seq
3245 \tl_new:N \l__spintent_spangle_refs_tl
3246 \tl_new:N \l__spintent_spangle_intent_arg_tl
3247 \int_new:N \l__spintent_spangle_pt_idx_int
3248 \str_new:N \l__spintent_spangle_kv_tl
3249 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in `#1` and the $\langle argument \rangle$ in `#2`. It calls `\__spintent_luafun_geo_angulo_parse_and_set:n` to process `#2`, returning an “error” if invalid, then sets `\l__spintent_spangle_intent_tl` to ángulo (or medida-del-ángulo for `\spmangle`), suffixed with ‘-recto’ when `recto` is true. No `intent` is built here: it depends on `\l__spintent_geo_lua` set `angulo_case_str` (tres-puntos, un-punto or nombre), which `\__spintent_spangle_mathml:n` branches on.

```

3250 \cs_new_protected:Nn \__spintent_spangle_process:nn
3251 {
3252   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3253   \str_if_eq:VnT \l__spintent_geo_lua_set_error_str { true }
3254   {
3255     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3256     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3257   }
3258   \str_if_eq:eeTF {#1} { mangle }
3259   {
3260     \tl_set:Nn \l__spintent_spangle_intent_tl { medida-del-ángulo }
3261   }
3262   {
3263     \tl_set:Nn \l__spintent_spangle_intent_tl { ángulo }
3264   }
3265   \bool_if:cT { l__spintent_sp #1 _recto_bool }
3266   {
3267     \tl_put_right:Nn \l__spintent_spangle_intent_tl { -recto }
3268   }
3269   \__spintent_spangle_mathml:n {#1}
3270 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n` For tres-puntos/un-punto, a single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(refs)` — the same shape as `\__spintent_spside_mathml:n`. The earlier split into two sibling `\MathMLintent` calls worked around a MathCAT bug with several explicit `$`-references in `:prefix(...)` that only affects the demo build; the stable NVDA/updated Acrobat Reader combination reads this shape correctly, so the split is no longer needed and both commands now share the same structure.

For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\__spintent_spangle_sym:n` inside the same `\MathMLintent`, exactly as `\__spintent_spside_mathml:n` does for its own `m`. `nombre` is unchanged (see below): it never references more than one thing, so it was never affected by the bug this simplifies away from.

```

3271 \cs_new_protected:Nn \__spintent_spangle_mathml:n
3272 {
3273   \__spintent_invisible_sep: % (luamml issues #26)
3274   \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
3275   {
3276     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3277     {
3278       \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3279     }
3280     {
3281       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3282       {
3283         \tl_set:Ne \l__spintent_spangle_intent_arg_tl
3284         {
3285           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3286           { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3287           \l__spintent_spangle_intent_tl :prefix
3288         }
3289       }
3290       {
3291         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3292       }
3293     }
3294     \tl_set:Ne \l__spintent_spangle_kv_tl
3295     { intent = "\l__spintent_spangle_intent_arg_tl" }
3296     \__spintent_luafun_wrap_mrow_intent_arg:nn
3297     { } { \l__spintent_spangle_kv_tl }
3298     { \__spintent_spangle_sym:n {#1} }
3299     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3300     {
3301       \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = "_-" }
3302       { \l__spintent_geo_luaset_print_tl }
3303     }
3304     {
3305       \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
3306       {
3307         \tl_set:Ne \l__spintent_spangle_kv_tl
3308         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3309         \__spintent_luafun_wrap_mrow_intent_arg:nn
3310         { } { \l__spintent_spangle_kv_tl }
3311         { \l__spintent_geo_luaset_print_tl }
3312       }
3313       { \l__spintent_geo_luaset_print_tl }
3314     }
3315   }
3316   {
3317     \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
3318     \tl_clear:N \l__spintent_spangle_refs_tl
3319     \int_zero:N \l__spintent_spangle_pt_idx_int
3320     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
3321     {
3322       \int_incr:N \l__spintent_spangle_pt_idx_int
3323       \tl_if_empty:NF \l__spintent_spangle_refs_tl
3324       { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
3325       \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_
3326     }
3327     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3328     {

```

```

3329         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3330     }
3331     {
3332         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3333         {
3334             \tl_set:Ne \l__spintent_spangle_intent_arg_tl
3335             {
3336                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3337                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3338                 \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
3339             }
3340         }
3341         {
3342             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3343         }
3344     }
3345     \bool_set_false:N \l__spintent_spangle_print_m_now_bool
3346     \str_if_eq:eeT {#1} { mangle }
3347     {
3348         \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3349         { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
3350     }
3351     \MathMLintent { \l__spintent_spangle_intent_arg_tl }
3352     {
3353         \__spintent_luafun_inner_sep:
3354         \bool_if:NT \l__spintent_spangle_print_m_now_bool
3355         {
3356             \__spintent_luafun_wrap_mrow_intent_arg:nn
3357             { } { intent = ":silent" }
3358             { \operatorname { m } }
3359         }
3360         \__spintent_luafun_wrap_mrow_intent_arg:nn
3361         { } { intent = ":silent" }
3362         { \__spintent_spangle_sym:n {#1} }
3363         \int_zero:N \l__spintent_spangle_pt_idx_int
3364         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
3365         {
3366             \int_incr:N \l__spintent_spangle_pt_idx_int
3367             \__spintent_mathmlarg_point:nnn
3368             { pt \int_use:N \l__spintent_spangle_pt_idx_int }
3369             { sp#1 }
3370             { ##1 }
3371         }
3372     }
3373 }
3374 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn` The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

3375 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
3376 {
3377     \keys_set:nn { spintent / sp #1 } { #2 }
3378     \__spintent_spangle_process:nn { #1 } { #3 }
3379 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle` `\spmangle` Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

3380 \NewDocumentCommand \spangle { 0{} m }
3381 {
3382     \mode_if_math:TF
3383     {
3384         \group_begin:
3385         \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
3386         \group_end:
3387     }
3388     { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }

```

```

3389   }
3390   \NewDocumentCommand \spmangle { 0{} m }
3391   {
3392     \mode_if_math:TF
3393     {
3394       \group_begin:
3395       \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
3396       \group_end:
3397     }
3398     { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
3399   }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 20.)

11.37 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.37.1 Definition of variables for `\sparc` and `\spmarc`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```

3400 \clist_map_inline:nn { arc , marc }
3401 {
3402   \str_new:c { \__spintent_sp #1 _mode_str }
3403   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3404 }

```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.37.2 Definition of error messages for `\sparc` and `\spmarc`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

3405 \clist_map_inline:nn { arc , marc }
3406 {
3407   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3408   {
3409     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3410   }
3411   {
3412     See~the~documentation~of~##1~for~the~accepted~point~formats.
3413   }
3414 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.37.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

read-cmd 3415 \clist_map_inline:nn { arc , marc }
prefix   3416 {
silent-cmd 3417   \keys_define:nn { spintent / sp #1 }
concept    3418   {
unknown    3419     read-cmd .choices:nn = { school , mathcat }
           3420     {
           3421       \int_case:nn { \l_keys_choice_int }
           3422       {
           3423         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
           3424         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
           3425       }
           3426     },
           3427     read-cmd .initial:n = school,
           3428     read-cmd .value_required:n = true,
           3429     read-cmd / unknown .code:n =
           3430     \msg_error:nneee { spintent } { unknown-choice }
           3431     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
           3432     prefix .code:n =
           3433     { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
           3434     prefix .initial:n = ,
           3435     prefix .value_required:n = true,

```

```

3436     silent-cmd      .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3437     silent-cmd      .initial:n  = false,
3438     silent-cmd      .value_required:n = true,
3439     concept         .bool_set:c = { l__spintent_sp #1 _concept_bool },
3440     concept         .initial:n  = true,
3441     concept         .value_required:n = true,
3442     unknown         .code:n =
3443     {
3444         \str_set:Nn \l__spintent_command_name_str { sp #1 }
3445         \__spintent_unknown_keys_cmd:n {##1}
3446     },
3447 }
3448 }

```

🔗 The key `print-m` is defined for both commands for simplicity, but has no effect on `\sparc`.

(End of definition for `read-cmd` and others.)

11.37.4 Internal functions for `\sparc` and `\spmarc`

`\__spintent_sparc_wide_arc:n`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of `TeX-SX`.

```

3449 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
3450 {
3451     \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
3452 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

`\__spintent_sparc_print_arc:n`

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

3453 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
3454 {
3455     \cs_if_exist:NTF \widearc
3456     { \widearc {#1} }
3457     { \__spintent_sparc_wide_arc:n {#1} }
3458 }

```

(End of definition for `\__spintent_sparc_print_arc:n`.)

`\l__spintent_sparc_intent_tl`

`\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list used in the concept `intent`.

```

3459 \tl_new:N \l__spintent_sparc_intent_tl
3460 \seq_new:N \l__spintent_sparc_pt_seq
3461 \tl_new:N \l__spintent_sparc_refs_tl
3462 \tl_new:N \l__spintent_sparc_intent_arg_tl
3463 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the `{\argument}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_math` builds after splitting the points.

```

3464 \cs_new_protected:Nn \__spintent_sparc_process:nn
3465 {
3466     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3467     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3468     {
3469         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3470         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3471     }
3472     \str_if_eq:eeTF {#1} { marc }
3473     { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
3474     { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
3475     \__spintent_sparc_mathml:n {#1}
3476 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each point by position. The concept `intent` is `_` when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent` prefixed with `prefix` when given. It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

3477 \cs_new_protected:Nn \__spintent_sparc_mathml:n
3478 {
3479   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_lua_set_points_clist
3480
3481   \tl_clear:N \l__spintent_sparc_refs_tl
3482   \int_zero:N \l__spintent_sparc_pt_idx_int
3483   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
3484   {
3485     \int_incr:N \l__spintent_sparc_pt_idx_int
3486     \tl_if_empty:NF \l__spintent_sparc_refs_tl
3487     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
3488     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
3489   }
3490
3491   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3492   {
3493     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
3494   }
3495   {
3496     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3497     {
3498       \tl_set:Ne \l__spintent_sparc_intent_arg_tl
3499       {
3500         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3501         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3502         \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
3503       }
3504     }
3505     {
3506       \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
3507     }
3508   }
3509
3510   \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
3511   \MathMLintent { \l__spintent_sparc_intent_arg_tl }
3512   {
3513     \__spintent_luafun_inner_sep:
3514     \str_if_eq:eeT {#1} { marc }
3515     {
3516       \__spintent_luafun_wrap_mrow_intent_arg:nn
3517       { } { intent = "silent" }
3518       { \operatorname { m } } }
3519   }
3520   \__spintent_sparc_print_arc:n
3521   {
3522     \int_zero:N \l__spintent_sparc_pt_idx_int
3523     \seq_map_inline:Nn \l__spintent_sparc_pt_seq
3524     {
3525       \int_incr:N \l__spintent_sparc_pt_idx_int
3526       \__spintent_mathmlarg_point:nnn
3527       { pt \int_use:N \l__spintent_sparc_pt_idx_int }
3528       { sp #1 }
3529       { ##1 }
3530     }
3531   }
3532 }
3533 }

```

(End of definition for `\__spintent_sparc_mathml:n`.)

`\__spintent_sparc_exec:nnn`

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’,

the optional $\langle keys \rangle$ passed in ‘#2’ and the $\langle argument \rangle$ passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

3534 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
3535 {
3536     \keys_set:nn { spintent / sp #1 } { #2 }
3537     \__spintent_sparc_process:nn { #1 } { #3 }
3538 }

```

(End of definition for `\__spintent_sparc_exec:nnn`.)

`\sparc` Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

3539 \NewDocumentCommand \sparc { 0{} m }
3540 {
3541     \mode_if_math:TF
3542     {
3543         \group_begin:
3544         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
3545         \group_end:
3546     }
3547     { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
3548 }
3549 \NewDocumentCommand \spmarc { 0{} m }
3550 {
3551     \mode_if_math:TF
3552     {
3553         \group_begin:
3554         \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
3555         \group_end:
3556     }
3557     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
3558 }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 21.)

11.38 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\l__spintent_geo_luaset_intent_str` (the full *intent*, with the figure name).

11.38.1 Definition of variables for `\spPoly`

`\l__spintent_spPoly_mode_str` The variable `\l__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

3559 \str_new:N \l__spintent_spPoly_mode_str
3560 \tl_new:N \l__spintent_spPoly_prefix_tl

```

(End of definition for `\l__spintent_spPoly_mode_str` and `\l__spintent_spPoly_prefix_tl`.)

11.38.2 Definition of error messages for `\spPoly`

`spPoly-invalid-points` Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

3561 \msg_new:nnnn { spintent } { spPoly-invalid-points }
3562 {
3563     Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
3564 }
3565 {
3566     See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
3567 }

```

(End of definition for `spPoly-invalid-points`.)

11.38.3 Definition of keys for `\spPoly`

`read-cmd` We define the $\langle keys \rangle$ `read-cmd`, `prefix`, `print-sym`, `silent-cmd` and `concept`.

```

3568 \keys_define:nn { spintent / spPoly }
3569 {
3570     read-cmd .choices:nn = { school , mathcat }
3571     {
3572         \int_case:nn { \l_keys_choice_int }
3573         {
3574             {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
3575             {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } } }

```

```

3576     }
3577   },
3578   read-cmd          .initial:n = school,
3579   read-cmd          .value_required:n = true,
3580   read-cmd / unknown .code:n =
3581     \msg_error:nneee { spintent } { unknown-choice }
3582     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3583   prefix            .code:n =
3584     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
3585   prefix            .initial:n = ,
3586   prefix            .value_required:n = true,
3587   print-sym         .bool_set:N = \l__spintent_spPoly_print_sym_bool,
3588   print-sym         .initial:n = true,
3589   print-sym         .value_required:n = true,
3590   silent-cmd        .bool_set:N = \l__spintent_spPoly_silent_cmd_bool,
3591   silent-cmd        .initial:n = false,
3592   silent-cmd        .value_required:n = true,
3593   concept           .bool_set:N = \l__spintent_spPoly_concept_bool,
3594   concept           .initial:n = true,
3595   concept           .value_required:n = true,
3596   unknown           .code:n =
3597     {
3598       \str_set:Nn \l__spintent_command_name_str { spPoly }
3599       \__spintent_unknown_keys_cmd:n {#1}
3600     },
3601   }

```

(End of definition for `read-cmd` and others.)

11.38.4 Internal functions for `\spPoly`

`\l__spintent_spPoly_pt_seq` Scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, \dots` reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int
3602 \seq_new:N \l__spintent_spPoly_pt_seq
3603 \tl_new:N \l__spintent_spPoly_refs_tl
3604 \tl_new:N \l__spintent_spPoly_intent_arg_tl
3605 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for `\l__spintent_spPoly_pt_seq` and others.)

`\__spintent_spPoly_sym:` Unchanged: reads `\l__spintent_geo_luaset_poly_sym_str` and typesets the corresponding symbol.

```

3606 \cs_new_protected:Nn \__spintent_spPoly_sym:
3607 {
3608   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
3609   {
3610     { triangle } { \triangle }
3611     { square } { \mdlgwhtsquare }
3612   }
3613 }

```

(End of definition for `\__spintent_spPoly_sym:`.)

`\__spintent_spPoly_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spPoly_pt_seq` and walks it once to build `\l__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, \dots` reference list, naming each vertex by position.

```

3614 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
3615 {
3616   \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
3617   \tl_clear:N \l__spintent_spPoly_refs_tl
3618   \int_zero:N \l__spintent_spPoly_pt_idx_int
3619   \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
3620   {
3621     \int_incr:N \l__spintent_spPoly_pt_idx_int
3622     \tl_if_empty:NF \l__spintent_spPoly_refs_tl
3623     { \tl_put_right:Nn \l__spintent_spPoly_refs_tl { , } }
3624     \tl_put_right:Ne \l__spintent_spPoly_refs_tl { $ pt \int_use:N \l__spintent_spPoly_pt_idx_int }
3625   }
3626 }

```

(End of definition for `\__spintent_spPoly_build_refs:`.)

```

\__spintrent_spPoly_build_concept: Builds \__spintrent_spPoly_intent_arg_tl, independently of print-sym: _(refs) when silent-
cmd is true or concept is false; otherwise \__spintrent_geo_luaset_concept_str:prefix(refs),
prefixed with prefix when given. Must be called after \__spintrent_spPoly_build_refs:.

3627 \cs_new_protected:Nn \__spintrent_spPoly_build_concept:
3628 {
3629   \bool_if:NTF \__spintrent_spPoly_silent_cmd_bool
3630   {
3631     \tl_set:Ne \__spintrent_spPoly_intent_arg_tl { _( \__spintrent_spPoly_refs_tl ) }
3632   }
3633   {
3634     \bool_if:NTF \__spintrent_spPoly_concept_bool
3635     {
3636       \tl_set:Ne \__spintrent_spPoly_intent_arg_tl
3637       {
3638         \tl_if_empty:NF \__spintrent_spPoly_prefix_tl
3639         { \tl_use:N \__spintrent_spPoly_prefix_tl - }
3640         \__spintrent_geo_luaset_concept_str :prefix( \__spintrent_spPoly_refs_tl )
3641       }
3642     }
3643     {
3644       \tl_set:Ne \__spintrent_spPoly_intent_arg_tl { _( \__spintrent_spPoly_refs_tl ) }
3645     }
3646   }
3647 }

```

(End of definition for `\__spintrent_spPoly_build_concept:`)

`\__spintrent_spPoly_points_loop:` Walks `\__spintrent_spPoly_pt_seq` a second time, calling `\__spintrent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

3648 \cs_new_protected:Nn \__spintrent_spPoly_points_loop:
3649 {
3650   \int_zero:N \__spintrent_spPoly_pt_idx_int
3651   \seq_map_inline:Nn \__spintrent_spPoly_pt_seq
3652   {
3653     \int_incr:N \__spintrent_spPoly_pt_idx_int
3654     \__spintrent_mathmlarg_point:nnn
3655     { pt \int_use:N \__spintrent_spPoly_pt_idx_int }
3656     { spPoly }
3657     { ##1 }
3658   }
3659 }

```

(End of definition for `\__spintrent_spPoly_points_loop:`)

`\__spintrent_spPoly_process:n` Calls `\__spintrent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintrent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintrent_spPoly_mathml:` builds after splitting the vertices.

```

3660 \cs_new_protected:Nn \__spintrent_spPoly_process:n
3661 {
3662   \__spintrent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
3663   \str_if_eq:VnT \__spintrent_geo_luaset_error_str { true }
3664   {
3665     \msg_error:nne { spintrent } { spPoly-invalid-points }
3666     { \exp_not:n {#1} }
3667   }
3668   \__spintrent_spPoly_mathml:
3669 }

```

(End of definition for `\__spintrent_spPoly_process:n`)

`\__spintrent_spPoly_mathml:` Builds the reference list and the concept `intent` (`\__spintrent_spPoly_build_refs:`, `\__spintrent_spPoly_build_concept:`) then wraps everything in a single native `\MathMLintent`, preceded by `\__spintrent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26). `print-sym` and `concept/silent-cmd` are independent: the former only decides whether the glyph (`\__spintrent_spPoly_sym:`, silenced with `\__spintrent_luafun_wrap_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\__spintrent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

3670 \cs_new_protected:Nn \__spintrent_spPoly_mathml:

```

```

3671 {
3672   \__spintent_invisible_sep: % (luamml issues #26)
3673   \__spintent_spPoly_build_refs:
3674   \__spintent_spPoly_build_concept:
3675   \MathMLintent { \__spintent_spPoly_intent_arg_tl }
3676   {
3677     \__spintent_luafun_inner_sep:
3678     \bool_if:NT \__spintent_spPoly_print_sym_bool
3679     {
3680       \str_if_empty:NF \__spintent_geo_luaset_poly_sym_str
3681       {
3682         \__spintent_luafun_wrap_mrow_intent_arg:nn
3683         { } { intent = ":silent" }
3684         { \__spintent_spPoly_sym: }
3685       }
3686     }
3687     \__spintent_spPoly_points_loop:
3688   }
3689 }

```

(End of definition for __spintent_spPoly_mathml:.)

__spintent_spPoly_exec:nn The function __spintent_spPoly_exec:nn receives the optional keys in #1 and the argument in #2, delegating the rendering to __spintent_spPoly_process:n.

```

3690 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
3691 {
3692   \keys_set:nn { spintent / spPoly } { #1 }
3693   \__spintent_spPoly_process:n { #2 }
3694 }

```

(End of definition for __spintent_spPoly_exec:nn.)

\spPoly The public command \spPoly enforces *math mode*, opens a local group, and delegates execution to __spintent_spPoly_exec:nn.

```

3695 \NewDocumentCommand \spPoly { 0 } { m }
3696 {
3697   \mode_if_math:TF
3698   {
3699     \group_begin:
3700     \__spintent_spPoly_exec:nn { #1 } { #2 }
3701     \group_end:
3702   }
3703   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
3704 }

```

(End of definition for \spPoly. This function is documented on page 22.)

11.39 The commands \spAfig and \spPfig

The user commands \spAfig and \spPfig typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by \spcmd/\spmc. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in __spintent_geo_luaset_poly_sym_str, and `expl3` decides whether to wrap it in \MathMLintent{<:silent>} — the same mechanism already used by \spPoly.

11.39.1 Definition of variables for \spAfig and \spPfig

__spintent_geo_final_print_tl Scratch variables used by __spintent_spfig_process:nn to assemble the visual and the intent. __spintent_geo_final_print_tl holds the final visual placed under \mathcal's subscript. The final literal intent and the concept forwarded when `read-cmd` is `mathcat`.

```

3705 \tl_new:N \__spintent_geo_final_print_tl

```

(End of definition for __spintent_geo_final_print_tl.)

__spintent_spAfig_mode_str These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

3706 \clist_map_inline:nn { Afig , Pfig }
3707 {
3708   \str_new:c { \__spintent_sp #1 _mode_str }
3709   \str_new:c { \__spintent_sp #1 _print_sym_str }
3710   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3711   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
3712 }

```

(End of definition for __spintent_spAfig_mode_str and others.)

11.39.2 Definition of error messages for \spAfig and \spPfig

spAfig-invalid-points All eight messages are generated by the same map, using the same ##1/##2 doubling convention as \spside/\spmside. sp #1-invalid-points is currently unused by __spintent_spfig_process:nn, which raises only sp #1-invalid-arg for any parse failure, regardless of whether a comma was present. sp #1-sym-letra-conflict and sp #1-sym-cardinality-conflict are raised when print-sym is forced to a value incompatible with the actual argument.

```

3713 \clist_map_inline:nn { Afig , Pfig }
3714 {
3715     \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3716     {
3717         Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3718     }
3719     {
3720         See~the~documentation~of~##1~for~the~accepted~point~formats.
3721     }
3722     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3723     {
3724         Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
3725     }
3726     {
3727         See~the~documentation~of~##1~for~the~accepted~argument~forms.
3728     }
3729     \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
3730     {
3731         print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
3732     }
3733     {
3734         See~the~documentation~of~##1~for~details~on~print-sym.
3735     }
3736     \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
3737     {
3738         print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
3739     }
3740     {
3741         See~the~documentation~of~##1~for~details~on~print-sym.
3742     }
3743 }

```

(End of definition for spAfig-invalid-points and others.)

11.39.3 Definition of keys for \spAfig and \spPfig

read-cmd The key print-sym chooses the visual symbol: auto (default) infers it from the vertex count (three points give \triangle, four give \mdlgwhtsquare, five or more give none); triangle/square force one, requiring the vertex count to match; none omits it, even with vertices. With a bare number/letter argument, auto and none behave the same (no symbol); triangle/square nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key read-arg overrides the spoken figure name for real vertices, independently of print-sym. The key read-sub controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

3744 \clist_map_inline:nn { Afig , Pfig }
3745 {
3746     \keys_define:nn { spintent / sp #1 }
3747     {
3748         read-cmd .choices:nn = { school , mathcat }
3749         {
3750             \int_case:nn { \_keys_choice_int }
3751             {
3752                 {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
3753                 {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
3754             }
3755         },
3756         read-cmd .initial:n = school,
3757         read-cmd .value_required:n = true,
3758         read-cmd / unknown .code:n =
3759         \msg_error:nneee { spintent } { unknown-choice }
3760         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3761         prefix .code:n =
3762         { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
3763         prefix .initial:n = ,
3764         prefix .value_required:n = true,
3765         print-sym .choices:nn = { auto , triangle , square , none }

```

```

3766     {
3767         \int_case:nn { \l_keys_choice_int }
3768         {
3769             {1} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { auto      } }
3770             {2} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { triangle } }
3771             {3} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { square   } }
3772             {4} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { none     } }
3773         }
3774     },
3775     print-sym          .initial:n = auto,
3776     print-sym          .value_required:n = true,
3777     print-sym / unknown .code:n =
3778         \msg_error:nneee { spintent } { unknown-choice }
3779         { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
3780     read-arg           .code:n =
3781         { \l__spintent_sanitizize_affix:cn { \l__spintent_sp #1 _read_arg_tl } {##1} },
3782     read-arg           .initial:n = ,
3783     read-arg           .value_required:n = true,
3784     read-sub           .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
3785     read-sub           .initial:n = false,
3786     read-sub           .value_required:n = true,
3787     silent-cmd         .bool_set:c = { \l__spintent_sp #1 _silent_cmd_bool },
3788     silent-cmd         .initial:n = false,
3789     silent-cmd         .value_required:n = true,
3790     concept            .bool_set:c = { \l__spintent_sp #1 _concept_bool },
3791     concept            .initial:n = true,
3792     concept            .value_required:n = true,
3793     unknown            .code:n =
3794         {
3795             \str_set:Nn \l__spintent_command_name_str { sp #1 }
3796             \l__spintent_unknown_keys_cmd:n {##1}
3797         },
3798     }
3799 }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \spAfig and \spPfig

\l__spintent_spfig_sym:

The function `\l__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

3800 \cs_new_protected:Nn \l__spintent_spfig_sym:
3801 {
3802     \str_case:Vn \l__spintent_geo_print_sym_str
3803     {
3804         { triangle } { \triangle      }
3805         { square   } { \mdlgwhtsquare }
3806     }
3807 }

```

(End of definition for \l__spintent_spfig_sym:.)

\l__spintent_spfig_op_tl
\l__spintent_spfig_intent_tl
\l__spintent_spfig_symbol_tl
\l__spintent_spfig_pt_seq
\l__spintent_spfig_refs_tl
\l__spintent_spfig_intent_arg_tl
\l__spintent_spfig_pt_idx_int
\l__spintent_geo_print_sym_str

`\l__spintent_spfig_op_tl` holds área or perímetro. `\l__spintent_spfig_intent_tl` holds the concept word for the vertices case (read-arg or the auto-detected figure name). The rest are scratch variables to split `\l__spintent_geo_luaseta_points_clist` and build the `$pt1, $pt2, ...` reference list.

```

3808 \tl_new:N \l__spintent_spfig_op_tl
3809 \tl_new:N \l__spintent_spfig_intent_tl
3810 \tl_new:N \l__spintent_spfig_desc_tl
3811 \tl_new:N \l__spintent_spfig_symbol_tl
3812 \seq_new:N \l__spintent_spfig_pt_seq
3813 \tl_new:N \l__spintent_spfig_refs_tl
3814 \tl_new:N \l__spintent_spfig_intent_arg_tl
3815 \int_new:N \l__spintent_spfig_pt_idx_int
3816 \str_new:N \l__spintent_geo_print_sym_str

```

(End of definition for \l__spintent_spfig_op_tl and others.)

\l__spintent_spfig_process:nn

Receives the short command name (Afig or Pfig) in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. Calls `\l__spintent_luafun_geo` or `\l__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, sets `\l__spintent_spfig_op_tl` to área/perímetro, then delegates to `\l__spintent_spfig_mathml:n`.

```

3817 \cs_new_protected:Nn \l__spintent_spfig_process:nn
3818 {

```

```

3819 \str_if_eq:eeTF {#1} { Afig }
3820 { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
3821 { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
3822 \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3823 {
3824   \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3825   { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3826 }
3827 \str_if_eq:eeTF {#1} { Afig }
3828 { \tl_set:Nn \__spintent_spfig_op_tl { área } }
3829 { \tl_set:Nn \__spintent_spfig_op_tl { perímetro } }
3830 \str_set:Ne \__spintent_geo_print_sym_str { \str_use:c { \__spintent_sp #1 _print_sym_str } }
3831 \__spintent_spfig_mathml:n {#1}
3832 }

```

(End of definition for __spintent_spfig_process:nn.)

__spintent_spfig_build_simple:n

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds \l__spintent_geo_final_print_tl and \l__spintent_spfig_desc_tl (the descriptive part, without área/perímetro). \l__spintent_spfig_intent_tl is \l__spintent_spfig_op_tl-\l__spintent_spfig_desc_tl. `silent-cmd` overrides \l__spintent_spfig_intent_tl with `_`; with `silent-cmd=false` and `concept false`, it becomes `\l__spintent_spfig_desc_tl` (the descriptive part alone, literally, without área/perímetro); with `concept true` it is \l__spintent_spfig_intent_tl. Preserves the sym- letra-conflict check (a table letter cannot combine with a forced `print-sym`). Receives the short command name in ‘#1’.

```

3833 \cs_new_protected:Nn \__spintent_spfig_build_simple:n
3834 {
3835   \tl_if_empty:nTF { \l__spintent_geo_luaset_print_tl }
3836   {
3837     \str_case:VnF \l__spintent_geo_print_sym_str
3838     {
3839       { auto }
3840       {
3841         \tl_clear:N \l__spintent_geo_final_print_tl
3842         \tl_clear:N \l__spintent_spfig_desc_tl
3843       }
3844       { none }
3845       {
3846         \tl_clear:N \l__spintent_geo_final_print_tl
3847         \tl_clear:N \l__spintent_spfig_desc_tl
3848       }
3849     }
3850     {
3851       \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfig_sym: }
3852       \tl_set:Ne \l__spintent_spfig_desc_tl
3853       {
3854         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triángulo } { cuadrado }
3855       }
3856     }
3857   }
3858 }
3859 {
3860   \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
3861   {
3862     \str_case:VnF \l__spintent_geo_print_sym_str
3863     {
3864       { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p }
3865       { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p }
3866     }
3867     {
3868       \tl_set:Nn \l__spintent_geo_final_print_tl
3869       {
3870         \__spintent_spfig_sym:
3871         \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
3872       }
3873     }
3874     \str_case:VnF \l__spintent_geo_print_sym_str
3875     {
3876       { auto }
3877       {
3878         \tl_set:Ne \l__spintent_spfig_desc_tl
3879         {

```

```

3880         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3881         \l__spintent_geo_luaset_sub_spoken_str
3882     }
3883 }
3884 { none }
3885 {
3886     \tl_set:Ne \l__spintent_spfig_desc_tl
3887     {
3888         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3889         \l__spintent_geo_luaset_sub_spoken_str
3890     }
3891 }
3892 }
3893 {
3894     \tl_set:Ne \l__spintent_spfig_desc_tl
3895     {
3896         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
3897         { triángulo } { cuadrado }
3898     }
3899     -
3900     \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3901     \l__spintent_geo_luaset_sub_spoken_str
3902 }
3903 }
3904 {
3905     \str_case:VnF \l__spintent_geo_print_sym_str
3906     {
3907         { auto } { }
3908         { none } { }
3909     }
3910     {
3911         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
3912         { \c_backslash_str sp #1 }
3913     }
3914     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3915     \tl_set:Ne \l__spintent_spfig_desc_tl
3916     {
3917         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
3918         { \l__spintent_geo_luaset_letra_conector_str - }
3919         \l__spintent_geo_luaset_letra_word_str
3920         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
3921         {
3922             -
3923             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3924             \l__spintent_geo_luaset_sub_spoken_str
3925         }
3926     }
3927 }
3928 }
3929 \tl_set:Ne \l__spintent_spfig_intent_tl
3930 {
3931     \tl_if_empty:NTF \l__spintent_spfig_desc_tl
3932     { \l__spintent_spfig_op_tl }
3933     { \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl }
3934 }
3935 \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3936 { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
3937 {
3938     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3939     { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { \l__spintent_spfig_intent_tl } }
3940     { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
3941 }
3942 }

```

(End of definition for `\__spintent_spfig_build_simple:n`)

`\__spintent_spfig_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

3943 \cs_new_protected:Nn \__spintent_spfig_build_refs:
3944 {

```

```

3945 \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
3946 \tl_clear:N \l__spintent_spfig_refs_tl
3947 \int_zero:N \l__spintent_spfig_pt_idx_int
3948 \seq_map_inline:Nn \l__spintent_spfig_pt_seq
3949 {
3950   \int_incr:N \l__spintent_spfig_pt_idx_int
3951   \tl_if_empty:NF \l__spintent_spfig_refs_tl
3952   { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
3953   \tl_put_right:Ne \l__spintent_spfig_refs_tl { $ pt \int_use:N \l__spintent_spfig_pt_idx_int
3954 }
3955 }

```

(End of definition for `\l__spintent_spfig_build_refs:`.)

`\l__spintent_spfig_build_concept_vertices:n` Handles the vertices case. `read-arg`, when given, overrides the auto-detected figure name; with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim, and never silenced (only `silent-cmd` silences when `read-arg` is given — same convention as `\l__spintent_spside_process:nn`). With `read-arg` empty, `concept` controls silencing as usual. Also determines `\l__spintent_spfig_symbol_tl` (empty when none applies), preserving the sym-cardinality-conflict check (a forced `print-sym` must match the actual vertex count). Receives the short command name in `#1`. Must be called after `\l__spintent_spfig_build_refs:`.

```

3956 \cs_new_protected:Nn \l__spintent_spfig_build_concept_vertices:n
3957 {
3958   \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3959   { \tl_set:Ne \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
3960   {
3961     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3962     {
3963       \tl_set:Ne \l__spintent_spfig_intent_tl
3964       {
3965         \l__spintent_spfig_op_tl -del-
3966         \tl_use:c { l__spintent_sp #1 _read_arg_tl }
3967       }
3968     }
3969     {
3970       \tl_set:Ne \l__spintent_spfig_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3971     }
3972   }
3973
3974   \str_case:VnF \l__spintent_geo_print_sym_str
3975   {
3976     { auto }
3977     {
3978       \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
3979       {
3980         { triangle } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \triangle } }
3981         { square } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \mdlgwhtsquare } }
3982       }
3983       \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
3984       { \tl_clear:N \l__spintent_spfig_symbol_tl }
3985     }
3986     { none }
3987     { \tl_clear:N \l__spintent_spfig_symbol_tl }
3988   }
3989   {
3990     \str_if_eq:VVTF \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
3991     { \tl_set:Nn \l__spintent_spfig_symbol_tl { \l__spintent_spfig_sym: } }
3992     {
3993       \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
3994       { \c_backslash_str sp #1 }
3995     }
3996   }
3997
3998   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3999   {
4000     \tl_set:Ne \l__spintent_spfig_intent_arg_tl { -( \l__spintent_spfig_refs_tl ) }
4001   }
4002   {
4003     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4004     {
4005       \bool_if:cTF { l__spintent_sp #1 _concept_bool }

```

```

4006         {
4007             \tl_set:Nc \l__spintent_spfig_intent_arg_tl
4008             {
4009                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4010                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4011                 \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4012             }
4013         }
4014         {
4015             \tl_set:Nc \l__spintent_spfig_intent_arg_tl { _ ( \l__spintent_spfig_refs_tl ) }
4016         }
4017     }
4018     {
4019         \tl_set:Nc \l__spintent_spfig_intent_arg_tl
4020         {
4021             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4022             { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4023             \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4024         }
4025     }
4026 }
4027 }

```

(End of definition for `\l__spintent_spfig_build_concept_vertices:n`)

`\l__spintent_spfig_points_loop:n`

Walks `\l__spintent_spfig_pt_seq` a second time, calling `\l__spintent_mathmlarg_point:nnn` once per vertex with its `pt` n reference name, looked up under `sp` $\#1$. Receives the short command name in `'#1'`.

```

4028 \cs_new_protected:Nn \l__spintent_spfig_points_loop:n
4029 {
4030     \int_zero:N \l__spintent_spfig_pt_idx_int
4031     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4032     {
4033         \int_incr:N \l__spintent_spfig_pt_idx_int
4034         \l__spintent_mathmlarg_point:nnn
4035         { pt \int_use:N \l__spintent_spfig_pt_idx_int }
4036         { sp #1 }
4037         { ##1 }
4038     }
4039 }

```

(End of definition for `\l__spintent_spfig_points_loop:n`)

`\l__spintent_spfig_mathml:n`

Receives the short command name in `'#1'`. Branches on whether `\l__spintent_geo_luaset_points_clist` is empty.

Without vertices, `\l__spintent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all.

With vertices, `\l__spintent_spfig_build_refs:` then `\l__spintent_spfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\l__spintent_mathcal:n{⟨A|P⟩}` is silenced with `\l__spintent_luafun_wrap_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`'s own symbol); `\l__spintent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\l__spintent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback's recursive lookup through a noad's sub field, not just its nucleus.

`\l__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).

```

4040 \cs_new_protected:Nn \l__spintent_spfig_mathml:n
4041 {
4042     \l__spintent_invisible_sep: % (luamml issues #26)
4043     \clist_if_empty:NTF \l__spintent_geo_luaset_points_clist
4044     {
4045         \l__spintent_spfig_build_simple:n {#1}
4046         \MathMLintent { \l__spintent_spfig_intent_arg_tl }
4047         {
4048             \l__spintent_luafun_inner_sep:
4049             \str_if_eq:eeTF {#1} { Afig } { \l__spintent_mathcal:n { A } } { \l__spintent_mathcal:n
4050             \tl_if_empty:NF \l__spintent_geo_final_print_tl
4051             {
4052                 \c_math_subscript_token { \l__spintent_geo_final_print_tl }
4053             }

```

```

4054     }
4055   }
4056   {
4057     \__spintent_spfig_build_refs:
4058     \__spintent_spfig_build_concept_vertices:n {#1}
4059     \MathMLintent { \l__spintent_spfig_intent_arg_tl }
4060     {
4061       \__spintent_luafun_inner_sep:
4062       \__spintent_luafun_wrap_mrow_intent_arg:nn
4063       { } { intent = ":silent" }
4064       {
4065         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal
4066       }
4067       \c_math_subscript_token
4068       {
4069         \tl_if_empty:NF \l__spintent_spfig_symbol_tl
4070         {
4071           \__spintent_luafun_wrap_mrow_intent_arg:nn
4072           { } { intent = ":silent" }
4073           { \tl_use:N \l__spintent_spfig_symbol_tl }
4074         }
4075         \__spintent_spfig_points_loop:n {#1}
4076       }
4077     }
4078   }
4079 }

```

(End of definition for __spintent_spfig_mathml:n.)

__spintent_spfig_exec:nnn The function __spintent_spfig_exec:nnn receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to __spintent_spfig_process:nn.

```

4080 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
4081 {
4082   \keys_set:nn { spintent / sp #1 } { #2 }
4083   \__spintent_spfig_process:nn { #1 } { #3 }
4084 }

```

(End of definition for __spintent_spfig_exec:nnn.)

\spAfig Both public commands \spAfig and \spPfig enforce math mode, open a local group, and delegate execution to __spintent_spfig_exec:nnn.

```

4085 \NewDocumentCommand \spAfig { 0{ } d() }
4086 {
4087   \mode_if_math:TF
4088   {
4089     \group_begin:
4090     \tl_if_novalue:nTF {#2}
4091     { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4092     { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4093     \group_end:
4094   }
4095   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4096 }
4097 \NewDocumentCommand \spPfig { 0{ } d() }
4098 {
4099   \mode_if_math:TF
4100   {
4101     \group_begin:
4102     \tl_if_novalue:nTF {#2}
4103     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4104     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
4105     \group_end:
4106   }
4107   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
4108 }

```

(End of definition for \spAfig and \spPfig. These functions are documented on page 22.)

11.40 The command \spcirc

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmc`/`\spmcm` and `\spAfig`/`\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.40.1 Definition of variables for \spcirc

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\__spintent_geo_luaset_circ_radio_type_str` holds one of `number`, `measure`, `label`, `segment`, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```
4109 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
4110 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str
```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (usual or `odot`). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```
4111 \str_new:N \__spintent_spcirc_mode_str
4112 \tl_new:N \__spintent_spcirc_prefix_tl
4113 \str_new:N \__spintent_spcirc_print_sym_str
4114 \str_new:N \__spintent_spcirc_read_sty_str
4115 \str_new:N \__spintent_spcirc_radio_sty_str
4116 \str_new:N \__spintent_spcirc_sub_index_str
4117 \str_new:N \__spintent_spcirc_articulo_str
4118 \tl_new:N \__spintent_spcirc_read_arg_tl
```

(End of definition for `\__spintent_spcirc_mode_str` and others.)

Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (`circunferencia/círculo` or the `read-arg` override) and the final literal `intent` string.

```
4119 \tl_new:N \__spintent_geo_circ_word_tl
4120 \str_new:N \__spintent_geo_circ_intent_str
```

(End of definition for `\__spintent_geo_circ_word_tl` and `\__spintent_geo_circ_intent_str`.)

11.40.2 Definition of error messages for \spcirc

`spcirc-invalid-arg` Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```
4121 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
4122 {
4123   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
4124 }
4125 {
4126   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
4127 }
```

(End of definition for `spcirc-invalid-arg`.)

11.40.3 Definition of keys for \spcirc

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```
4128 \keys_define:nn { spintent / spcirc }
4129 {
4130   read-cmd      .choices:nn = { school , mathcat }
4131   {
4132     \int_case:nn { \l_keys_choice_int }
read-cmd
prefix
print-sym
read-sty
diameter
radio-sty
sub-index
read-sub
read-arg
silent-cmd
concept
spside
spmside
```

```

4133         {
4134             {1} { \str_set:Nn \l__spintent_spcirc_mode_str { school } }
4135             {2} { \str_set:Nn \l__spintent_spcirc_mode_str { mathcat } }
4136         }
4137     },
4138     read-cmd      .initial:n = school,
4139     read-cmd      .value_required:n = true,
4140     prefix        .code:n =
4141     { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
4142     prefix        .initial:n = ,
4143     prefix        .value_required:n = true,
4144     print-sym     .choices:nn = { usual , odot }
4145     {
4146         \int_case:nn { \l_keys_choice_int }
4147         {
4148             {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
4149             {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
4150         }
4151     },
4152     print-sym     .initial:n = odot,
4153     print-sym     .value_required:n = true,
4154     read-sty      .choices:nn = { circun , circle }
4155     {
4156         \int_case:nn { \l_keys_choice_int }
4157         {
4158             {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
4159             {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
4160         }
4161     },
4162     read-sty      .initial:n = circun,
4163     read-sty      .value_required:n = true,
4164     diameter      .bool_set:N = \l__spintent_spcirc_diameter_bool,
4165     diameter      .initial:n = false,
4166     diameter      .value_required:n = true,
4167     radio-sty     .choices:nn = { side , mside }
4168     {
4169         \int_case:nn { \l_keys_choice_int }
4170         {
4171             {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
4172             {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
4173         }
4174     },
4175     radio-sty     .initial:n = mside,
4176     radio-sty     .value_required:n = true,
4177     sub-index     .code:n =
4178     { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
4179     sub-index     .initial:n = ,
4180     sub-index     .value_required:n = true,
4181     read-sub      .bool_set:N = \l__spintent_spcirc_read_sub_bool,
4182     read-sub      .initial:n = false,
4183     read-sub      .value_required:n = true,
4184     read-arg      .code:n =
4185     { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
4186     read-arg      .initial:n = ,
4187     read-arg      .value_required:n = true,
4188     silent-cmd    .bool_set:N = \l__spintent_spcirc_silent_cmd_bool,
4189     silent-cmd    .initial:n = false,
4190     silent-cmd    .value_required:n = true,
4191     concept       .bool_set:N = \l__spintent_spcirc_concept_bool,
4192     concept       .initial:n = true,
4193     concept       .value_required:n = true,
4194     spside        .meta:nn = { spintent / spside } { #1 },
4195     spside        .value_required:n = true,
4196     spmside       .meta:nn = { spintent / spmside } { #1 },
4197     spmside       .value_required:n = true,
4198     unknown       .code:n =
4199     {
4200         \str_set:Nn \l__spintent_command_name_str { spcirc }
4201         \__spintent_unknown_keys_cmd:n {#1}
4202     },
4203 }

```

(End of definition for `read-cmd` and others.)

`\l_spintenc_spcirc_has_arg_bool` `\l__spintenc_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l_spintenc_spcirc_intent_tl` holds the word (`circunferencia` or `círculo`, with an optional `sub-index` suffix) paired with the symbol. `\l_spintenc_spcirc_intent_arg_tl` `\l__spintenc_spcirc_intent_arg_tl` holds `de-centro-en`, paired with the `(center)` group; the center is read through `\__spintenc_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l_spintenc_spcirc_radio_arg_tl`, is spoken directly on the `,` delimiter after the center, replacing its own reading rather than pairing with a group.

```
4204 \bool_new:N \l__spintenc_spcirc_has_arg_bool
4205 \tl_new:N \l__spintenc_spcirc_intent_tl
4206 \tl_new:N \l__spintenc_spcirc_intent_arg_tl
4207 \tl_new:N \l__spintenc_spcirc_connector_tl
4208 \tl_new:N \l__spintenc_spcirc_connector_arg_tl
4209 \tl_new:N \l__spintenc_spcirc_radio_arg_tl
```

(End of definition for `\l_spintenc_spcirc_has_arg_bool` and others.)

`\__spintenc_spcirc_sym:` Typesets `\bigodot` or the plain letter `C`, according to `print-sym`.

```
4210 \cs_new_protected:Nn \__spintenc_spcirc_sym:
4211 {
4212   \str_if_eq:NnTF \l__spintenc_spcirc_print_sym_str { odot }
4213   {
4214     \group_begin:
4215     \textstyle\bigodot
4216     \group_end:
4217   }
4218   { C }
4219 }
```

(End of definition for `\__spintenc_spcirc_sym:`)

`\_spintenc_spcirc_word_and_article:` Sets `\l__spintenc_geo_circ_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintenc_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```
4220 \cs_new_protected:Nn \_spintenc_spcirc_word_and_article:
4221 {
4222   \tl_if_empty:NnTF \l__spintenc_spcirc_read_arg_tl
4223   {
4224     \str_if_eq:NnTF \l__spintenc_spcirc_read_sty_str { circun }
4225     {
4226       \tl_set:Nn \l__spintenc_geo_circ_word_tl { circunferencia }
4227       \str_set:Nn \l__spintenc_spcirc_articulo_str { la }
4228     }
4229     {
4230       \tl_set:Nn \l__spintenc_geo_circ_word_tl { círculo }
4231       \str_set:Nn \l__spintenc_spcirc_articulo_str { el }
4232     }
4233   }
4234   {
4235     \tl_set:Nn \l__spintenc_geo_circ_word_tl { \l__spintenc_spcirc_read_arg_tl }
4236     \str_if_eq:NnTF \l__spintenc_spcirc_read_sty_str { circun }
4237     { \str_set:Nn \l__spintenc_spcirc_articulo_str { la } }
4238     { \str_set:Nn \l__spintenc_spcirc_articulo_str { el } }
4239   }
4240 }
```

(End of definition for `\_spintenc_spcirc_word_and_article:`)

`\_spintenc_spcirc_radio_print:` Dispatches on `\l__spintenc_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```
4241 \cs_new_protected:Nn \_spintenc_spcirc_radio_print:
4242 {
4243   \str_case:Nn \l__spintenc_geo_luaset_circ_radio_type_str
4244   {
4245     { number } { \exp_args:No \spnum { \l__spintenc_geo_luaset_circ_radio_raw_str } }
4246     { measure } { \exp_args:No \spunit { \l__spintenc_geo_luaset_circ_radio_raw_str } }
4247     { label } { \l__spintenc_geo_luaset_circ_radio_raw_str }
4248     { segment }
4249     {
```

```

4250         \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
4251         { \exp_args:No \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
4252         { \exp_args:No \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
4253     }
4254 }
4255 }

```

(End of definition for `\l__spintent_spcirc_radio_print:`.)

`\l__spintent_spcirc_build_concept:`

Builds two words: `\l__spintent_spcirc_intent_tl` (via `\l__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\l__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

`\l__spintent_spcirc_intent_arg_tl` (the symbol's word) is bare — the group it wraps has nothing to reference, only a silenced symbol — while `\l__spintent_spcirc_connector_arg_tl` (the connector word) is `:prefix($pt)`: both when `concept` is `true`, prefixed with `prefix` when given. `\l__spintent_spcirc_radio_arg.` — spoken directly on the , delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\l__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

`silent-cmd true` overrides all three with `_` (the connector with `_( $pt)` instead — the center is always read), regardless of `read-arg`. With `concept false` and `read-arg` empty, the same `_/( $pt)` silencing applies; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\l__spintent_spside_process:nn`).

```

4256 \cs_new_protected:Nn \l__spintent_spcirc_build_concept:
4257 {
4258     \l__spintent_spcirc_word_and_article:
4259     \tl_set:Nc \l__spintent_spcirc_intent_tl { \l__spintent_geo_circ_word_tl }
4260     \str_if_empty:NF \l__spintent_spcirc_sub_index_str
4261     {
4262         \tl_put_right:Nc \l__spintent_spcirc_intent_tl
4263         {
4264             -
4265             \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
4266             \l__spintent_spcirc_sub_index_str
4267         }
4268     }
4269     \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }
4270     \bool_if:NTF \l__spintent_spcirc_silent_cmd_bool
4271     {
4272         \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
4273         \tl_set:Nc \l__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
4274         \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
4275     }
4276     {
4277         \bool_if:NTF \l__spintent_spcirc_concept_bool
4278         {
4279             \tl_set:Nc \l__spintent_spcirc_intent_arg_tl
4280             {
4281                 \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
4282                 { \tl_use:N \l__spintent_spcirc_prefix_tl - }
4283                 \l__spintent_spcirc_intent_tl :prefix
4284             }
4285             \tl_set:Nc \l__spintent_spcirc_connector_arg_tl
4286             { \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
4287             \bool_if:NTF \l__spintent_spcirc_diameter_bool
4288             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
4289             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
4290         }
4291         {
4292             \tl_if_empty:NF \l__spintent_spcirc_read_arg_tl
4293             {
4294                 \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
4295                 \tl_set:Nc \l__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
4296                 \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
4297             }
4298             {
4299                 \tl_set:Nc \l__spintent_spcirc_intent_arg_tl { \l__spintent_spcirc_intent_tl }

```

```

4300         \tl_set:Nx \__spintent_spcirc_connector_arg_tl
4301         { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
4302         \bool_if:NTF \__spintent_spcirc_diameter_bool
4303         { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
4304         { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
4305     }
4306 }
4307 }
4308 }

```

(End of definition for `\__spintent_spcirc_build_concept:`.)

`\__spintent_spcirc_process:n` Receives the `d()` argument in ‘#1’, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

4309 \cs_new_protected:Nn \__spintent_spcirc_process:n
4310 {
4311     \tl_if_empty:NTF {#1}
4312     {
4313         \bool_set_false:N \__spintent_spcirc_has_arg_bool
4314     }
4315     {
4316         \bool_set_true:N \__spintent_spcirc_has_arg_bool
4317         \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
4318         \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4319         {
4320             \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
4321         }
4322     }
4323     \__spintent_spcirc_mathml:
4324 }

```

(End of definition for `\__spintent_spcirc_process:n`.)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius). Otherwise, the `(center)` group (`\__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol’s own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`’s symbol and `\sparc`’s accent. When a radius is present, the `,` delimiter after the center carries `\__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_wrap_mrow_separately` — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back’s recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `\uamml` only keeps a delimiter’s own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

4325 \cs_new_protected:Nn \__spintent_spcirc_mathml:
4326 {
4327     \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4328     \__spintent_spcirc_build_concept:
4329     \MathMLintent { \__spintent_spcirc_intent_arg_tl }
4330     {
4331         \__spintent_luafun_inner_sep:
4332         \__spintent_luafun_wrap_mrow_intent_arg:nn
4333         { } { intent = ":silent" }
4334         { \__spintent_spcirc_sym: }

```

```

4335     \str_if_empty:NF \l__spintent_spcirc_sub_index_str
4336     {
4337         \c_math_subscript_token
4338         {
4339             \__spintent_luafun_wrap_mrow_intent_arg:nn
4340             { } { intent = ":silent" }
4341             { \l__spintent_spcirc_sub_index_str }
4342         }
4343     }
4344 }
4345 \bool_if:NT \l__spintent_spcirc_has_arg_bool
4346 {
4347     \str_if_empty:NTF \l__spintent_geo_luaset_circ_radio_type_str
4348     {
4349         \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
4350         {
4351             \__spintent_luafun_inner_sep:
4352             \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
4353             \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points_
4354             \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } )
4355         }
4356     }
4357     {
4358         \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
4359         {
4360             \__spintent_luafun_inner_sep:
4361             \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
4362             \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points_
4363             }
4364         \MathMLintent { \l__spintent_spcirc_radio_arg_tl } { , }
4365         \__spintent_spcirc_radio_print:
4366         \MathMLintent { :silent } { \rparen }
4367     }
4368 }
4369 }

```

(End of definition for `\__spintent_spcirc_mathml:`.)

`\__spintent_spcirc_exec:nn`

The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spcirc_print:n`.

```

4370 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
4371 {
4372     \keys_set:nn { spintent / spcirc } { #1 }
4373     \__spintent_spcirc_process:n { #2 }
4374 }

```

(End of definition for `\__spintent_spcirc_exec:nn`.)

`\spcirc`

The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

4375 \NewDocumentCommand \spcirc { 0{} d() }
4376 {
4377     \mode_if_math:TF
4378     {
4379         \group_begin:
4380         \tl_if_novalue:nTF {#2}
4381         { \__spintent_spcirc_exec:nn { #1 } { } }
4382         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
4383         \group_end:
4384     }
4385     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
4386 }

```

(End of definition for `\spcirc`. This function is documented on page ??.)

11.41 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.41.1 Definition of variables for \spAcirc and \spPcirc

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```
\__spinttent_spAcirc_mode_str
\__spinttent_spPcirc_mode_str
\__spinttent_spAcirc_print_sym_str
\__spinttent_spPcirc_print_sym_str
\__spinttent_spAcirc_read_sty_str
\__spinttent_spPcirc_read_sty_str
\__spinttent_spAcirc_radio_sty_str
\__spinttent_spPcirc_radio_sty_str
\__spinttent_spAcirc_sub_index_str
\__spinttent_spPcirc_sub_index_str
\__spinttent_spAcirc_prefix_tl
\__spinttent_spPcirc_prefix_tl
\__spinttent_spAcirc_read_arg_tl
\__spinttent_spPcirc_read_arg_tl
\__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_spAcirc_mode_str and others.)

The variable `\__spinttent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (circle) or as the bare number/letter shortcut (bare).

```
4397 \str_new:N \__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_geo_circfig_mode_str.)

11.41.2 Definition of error messages for \spAcirc and \spPcirc

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```
spAcirc-invalid-arg
spPcirc-invalid-arg
```

```
4398 \clist_map_inline:nn { Acirc , Pcirc }
4399 {
4400   \msg_new:nnnn { spinttent } { sp #1 -invalid-arg }
4401   {
4402     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
4403   }
4404   {
4405     See~the~documentation~of~##1~for~the~accepted~argument.
4406   }
}
```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

11.41.3 Definition of keys for \spAcirc and \spPcirc

We define the shared *(keys)* `read-cmd`, `prefix`, `print-m`, `read-sty`, `sub-index`, `read-sub`, `read-arg`, and `silent-cmd`.

The keys `spside` and `spmside` are of the type `.meta:nn` passed to the commands `\spside` and `\spmside`

```
4407 \keys_define:nn { spinttent / sp #1 }
4408 {
4409   read-cmd .choices:nn = { school , mathcat }
4410   {
4411     \int_case:nn { \l_keys_choice_int }
4412     {
4413       {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
4414       {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
4415     }
4416   },
4417   read-cmd .initial:n = school,
4418   read-cmd .value_required:n = true,
4419   read-cmd / unknown .code:n =
4420     \msg_error:nneee { spinttent } { unknown-choice }
4421     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4422   prefix .code:n =
4423     { \__spinttent_sanitise_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
4424   prefix .initial:n = ,
4425   prefix .value_required:n = true,
4426   print-sym .choices:nn = { usual , odot }
4427   {
4428     \int_case:nn { \l_keys_choice_int }
4429     {
4430       {1} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { usual } }
4431       {2} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { odot } }
4432     }
4433   },
4434   print-sym .initial:n = odot,
4435   print-sym .value_required:n = true,
```

```

4436 print-sym / unknown .code:n =
4437   \msg_error:nneee { spintent } { unknown-choice }
4438   { print-sym } { usual,~odot } { \exp_not:n {##1} },
4439 read-sty               .choices:nn = { circun , circle }
4440   {
4441     \int_case:nn { \l_keys_choice_int }
4442     {
4443       {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
4444       {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
4445     }
4446   },
4447 read-sty               .initial:n = circun,
4448 read-sty               .value_required:n = true,
4449 concept               .bool_set:c = { l__spintent_sp #1 _concept_bool },
4450 concept               .initial:n = true,
4451 concept               .value_required:n = true,
4452 diameter              .bool_set:c = { l__spintent_sp #1 _diameter_bool },
4453 diameter              .initial:n = false,
4454 diameter              .value_required:n = true,
4455 radio-sty             .choices:nn = { side , mside }
4456   {
4457     \int_case:nn { \l_keys_choice_int }
4458     {
4459       {1} { \str_set:cn { l__spintent_sp #1 _radio_sty_str } { side } }
4460       {2} { \str_set:cn { l__spintent_sp #1 _radio_sty_str } { mside } }
4461     }
4462   },
4463 radio-sty             .initial:n = mside,
4464 radio-sty             .value_required:n = true,
4465 sub-index             .code:n =
4466   { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
4467 sub-index             .initial:n = ,
4468 sub-index             .value_required:n = true,
4469 read-sub              .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4470 read-sub              .initial:n = false,
4471 read-sub              .value_required:n = true,
4472 read-arg              .code:n =
4473   { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4474 read-arg              .initial:n = ,
4475 read-arg              .value_required:n = true,
4476 silent-cmd           .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4477 silent-cmd           .initial:n = false,
4478 silent-cmd           .value_required:n = true,
4479 spside                .meta:nn = { spintent / spside } { ##1 },
4480 spside                .value_required:n = true,
4481 spmside              .meta:nn = { spintent / spmside } { ##1 },
4482 spmside              .value_required:n = true,
4483 unknown              .code:n =
4484   {
4485     \str_set:Nn \l__spintent_command_name_str { sp #1 }
4486     \__spintent_unknown_keys_cmd:n {##1}
4487   },
4488 }
4489 }

```

(End of definition for read-cmd and others.)

11.41.4 Internal functions for \spAcirc and \spPcirc

11.41.5 Internal functions for \spAcirc and \spPcirc

`\__spintent_spcircfig_sym:n` The function `\__spintent_spcircfig_sym:n` typesets `\bigodot` or the plain letter C, according to the value set by the key `print-sym`.

```

4490 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
4491   {
4492     \str_if_eq:vnTF { l__spintent_sp #1 _print_sym_str } { odot }
4493     {
4494       \bigodot
4495     }
4496     { C }
4497   }

```

(End of definition for `\__spintent_spcircfig_sym:n`.)

`\__spintent_spcircfig_radio_print:n`

The function `\__spintent_spcircfig_radio_print:n` typesets the radius according to the type set by `\__spintent_luafun_geo_circ_parse_and_set:n`: `\spnum` for a plain number, `\spunit` for a measure, the raw text for a label, or `\spside`/`\spmside` for a segment, according to the value set by the key `radio-sty`.

```

4498 \cs_new_protected:Nn \__spintent_spcircfig_radio_print:n
4499 {
4500   \str_case:Vn \__spintent_geo_luaset_circ_radio_type_str
4501   {
4502     { number } { \exp_args:No \spnum { \__spintent_geo_luaset_circ_radio_raw_str } }
4503     { measure } { \exp_args:No \spunit { \__spintent_geo_luaset_circ_radio_raw_str } }
4504     { label } { \__spintent_geo_luaset_circ_radio_raw_str }
4505     { segment }
4506     {
4507       \str_if_eq:vnTF { \__spintent_sp #1 _radio_sty_str } { side }
4508       { \exp_args:No \spside { \__spintent_geo_luaset_circ_radio_raw_str } }
4509       { \exp_args:No \spmside { \__spintent_geo_luaset_circ_radio_raw_str } }
4510     }
4511   }
4512 }
```

(End of definition for `\__spintent_spcircfig_radio_print:n`.)

`\__spintent_spcircfig_word_and_article:n`

The function `\__spintent_spcircfig_word_and_article:n` sets `\__spintent_geo_circ_word_tl` to the chosen word — either the value set by the key `read-arg`, or `circunferencia/círculo` according to the key `read-sty` — and `\__spintent_spcirc_articulo_str` to `la` or `el`, matching the grammatical gender of the chosen word.

```

4513 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
4514 {
4515   \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
4516   {
4517     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
4518     {
4519       \tl_set:Nn \__spintent_geo_circ_word_tl { circunferencia }
4520       \str_set:Nn \__spintent_spcirc_articulo_str { la }
4521     }
4522     {
4523       \tl_set:Nn \__spintent_geo_circ_word_tl { círculo }
4524       \str_set:Nn \__spintent_spcirc_articulo_str { el }
4525     }
4526   }
4527   {
4528     \tl_set:Ne \__spintent_geo_circ_word_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
4529     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
4530     { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
4531     { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
4532   }
4533 }
```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_process:nn`

The function `\__spintent_spcircfig_process:nn` sets `\__spintent_geo_fig_op_tl` to `área` or `perímetro`. When `#2` is not empty, it calls `\__spintent_luafun_geo_circ_parse_and_set:n`; if that call reports an error and `#2` contains no comma, it falls back to `\__spintent_luafun_geo_afig_parse_and_set:n` / `\__spintent_luafun_geo_pfig_parse_and_set:n` (the same Lua entry point used by `\spAfig`/`\spPfig`) to try the bare number/letter shortcut, setting `\__spintent_geo_circfig_mode_str` to `bare` on success. A table letter match from that fallback is rejected (forced back to an error), since table letters are not meaningful for a circle.

`\__spintent_spcircfig_word_and_article:n` sets the word/article used to build `\__spintent_geo_circ_int` for the bare, empty, and real-center cases, delegating the rendering to `\__spintent_spcircfig_mathml:nn`.

```

4534 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
4535 {
4536   \str_if_eq:eeTF {#1} { Acirc }
4537   { \tl_set:Nn \__spintent_geo_fig_op_tl { área } }
4538   { \tl_set:Nn \__spintent_geo_fig_op_tl { perímetro } }
4539   \str_set:Nn \__spintent_geo_circfig_mode_str { circle }
4540   \tl_if_empty:nF {#2}
4541   {
4542     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
4543     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4544     {
4545       \tl_if_in:nnF {#2} { , }

```

```

4546         {
4547             \str_if_eq:eeTF {#1} { Acirc }
4548             { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4549             { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4550             \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
4551             {
4552                 \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
4553                 { \str_set:Nn \__spintent_geo_circfig_mode_str { bare } }
4554                 { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
4555             }
4556         }
4557     }
4558     \str_if_eq:VnF \__spintent_geo_circfig_mode_str { bare }
4559     {
4560         \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4561         {
4562             \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4563             { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4564         }
4565     }
4566 }
4567 \__spintent_spcircfig_word_and_article:n {#1}
4568 \str_if_eq:VnTF \__spintent_geo_circfig_mode_str { bare }
4569 {
4570     \str_set:Ne \__spintent_geo_circ_intent_str
4571     {
4572         \__spintent_geo_fig_op_tl -
4573         \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4574         \__spintent_geo_luaset_sub_spoken_str
4575     }
4576     \str_if_eq:eeT { \str_use:c { \__spintent_sp #1 _mode_str } } { mathcat }
4577     {
4578         \str_set:Ne \__spintent_geo_circ_intent_str
4579         {
4580             _ \__spintent_geo_fig_op_tl -
4581             \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4582             \__spintent_geo_luaset_sub_spoken_str :prefix($pts)
4583         }
4584     }
4585 }
4586 {
4587     \tl_if_empty:NTF {#2}
4588     {
4589         \str_set:Ne \__spintent_geo_circ_intent_str
4590         {
4591             \__spintent_geo_fig_op_tl -de- \__spintent_spcirc_articulo_str -
4592             \__spintent_geo_circ_word_tl
4593         }
4594         \str_if_eq:eeT { \str_use:c { \__spintent_sp #1 _mode_str } } { mathcat }
4595         {
4596             \str_set:Ne \__spintent_geo_circ_intent_str
4597             {
4598                 _ \__spintent_geo_fig_op_tl -de-
4599                 \__spintent_spcirc_articulo_str -
4600                 \__spintent_geo_circ_word_tl :prefix($pts)
4601             }
4602         }
4603     }
4604     {
4605         \str_set:Ne \__spintent_geo_circ_intent_str
4606         {
4607             \__spintent_geo_fig_op_tl -de- \__spintent_spcirc_articulo_str -
4608             \__spintent_geo_circ_word_tl
4609         }
4610         \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
4611         {
4612             \str_put_right:Ne \__spintent_geo_circ_intent_str
4613             {
4614                 -
4615                 \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4616                 \str_use:c { \__spintent_sp #1 _sub_index_str }

```

```

4617     }
4618   }
4619   \str_put_right:Ne \l__spintent_geo_circ_intent_str
4620   { -de-centro-en- \l__spintent_geo_luaset_print_tl }
4621   \str_if_empty:NF \l__spintent_geo_luaset_circ_radio_type_str
4622   { \str_put_right:Nn \l__spintent_geo_circ_intent_str { -y-radio } }
4623   \str_if_eq:eeT { \str_use:c { \l__spintent_sp #1 _mode_str } } { mathcat }
4624   {
4625     \str_set:Ne \l__spintent_geo_circ_intent_str
4626     {
4627       _ \l__spintent_geo_fig_op_tl -de-
4628       \l__spintent_spcirc_articulo_str -
4629       \l__spintent_geo_circ_word_tl :prefix($pts)
4630     }
4631   }
4632 }
4633 }
4634 \__spintent_spcircfig_mathml:n { #1 } { #2 }
4635 }

```

(End of definition for `\__spintent_spcircfig_process:nn`.)

`\__spintent_spcircfig_mathml:n`

The function `\__spintent_spcircfig_mathml:n` reads the state already set by `\__spintent_spcircfig_process:nn` and renders it. Three cases follow: the bare shortcut prints the symbol with the shortcut value as its own subscript; an empty `#2` prints the symbol alone; and a real center (with an optional radius) prints the symbol, the optional sub-index subscript, and the center between parentheses. When a radius is present, `\mathcal` (`\__spintent_mathcal:n`) gets its own `\MathMLintent` holding the full spoken intent, and the circle's own visual pieces — symbol/center/comma, the radius (via `\__spintent_spcircfig_radio_print:n`, kept audible), and the closing parenthesis — become three sibling `\MathMLintent` calls sharing a single `\c_math_subscript_token` group, so the whole notation is typeset at subscript size.

```

4636 \cs_new_protected:Nn \__spintent_spcircfig_mathml:n
4637 {
4638   \__spintent_invisible_sep:
4639   \str_if_eq:VnTF \l__spintent_geo_circfig_mode_str { bare }
4640   {
4641     %%\__spintent_invisible_sep:
4642     \MathMLintent
4643     {
4644       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4645       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4646       \l__spintent_geo_circ_intent_str
4647     }
4648     {
4649       \c__spintent_invisible_sep_tl
4650       \str_if_eq:eeTF {#1} { Acirc }
4651       { \__spintent_mathcal:n { A } }
4652       { \__spintent_mathcal:n { P } }
4653       \c_math_subscript_token
4654       {
4655         \__spintent_spcircfig_sym:n {#1}
4656         \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4657       }
4658     }
4659   }
4660   {
4661     \tl_if_empty:nTF {#2}
4662     {
4663       %%\__spintent_invisible_sep:
4664       \MathMLintent
4665       {
4666         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4667         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4668         \l__spintent_geo_circ_intent_str
4669       }
4670       {
4671         \c__spintent_invisible_sep_tl
4672         \str_if_eq:eeTF {#1} { Acirc }
4673         { \__spintent_mathcal:n { A } }
4674         { \__spintent_mathcal:n { P } }
4675         \c_math_subscript_token { \__spintent_spcircfig_sym:n {#1} }
4676       }
4677     }
4678   }
4679 }

```

```

4677     }
4678   {
4679     %%\__spintent_invisible_sep:
4680     \str_if_empty:NTF \__spintent_geo_luaset_circ_radio_type_str
4681     {
4682       \MathMLintent
4683       {
4684         \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
4685         { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
4686         \__spintent_geo_circ_intent_str
4687       }
4688       {
4689         \c__spintent_invisible_sep_tl
4690         \str_if_eq:eeTF {#1} { Acirc }
4691         { \__spintent_mathcal:n { A } }
4692         { \__spintent_mathcal:n { P } }
4693         \c_math_subscript_token
4694         {
4695           \__spintent_spcircfig_sym:n {#1}
4696           \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
4697           {
4698             \c_math_subscript_token
4699             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4700           }
4701           ( \__spintent_geo_luaset_print_tl )
4702         }
4703       }
4704     }
4705   {
4706     \MathMLintent
4707     {
4708       \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
4709       { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
4710       \__spintent_geo_circ_intent_str
4711     }
4712     {
4713       \c__spintent_invisible_sep_tl
4714       \str_if_eq:eeTF {#1} { Acirc }
4715       { \__spintent_mathcal:n { A } }
4716       { \__spintent_mathcal:n { P } }
4717     }
4718     \c_math_subscript_token
4719     {
4720       \MathMLintent
4721       { :silent }
4722       {
4723         \MathMLintent{ :silent } { \__spintent_spcircfig_sym:n {#1} }
4724         \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
4725         {
4726           \c_math_subscript_token
4727           { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4728         }
4729         ( \__spintent_geo_luaset_print_tl , \,
4730         )
4731       }
4732       %%\__spintent_invisible_sep:
4733       \__spintent_spcircfig_radio_print:n {#1}
4734       \MathMLintent { :silent } { ) }
4735     }
4736   }
4737 }
4738 }

```

(End of definition for __spintent_spcircfig_mathml:nn.)

__spintent_spcircfig_exec:nnn

The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

4739 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
4740 {
4741   \keys_set:nn { spintent / sp #1 } { #2 }
4742   \__spintent_spcircfig_process:nn { #1 } { #3 }
4743 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc` Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

```

4744 \NewDocumentCommand \spAcirc { 0{} d() }
4745 {
4746   \mode_if_math:TF
4747   {
4748     \group_begin:
4749     \tl_if_novalue:nTF {#2}
4750     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
4751     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
4752     \group_end:
4753   }
4754   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
4755 }
4756 \NewDocumentCommand \spPcirc { 0{} d() }
4757 {
4758   \mode_if_math:TF
4759   {
4760     \group_begin:
4761     \tl_if_novalue:nTF {#2}
4762     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
4763     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
4764     \group_end:
4765   }
4766   { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
4767 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page ??.)

11.42 Finish package

Finish package implementation.

```

4768 \file_input_stop:
4769 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	96
C	
cifras-sep	<u>269</u> , <u>554</u> , <u>827</u> , <u>1780</u> , <u>1851</u> , <u>2263</u>
Document class:	
l3doc	28
cmd-key-unknown	<u>189</u>
cmd-key-value-unknown	<u>189</u>
Commands provide by spintent :	
\setspintent	31
\spmoney	45
\spnum	35
\spsiglo	54, 57
\spunit	40
concept	<u>2979</u> , <u>3184</u> , <u>3415</u> , <u>3568</u> , <u>3744</u> , <u>4128</u> , <u>4407</u>
currency	<u>827</u>
currency-no-subunits	<u>796</u>
D	
diameter	<u>4128</u> , <u>4407</u>
dolar-math	<u>827</u>
I	
inline-numeric-denominator	532
invalid-angle-format	2213
invalid-century-format	1223
invalid-currency	796
invalid-date-format	1453
invalid-time-format	1374
J	
join-word	<u>2341</u>
K	
Keys for \spAcirc provide by spintent :	
prefix	119
print-m	119
print-sym	119, 120
radio-sty	119, 121
read-arg	119, 121
read-cmd	119
read-sty	119, 121
read-sub	119
silent-cmd	119
spmside	119
spside	119
sub-index	119
Keys for \spAfig provide by spintent :	
prefix	105
print-sym	105, 106
read-arg	105, 106
read-cmd	105
read-sub	106
Keys for \spangle provide by spintent :	
concept	95
prefix	95
print-m	95
read-cmd	95
recto	94, 95
silent-cmd	95
Keys for \sparc provide by spintent :	
concept	99
prefix	99
read-cmd	99
silent-cmd	99
Keys for \spcirc provide by spintent :	
prefix	113
print-sym	113, 115
radio-sty	113
read-arg	113
read-cmd	113
read-sty	113
read-sub	113
sub-index	113
Keys for \spdiv provide by spintent :	
cifras-sep	65
notation-sym	65
read-parens	65
read-period-sep	65
read-sign	65
read-sym	65
set-sym	65
wrap-parens	65
Keys for \spdsym provide by spintent :	
prefix	63
read-sym	63
set-sym	63
suffix	63
Keys for \spmangle provide by spintent :	
concept	95
prefix	95
print-m	94, 95
read-cmd	95
recto	94, 95
silent-cmd	95
Keys for \spmarc provide by spintent :	
concept	99
prefix	99
read-cmd	99
silent-cmd	99
Keys for \spmcid provide by spintent :	
prefix	69, 71
read-cmd	69
result-num	71
result	69
sample-arg	69
silent-cmd	69
upper-case	69
Keys for \spmcm provide by spintent :	
prefix	69, 71
read-cmd	69
result-num	71
result	69
sample-arg	69
silent-cmd	69
upper-case	69
Keys for \spmoney provide by spintent :	
cifras-sep	46
currency	46

dolar-math	46	read-sub	106
sub-units	46	Keys for \spPoint provide by spintent :	
Keys for \spmQ provide by spintent :		concept	82
join-word	77, 79	prefix	82
print-dfrac	77-81	read-cmd	82
read-parens	77, 78	silent-cmd	82
use-spnZ	77, 80, 81	Keys for \spPoly provide by spintent :	
wrap-parens	77, 78	concept	102
Keys for \spmside provide by spintent :		prefix	102
concept	91	print-sym	102
prefix	90, 91	read-cmd	102
print-m	90, 91	silent-cmd	102
read-arg	90, 91	Keys for \sprayo provide by spintent :	
read-cmd	90, 91, 93	concept	88
read-sty	90, 91	prefix	88
silent-cmd	91, 93	read-cmd	88, 89
Keys for \spmsym provide by spintent :		read-sty	88
not-print	64	silent-cmd	88, 89
prefix	64	Keys for \sprecta provide by spintent :	
read-sym	64	prefix	85
set-sym	64	read-cmd	85, 86
suffix	64	silent-cmd	85, 86
Keys for \spm\ provide by spintent :		Keys for \spshort provide by spintent :	
cifras-sep	67	read-arg	52
notation-sym	67	small-caps	52
read-parens	67	Keys for \spside provide by spintent :	
read-period-sep	67	concept	91
read-sign	67	prefix	90, 91
read-sym	67	read-arg	90, 91
set-sym	67	read-cmd	90, 91, 93
wrap-parens	67	read-sty	90, 91
Keys for \spnD provide by spintent :		silent-cmd	91, 93
result	71	Keys for \spsiglo provide by spintent :	
silent-cmd	71	print-era	54, 56
Keys for \spnM provide by spintent :		Keys for \spunit provide by spintent :	
result	71	cifras-sep	41
silent-cmd	71	notation-sym	41
Keys for \spnum provide by spintent :		print-cdot	41
cifras-sep	34-36	read-cdot	41
notation-sym	35	read-period-sep	41
read-period-sep	35, 37	read-slash	41
read-sign	35	Keys for \spusep provide by spintent :	
Keys for \spnZ provide by spintent :		silent	60
cifras-sep	75	size	60
read-parens	75, 76		
read-sign	75		
wrap-parens	75, 76		
Keys for \spPcirc provide by spintent :			
prefix	119		
print-m	119		
print-sym	119, 120		
radio-sty	119, 121		
read-arg	119, 121		
read-cmd	119		
read-sty	119, 121		
read-sub	119		
silent-cmd	119		
spmside	119		
spside	119		
sub-index	119		
Keys for \spPfig provide by spintent :			
prefix	105		
print-sym	105, 106		
read-arg	105, 106		
read-cmd	105		

M

mathcal-fallback	70
\MathMLintent	97
mc-invalid-arguments	1933
md-invalid-argument	2105
Lua module provide by spintent :	
spintent.lua	28, 29, 33-35, 40, 44-46, 51, 52, 54, 56, 57, 59, 68, 71, 74, 76, 81, 82

N

not-print	1706
notation-sym	269, 554, 1780, 1851

P

Packages :	
expl3	33, 35
lua-unicode-math	28
luamml	30, 77, 79
tagpdf	51, 55
unicode-math	28, 61

prefix [1636](#), [1706](#), [1940](#), [2079](#), [2548](#), [2690](#), [2843](#), [2979](#), [3184](#),
[3415](#), [3568](#), [3744](#), [4128](#), [4407](#)
 print-dfrac [2341](#)
 print-era [1231](#)
 print-m [2979](#), [3184](#)
 print-sym [3568](#), [3744](#), [4128](#), [4407](#)

R

radio-sty [4128](#), [4407](#)
 read-arg [1101](#), [2979](#), [3744](#), [4128](#), [4407](#)
 read-cdot [554](#)
 read-cmd .. [1940](#), [2548](#), [2690](#), [2843](#), [2979](#), [3184](#), [3415](#), [3568](#),
[3744](#), [4128](#), [4407](#)
 read-parens [1780](#), [1851](#), [2263](#), [2341](#)
 read-period-sep [269](#), [554](#), [1780](#), [1851](#)
 read-sign [269](#), [1780](#), [1851](#), [2263](#)
 read-slash [554](#)
 read-sty [2843](#), [2979](#), [4128](#), [4407](#)
 read-sub [3744](#), [4128](#), [4407](#)
 read-sym [1636](#), [1706](#), [1780](#), [1851](#)
 recto [3184](#)
 result [1940](#), [2079](#)
 result-num [2079](#)
 right-angle-sqr-fallback [70](#)
 \rightangle [96](#)

S

sample-arg [1940](#), [2079](#)
 seq commands:
 \seq_set_from_clist:Nn [58](#)
 set-sym [1636](#), [1706](#), [1780](#), [1851](#)
 \setspintent [3](#), [31](#), [177](#)
 silent [1501](#)
 silent-cmd [1940](#), [2079](#), [2548](#), [2690](#), [2843](#), [2979](#), [3184](#), [3415](#),
[3568](#), [3744](#), [4128](#), [4407](#)
 size [1501](#)
 small-caps [1101](#)
 \spAcirc [118](#), [4744](#)
 spAcirc-invalid-arg [4398](#)
 \spAfig [22](#), [105](#), [4085](#)
 spAfig-invalid-arg [3713](#)
 spAfig-invalid-points [3713](#)
 spAfig-sym-cardinality-conflict [3713](#)
 spAfig-sym-letra-conflict [3713](#)
 \spang [21](#), [74](#), [2253](#)
 \spangle [20](#), [94](#), [3380](#)
 spangle-invalid-arg [3174](#)
 \sparc [21](#), [99](#), [3539](#)
 sparc-invalid-points [3405](#)
 \spcirc [113](#), [4375](#)
 spcirc-invalid-arg [4121](#)
 \spdate [11](#), [59](#), [1477](#)
 \spdiv [13](#), [65](#), [1841](#)
 spdiv-empty-argument [1800](#)
 \spdsym [12](#), [63](#), [1683](#)
 spintent internal commands:
 __spintent_collapse_hyphens:N [114](#)
 \l__spintent_command_name_str [189](#)
 \l__spintent_geo_circ_intent_str [4119](#)
 \l__spintent_geo_circ_word_tl [4119](#)
 \l__spintent_geo_circfig_mode_str [4397](#)
 \l__spintent_geo_fig_op_tl [2524](#)

\l__spintent_geo_final_print_tl [3705](#)
 \l__spintent_geo_luaset_angulo_case_-
 str [2524](#)
 \l__spintent_geo_luaset_circ_radio_raw_-
 str [4109](#)
 \l__spintent_geo_luaset_circ_radio_type_-
 str [4109](#)
 \l__spintent_geo_luaset_concept_str ... [2524](#)
 \l__spintent_geo_luaset_error_str . [81](#), [2524](#)
 \l__spintent_geo_luaset_intent_str [81](#), [2524](#)
 \l__spintent_geo_luaset_letra_word_str [2524](#)
 \l__spintent_geo_luaset_nombre_es_num_-
 str [2524](#)
 \l__spintent_geo_luaset_points_-
 clist\l__spintent_geo_luaset_is_-
 name_str [2524](#)
 \l__spintent_geo_luaset_poly_sym_str .. [2524](#)
 \l__spintent_geo_luaset_print_tl .. [81](#), [2524](#)
 \l__spintent_geo_luaset_sub_spoken_str [2524](#)
 \l__spintent_geo_print_sym_str [3808](#)
 __spintent_invisible_sep: [106](#)
 \c__spintent_invisible_sep_tl [79](#), [106](#)
 __spintent_luafun_clean_split_and_set:n [65](#),
[226](#)
 __spintent_luafun_spang_parse:n [74](#)
 __spintent_luafun_sdate_parse:n [59](#)
 __spintent_luafun_spmoney_lookup_data:n [65](#)
 __spintent_luafun_spmQ_scale_int:nn ... [65](#)
 __spintent_luafun_sptime_parse:n [58](#)
 __spintent_luafun_spunit_lookup_alias:n [65](#)
 \l__spintent_luaset_arg_above_tl [245](#)
 \l__spintent_luaset_arg_below_tl [245](#)
 \l__spintent_luaset_cal_fam_ok_str [63](#)
 \l__spintent_luaset_dec_is_all_zeros_-
 str [238](#)
 \l__spintent_luaset_dec_sep_tl [226](#)
 \l__spintent_luaset_decimal_period_str [238](#)
 \l__spintent_luaset_denom_has_number_-
 str [245](#)
 \l__spintent_luaset_exponent_tl [238](#)
 \l__spintent_luaset_format_part_dec_str [233](#)
 \l__spintent_luaset_format_part_int_str [233](#)
 \l__spintent_luaset_has_exponent_str .. [238](#)
 \l__spintent_luaset_has_integer_str .. [79-81](#),
[226](#)
 \l__spintent_luaset_has_natural_str ... [226](#)
 \l__spintent_luaset_has_units_str [245](#)
 \l__spintent_luaset_millions_str [238](#)
 \l__spintent_luaset_not_decimal_not_-
 period_str [238](#)
 \l__spintent_luaset_not_decimal_period_-
 str [238](#)
 \l__spintent_luaset_only_part_dec_str . [233](#)
 \l__spintent_luaset_only_part_int_str . [233](#)
 \l__spintent_luaset_only_part_period_-
 str [233](#)
 \l__spintent_luaset_part_dec_tl [226](#)
 \l__spintent_luaset_part_int_tl [79](#), [226](#)
 \l__spintent_luaset_part_period_tl ... [226](#)
 \l__spintent_luaset_sign_tl [79](#), [226](#)
 \l__spintent_luaset_status_str [245](#)
 \l__spintent_luaset_sym_fam_ok_str [63](#)
 __spintent_mathcal:n [84](#)

<code>__spintent_mathmlarg_point:nnn</code>	2644	<code>__spintent_spcirc_mathml:</code>	4325
<code>\l__spintent_mc_args_seq</code>	1922	<code>\l__spintent_spcirc_mode_str</code>	4111
<code>__spintent_mc_process_args:nn</code>	1981	<code>\l__spintent_spcirc_prefix_tl</code>	4111
<code>\l__spintent_md_out_seq</code>	2069	<code>\l__spintent_spcirc_print_sym_str</code>	4111
<code>__spintent_print_spunit_hierarchy:</code>	655	<code>__spintent_spcirc_process:n</code>	4309
<code>__spintent_right_angle_sqr:</code>	97	<code>\l__spintent_spcirc_radio_arg_tl</code>	4204
<code>__spintent_sanitize_affix:Nn</code>	122	<code>__spintent_spcirc_radio_print:</code>	4241
<code>\l__spintent_saved_luamml_flag_int</code>	79, 2322	<code>\l__spintent_spcirc_radio_sty_str</code>	4111
<code>\l__spintent_setspintent_input_arg_cmd-</code>		<code>\l__spintent_spcirc_read_sty_str</code>	4111
<code>tl</code>	140	<code>\l__spintent_spcirc_sub_index_str</code>	4111
<code>__spintent_setspintent_parse_args:nn</code>	160	<code>__spintent_spcirc_sym:</code>	4210
<code>__spintent_setspintent_set_keys:nn</code>	149	<code>__spintent_spcirc_word_and_article:</code>	4220
<code>\l__spintent_smparc_prefix_tl</code>	3400	<code>__spintent_spcircfig_exec:nnn</code>	4739
<code>\l__spintent_spAcirc_mode_str</code>	4387	<code>__spintent_spcircfig_mathml:nn</code>	4636
<code>\l__spintent_spAcirc_prefix_tl</code>	4387	<code>__spintent_spcircfig_process:nn</code>	4534
<code>\l__spintent_spAcirc_print_sym_str</code>	4387	<code>__spintent_spcircfig_radio_print:n</code>	4498
<code>\l__spintent_spAcirc_radio_sty_str</code>	4387	<code>__spintent_spcircfig_sym:n</code>	4490
<code>\l__spintent_spAcirc_read_arg_tl</code>	4387	<code>__spintent_spcircfig_word_and-</code>	
<code>\l__spintent_spAcirc_read_sty_str</code>	4387	<code>article:n</code>	4513
<code>\l__spintent_spAcirc_sub_index_str</code>	4387	<code>\l__spintent_spdate_luaset_error_str</code>	1451
<code>\l__spintent_spAfig_mode_str</code>	3706	<code>\l__spintent_spdate_luaset_output_str</code>	1451
<code>\l__spintent_spAfig_prefix_tl</code>	3706	<code>__spintent_spdate_process:n</code>	59, 1462
<code>\l__spintent_spAfig_print_sym_str</code>	3706	<code>__spintent_spdiv_process:nnn</code>	1826
<code>\l__spintent_spAfig_read_arg_tl</code>	3706	<code>__spintent_spdiv_render_op:nn</code>	1805
<code>\l__spintent_spang_luaset_error_str</code>	2209	<code>__spintent_spdiv_render_parens:nn</code>	1814
<code>\l__spintent_spang_luaset_grado_str</code>	2209	<code>\l__spintent_spdsym_prefix_tl</code>	1632
<code>\l__spintent_spang_luaset_minuto_str</code>	2209	<code>\l__spintent_spdsym_read_sym_tl</code>	1632
<code>\l__spintent_spang_luaset_segundo_str</code>	2209	<code>\l__spintent_spdsym_suffix_tl</code>	1632
<code>__spintent_spang_process:n</code>	2218	<code>\l__spintent_spdsym_symbol_tl</code>	1632
<code>__spintent_spangle_exec:nnn</code>	3375	<code>__spintent_spfig_build_concept-</code>	
<code>\l__spintent_spangle_intent_arg_tl</code>	3243	<code>vertices:n</code>	3956
<code>\l__spintent_spangle_intent_tl</code>	3243	<code>__spintent_spfig_build_refs:</code>	3943
<code>__spintent_spangle_mathml:n</code>	3271	<code>__spintent_spfig_build_simple:n</code>	3833
<code>\l__spintent_spangle_mode_str</code>	3169	<code>__spintent_spfig_exec:nnn</code>	4080
<code>\l__spintent_spangle_prefix_tl</code>	3169	<code>\l__spintent_spfig_intent_arg_tl</code>	3808
<code>\l__spintent_spangle_print_m_now_bool</code>	3243	<code>\l__spintent_spfig_intent_tl</code>	3808
<code>__spintent_spangle_process:nn</code>	3250	<code>__spintent_spfig_mathml:n</code>	4040
<code>\l__spintent_spangle_pt_idx_int</code>	3243	<code>\l__spintent_spfig_op_tl</code>	3808
<code>\l__spintent_spangle_pt_seq</code>	3243	<code>__spintent_spfig_points_loop:n</code>	4028
<code>\l__spintent_spangle_refs_tl</code>	3243	<code>__spintent_spfig_process:nn</code>	3817
<code>__spintent_spangle_sym:n</code>	3224	<code>\l__spintent_spfig_pt_idx_int</code>	3808
<code>__spintent_sparc_exec:nnn</code>	3534	<code>\l__spintent_spfig_pt_seq</code>	3808
<code>\l__spintent_sparc_intent_arg_tl</code>	3459	<code>\l__spintent_spfig_refs_tl</code>	3808
<code>\l__spintent_sparc_intent_tl</code>	3459	<code>__spintent_spfig_sym:</code>	3800
<code>__spintent_sparc_mathml:n</code>	3477	<code>\l__spintent_spfig_symbol_tl</code>	3808
<code>\l__spintent_sparc_mode_str</code>	3400	<code>\l__spintent_spmangle_mode_str</code>	3169
<code>\l__spintent_sparc_prefix_tl</code>	3400	<code>\l__spintent_spmangle_prefix_tl</code>	3169
<code>__spintent_sparc_print_arc:n</code>	3453	<code>\l__spintent_spmarc_mode_str</code>	3400
<code>__spintent_sparc_process:nn</code>	3464	<code>__spintent_spmc_process:nnnn</code>	2003
<code>\l__spintent_sparc_pt_idx_int</code>	3459	<code>\l__spintent_spmcd_prefix_tl</code>	1922
<code>\l__spintent_sparc_pt_seq</code>	3459	<code>\l__spintent_spmcd_print_result_bool</code>	1922
<code>\l__spintent_sparc_refs_tl</code>	3459	<code>\l__spintent_spmcd_print_upper_bool</code>	1922
<code>__spintent_sparc_wide_arc:n</code>	3449	<code>\l__spintent_spmcd_read_terse_bool</code>	1922
<code>\l__spintent_spcirc_articulo_str</code>	4111	<code>\l__spintent_spmcd_suffix_tl</code>	1922
<code>__spintent_spcirc_build_concept:</code>	4256	<code>\l__spintent_spmcm_prefix_tl</code>	1922
<code>\l__spintent_spcirc_connector_arg_tl</code>	4204	<code>\l__spintent_spmcm_print_result_bool</code>	1922
<code>\l__spintent_spcirc_connector_tl</code>	4204	<code>\l__spintent_spmcm_print_upper_bool</code>	1922
<code>__spintent_spcirc_exec:nn</code>	4370	<code>\l__spintent_spmcm_read_terse_bool</code>	1922
<code>\l__spintent_spcirc_has_arg_bool</code>	4204	<code>\l__spintent_spmcm_spmcd_luaset_error-</code>	
<code>\l__spintent_spcirc_intent_arg_tl</code>	4204	<code>str</code>	1922
<code>\l__spintent_spcirc_intent_tl</code>	4204	<code>\l__spintent_spmcm_suffix_tl</code>	1922
		<code>__spintent_spmoney_build_tree:</code>	1006

<code>__spintent_spmoney_check_sub_units: ..</code>	959	<code>\l__spintent_spnD_luaset_is_truncated_-</code>	2069
<code>\l__spintent_spmoney_extracted_curr_str</code>	788	<code>str</code>	2069
<code>\l__spintent_spmoney_extracted_num_tl ..</code>	788	<code>\l__spintent_spnD_luaset_result_tl ...</code>	2069
<code>__spintent_spmoney_intent_int:</code>	895	<code>\l__spintent_spnD_prefix_tl</code>	2069
<code>\l__spintent_spmoney_intent_money_str ..</code>	788	<code>\l__spintent_spnD_print_result_bool ...</code>	2069
<code>\l__spintent_spmoney_intent_sign_str ..</code>	788	<code>\l__spintent_spnD_print_result_num_tl ..</code>	2069
<code>\l__spintent_spmoney_intent_sub_unit_-</code>	788	<code>\l__spintent_spnD_silent_cmd_bool</code>	2069
<code>str</code>	788	<code>\l__spintent_spnM_luaset_is_truncated_-</code>	2069
<code>\l__spintent_spmoney_luaset_conde_str ..</code>	778	<code>str</code>	2069
<code>\l__spintent_spmoney_luaset_currency_arg_-</code>	778	<code>\l__spintent_spnM_luaset_result_tl ...</code>	2069
<code>str</code>	778	<code>\l__spintent_spnM_prefix_tl</code>	2069
<code>\l__spintent_spmoney_luaset_plur_str ..</code>	778	<code>\l__spintent_spnM_print_result_bool ...</code>	2069
<code>\l__spintent_spmoney_luaset_position_-</code>	778	<code>\l__spintent_spnM_print_result_num_tl ..</code>	2069
<code>str</code>	778	<code>\l__spintent_spnM_silent_cmd_bool</code>	2069
<code>\l__spintent_spmoney_luaset_print_iso_-</code>	778	<code>__spintent_spnM_spnD_exec:nnnn</code>	2150
<code>str</code>	778	<code>__spintent_spnM_spnD_intent:nn</code>	2115
<code>\l__spintent_spmoney_luaset_sing_str ..</code>	778	<code>__spintent_spnM_spnD_process:nnnn ..</code>	2110
<code>\l__spintent_spmoney_luaset_status_str</code>	778	<code>__spintent_spnM_spnD_result:n</code>	2131
<code>\l__spintent_spmoney_luaset_subplur_str</code>	778	<code>__spintent_spnM_spnD_silent:nn</code>	2123
<code>\l__spintent_spmoney_luaset_subsing_str</code>	778	<code>\l__spintent_spnQ_intent_read_sign_str</code>	2322
<code>\l__spintent_spmoney_luaset_symbol_str</code>	778	<code>\l__spintent_spnQ_read_parens_bool</code>	78
<code>__spintent_spmoney_normalize_key:n ...</code>	813	<code>\l__spintent_spnQ_wrap_parens_l_tl</code>	78, 2322
<code>__spintent_spmoney_parse_and_route:n ..</code>	1054	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	78, 2322
<code>__spintent_spmoney_print_dec_sub_-</code>	1006	<code>\l__spintent_spnnum__notation_sym_tl</code>	35
<code>unit:</code>	1006	<code>\l__spintent_spnnum_intent_read_dec_sep_-</code>	35, 37, 264
<code>__spintent_spmoney_print_int:</code>	895	<code>str</code>	35, 37, 264
<code>__spintent_spmoney_print_sep_con: ...</code>	1006	<code>\l__spintent_spnnum_intent_read_sign_str</code>	35, 264
<code>__spintent_spmoney_print_symbol:</code>	861	<code>\l__spintent_spnnum_notation_sym_tl ...</code>	264
<code>__spintent_spmoney_render_layout: ...</code>	874	<code>__spintent_spnnum_print_integer:</code>	431
<code>__spintent_spmoney_render_number_node:</code>	886	<code>__spintent_spnnum_print_num_start: ...</code>	431
<code>__spintent_spmoney_render_only_integer_-</code>	886	<code>__spintent_spnnum_print_number_core: ..</code>	431
<code>node:</code>	886	<code>__spintent_spnnum_print_part_dec:</code>	410
<code>\l__spintent_spmoney_res_main_voice_str</code>	788	<code>__spintent_spnnum_print_part_period: ..</code>	410
<code>__spintent_spmoney_set_and_print_sub_-</code>	959	<code>__spintent_spnnum_print_period_bar: ...</code>	339
<code>units:</code>	959	<code>\l__spintent_spnnum_read_period_sep_str ..</code>	35, 264
<code>__spintent_spmoney_set_currency:n ...</code>	813	<code>\l__spintent_spnnum_read_terse_bool ..</code>	35, 264
<code>\l__spintent_spmoney_sub_units_bool ...</code>	788	<code>__spintent_spnnum_render_dec:</code>	322
<code>\l__spintent_spmoney_subnum_int</code>	788	<code>__spintent_spnnum_render_decimal_sep: ..</code>	376
<code>__spintent_spmoney_validate_money: ...</code>	845	<code>__spintent_spnnum_render_int:</code>	322
<code>__spintent_spmQ_exec:nnnn</code>	2502	<code>__spintent_spnnum_render_only_period: ..</code>	339
<code>\l__spintent_spmQ_join_word_str ...</code>	79, 2322	<code>__spintent_spnnum_render_part:n</code>	322
<code>__spintent_spmQ_print:nnn</code>	2494	<code>__spintent_spnnum_render_period_sep: ..</code>	339
<code>\l__spintent_spmQ_print_dfrac_bool</code>	80	<code>__spintent_spnz_process:nn</code>	2296
<code>__spintent_spmQ_print_scale_int:</code>	2440	<code>__spintent_spnz_render_parens:</code>	2284
<code>__spintent_spmQ_print_scale_int:n ...</code>	2449	<code>\l__spintent_spPcirc_mode_str</code>	4387
<code>__spintent_spmQ_print_spnZ_dfrac:nn ..</code>	2477	<code>\l__spintent_spPcirc_prefix_tl</code>	4387
<code>__spintent_spmQ_set_join_word:</code>	2431	<code>\l__spintent_spPcirc_print_sym_str ...</code>	4387
<code>__spintent_spmQ_set_wrap_parens_size:</code>	2376	<code>\l__spintent_spPcirc_radio_sty_str ...</code>	4387
<code>__spintent_spmQ_start_wrap_parens: ...</code>	2411	<code>\l__spintent_spPcirc_read_arg_tl</code>	4387
<code>__spintent_spmQ_stop_wrap_parens: ...</code>	2411	<code>\l__spintent_spPcirc_read_sty_str</code>	4387
<code>\l__spintent_spmQ_use_spnZ_bool</code>	80	<code>\l__spintent_spPcirc_sub_index_str ...</code>	4387
<code>\l__spintent_spm_side_mode_str</code>	2962	<code>\l__spintent_spPfig_mode_str</code>	3706
<code>\l__spintent_spm_side_prefix_tl</code>	2962	<code>\l__spintent_spPfig_prefix_tl</code>	3706
<code>\l__spintent_spm_side_read_arg_tl</code>	2962	<code>\l__spintent_spPfig_print_sym_str ...</code>	3706
<code>\l__spintent_spm_side_read_sty_str</code>	2962	<code>\l__spintent_spPfig_read_arg_tl</code>	3706
<code>\l__spintent_spm_sym_prefix_tl</code>	1702	<code>__spintent_spPoint_exec:nn</code>	2629
<code>\l__spintent_spm_sym_read_sym_tl</code>	1702	<code>\l__spintent_spPoint_intent_arg_tl ...</code>	2536
<code>\l__spintent_spm_sym_suffix_tl</code>	1702	<code>\l__spintent_spPoint_kv_tl</code>	2536
<code>\l__spintent_spm_sym_symbol_tl</code>	1702	<code>__spintent_spPoint_mathml:</code>	2607
<code>__spintent_spmul_process:nnn</code>	1897		
<code>__spintent_spmul_render_op:nn</code>	1876		
<code>__spintent_spmul_render_parens:nn ...</code>	1885		

<code>\l__spintent_spPoint_mode_str</code>	2536
<code>\l__spintent_spPoint_prefix_tl</code>	2536
<code>__spintent_spPoint_print:n</code>	2579
<code>__spintent_spPoly_build_concept:</code>	3627
<code>__spintent_spPoly_build_refs:</code>	3614
<code>__spintent_spPoly_exec:nn</code>	3690
<code>\l__spintent_spPoly_intent_arg_tl</code>	3602
<code>__spintent_spPoly_mathml:</code>	3670
<code>\l__spintent_spPoly_mode_str</code>	3559
<code>__spintent_spPoly_points_loop:</code>	3648
<code>\l__spintent_spPoly_prefix_tl</code>	3559
<code>__spintent_spPoly_process:n</code>	3660
<code>\l__spintent_spPoly_pt_idx_int</code>	3602
<code>\l__spintent_spPoly_pt_seq</code>	3602
<code>\l__spintent_spPoly_refs_tl</code>	3602
<code>__spintent_spPoly_sym:</code>	3606
<code>__spintent_sprayo_exec:nn</code>	2947
<code>\l__spintent_sprayo_intent_arg_tl</code>	2887
<code>__spintent_sprayo_mathml:</code>	2900
<code>\l__spintent_sprayo_mode_str</code>	2833
<code>\l__spintent_sprayo_prefix_tl</code>	2833
<code>__spintent_sprayo_print:n</code>	2891
<code>\l__spintent_sprayo_pt_idx_int</code>	2887
<code>\l__spintent_sprayo_pt_seq</code>	2887
<code>\l__spintent_sprayo_read_sty_str</code>	2833
<code>\l__spintent_sprayo_refs_tl</code>	2887
<code>__spintent_spread_exec:nn</code>	1613
<code>\l__spintent_spread_intent_tl</code>	1612
<code>__spintent_sprecta_exec:nn</code>	2818
<code>\l__spintent_sprecta_intent_arg_tl</code> ...	2676
<code>\l__spintent_sprecta_kv_tl</code>	2676
<code>__spintent_sprecta_mathml:</code>	2730
<code>\l__spintent_sprecta_mode_str</code>	2676
<code>\l__spintent_sprecta_prefix_tl</code>	2676
<code>__spintent_sprecta_print:n</code>	2721
<code>\l__spintent_sprecta_pt_idx_int</code>	2676
<code>\l__spintent_sprecta_pt_seq</code>	2676
<code>\l__spintent_sprecta_refs_tl</code>	2676
<code>__spintent_spshort_execute_layout:</code> ...	1162
<code>\l__spintent_spshort_luaset_base_str</code> ..	1079
<code>\l__spintent_spshort_luaset_expanded_- str</code>	1079
<code>\l__spintent_spshort_luaset_layout_str</code>	1079
<code>\l__spintent_spshort_luaset_output_str</code>	1079
<code>\l__spintent_spshort_luaset_status_str</code>	1079
<code>\l__spintent_spshort_luaset_suffix_str</code>	1079
<code>__spintent_spshort_print:nn</code>	1115
<code>__spintent_spshort_process_and_- render:n</code>	1127
<code>__spintent_spshort_render_bypass:n</code> ...	1138
<code>__spintent_spshort_render_error:n</code> ...	1138
<code>__spintent_spshort_render_fallback:</code> ..	1138
<code>__spintent_spshort_render_success:</code> ...	1138
<code>__spintent_spshort_tag_span:nn</code>	1088
<code>__spintent_spside_exec:nnn</code>	3144
<code>\l__spintent_spside_intent_arg_tl</code>	3036
<code>\l__spintent_spside_intent_tl</code>	3036
<code>__spintent_spside_mathml:n</code>	3069
<code>\l__spintent_spside_mode_str</code>	2962
<code>__spintent_spside_points_loop:n</code>	3132
<code>\l__spintent_spside_prefix_tl</code>	2962
<code>__spintent_spside_process:nn</code>	3041
<code>\l__spintent_spside_pt_idx_int</code>	3036
<code>\l__spintent_spside_pt_seq</code>	3036
<code>\l__spintent_spside_read_arg_tl</code>	2962
<code>\l__spintent_spside_read_sty_str</code>	2962
<code>\l__spintent_spside_refs_tl</code>	3036
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1219
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1219
<code>\l__spintent_spsiglo_luaset_roman_min_- str</code>	1219
<code>\l__spintent_spsiglo_print_era_str</code> ...	1219
<code>__spintent_spsiglo_print_math:n</code>	1303
<code>__spintent_spsiglo_print_text:n</code>	1303
<code>__spintent_spsiglo_render_simple:nn</code> ..	1254
<code>__spintent_spsiglo_render_with_- era:nnn</code>	1254
<code>__spintent_spsiglo_tag_span:nn</code>	1290
<code>__spintent_sptime_build_seconds:nn</code> ...	1383
<code>__spintent_sptime_build_tree:</code>	58, 1401
<code>\l__spintent_sptime_luaset_base_str</code> ...	1369
<code>\l__spintent_sptime_luaset_error_str</code> ..	1369
<code>\l__spintent_sptime_luaset_has_seconds_- str</code>	1369
<code>\l__spintent_sptime_luaset_is_fraction_- str</code>	1369
<code>\l__spintent_sptime_luaset_seconds_str</code>	1369
<code>\l__spintent_spunit_exp_seq</code>	525
<code>\l__spintent_spunit_exponent_tl</code>	525
<code>__spintent_spunit_format_canonical:</code> ..	588
<code>\l__spintent_spunit_is_mult_bool</code>	525
<code>\l__spintent_spunit_luaset_canonical_- str</code>	518
<code>\l__spintent_spunit_luaset_compact_exp_- str</code>	518
<code>\l__spintent_spunit_luaset_is_compact_- str</code>	518
<code>\l__spintent_spunit_luaset_is_- sexagesimal_str</code>	518
<code>\l__spintent_spunit_luaset_lookup_status_- str</code>	518
<code>\l__spintent_spunit_luaset_read_str</code> ...	518
<code>\l__spintent_spunit_luaset_status_str</code> .	518
<code>\l__spintent_spunit_mult_seq</code>	525
<code>__spintent_spunit_process_base_exp:n</code> .	588
<code>__spintent_spunit_process_mult:n</code>	648
<code>\l__spintent_spunit_read_cdot_bool</code> ...	525
<code>\l__spintent_spunit_read_slash_str</code> ...	525
<code>__spintent_spunit_safe_exec:n</code>	667
<code>\l__spintent_spunit_unit_name_tl</code>	525
<code>\l__spintent_spusep_arg_tl</code>	60, 61, 1498
<code>__spintent_spusep_convert:n</code>	1560
<code>\l__spintent_spusep_left_tl</code>	60, 61, 1498
<code>\l__spintent_spusep_right_tl</code>	60, 61, 1498
<code>__spintent_spusep_set_parens:nn</code>	1549
<code>\l__spintent_spusep_silent_bool</code>	61
<code>__spintent_unknown_keys:n</code>	216
<code>__spintent_unknown_keys:nn</code>	216
<code>\spmangle</code>	20, 94, 3380
<code>spmangle-invalid-arg</code>	3174
<code>\spmarc</code>	21, 99, 3539
<code>spmarc-invalid-points</code>	3405
<code>\spmcd</code>	15, 68, 2049

